Lab 1: Comparison of Virtual Machines (VMs) and Containers

1. Objective

The primary goals of this experiment are:

- To understand the conceptual and practical differences between Virtual Machines (VMs) and Containers.
- To install and configure an Ubuntu-based Nginx web server using both VirtualBox/Vagrant and Docker inside WSL.
- To compare resource utilization, performance, and operational characteristics of both environments.

2. Hardware and Software Requirements

Hardware

- 64-bit system with virtualization enabled in BIOS
- Minimum 8 GB RAM (4 GB acceptable)
- Internet connection

Software

- Oracle VirtualBox
- Vagrant
- Windows Subsystem for Linux (WSL 2) with Ubuntu
- Docker Engine (docker.io)

3. Theory

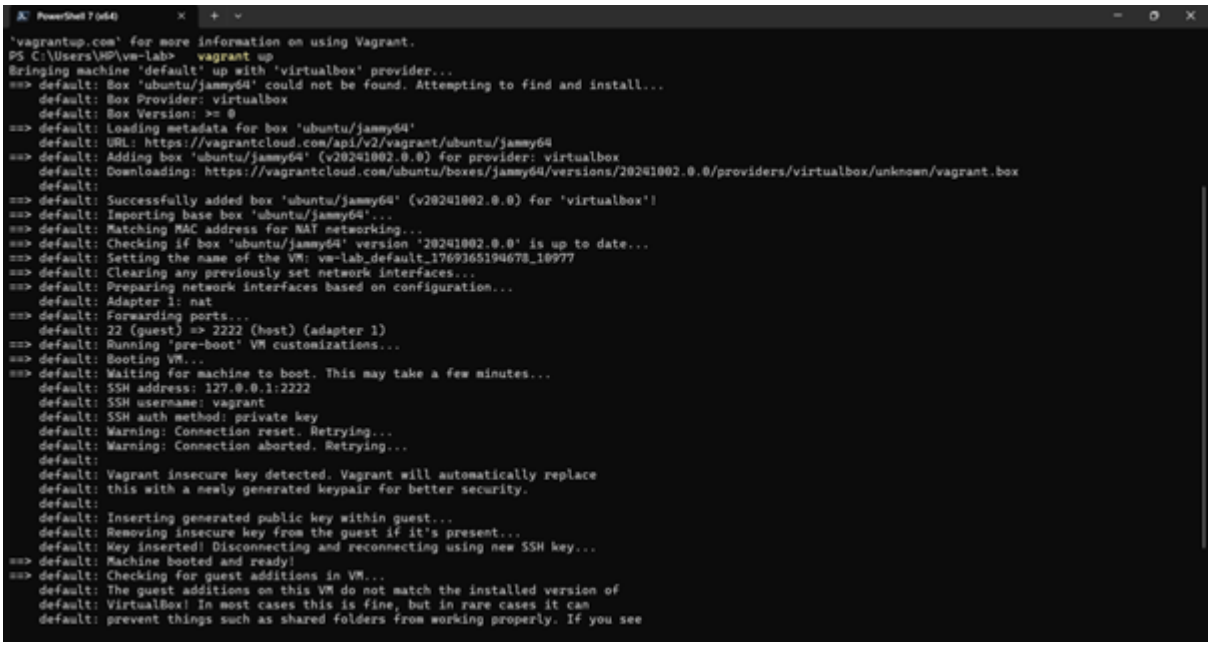
Feature	Virtual Machine (VM)	Container
Virtualization Level	Emulates complete hardware and kernel	Virtualizes at OS level, shares host kernel
Isolation	Strong (Full OS isolation)	Moderate (Process-level isolation)
Resource Usage	Higher (Dedicated RAM/CPU for OS)	Lightweight and efficient
Startup Time	Slower (Seconds/Minutes)	Fast (Milliseconds/Seconds)

4. Experiment Setup – Part A: Virtual Machine

Step 1: Initialization and Deployment

Using Vagrant, an Ubuntu VM was initialized and started.

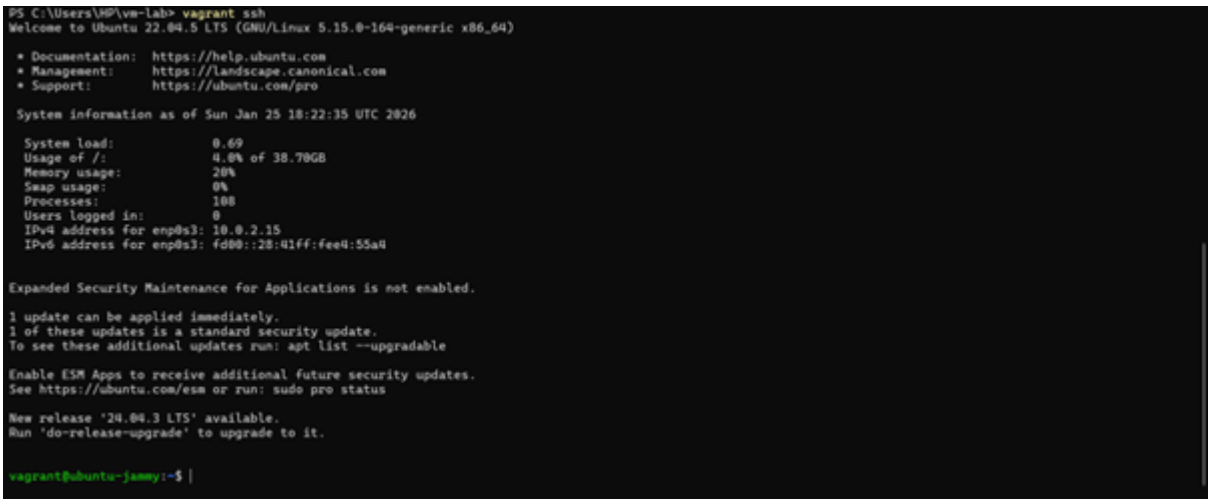
```
vagrant init ubuntu/jammy64
vagrant up
```



Observation:
The system downloads the Ubuntu Jammy base box and configures the VirtualBox provider. Port forwarding (2222 → 22) is established.

Step 2: Accessing the VM (SSH)

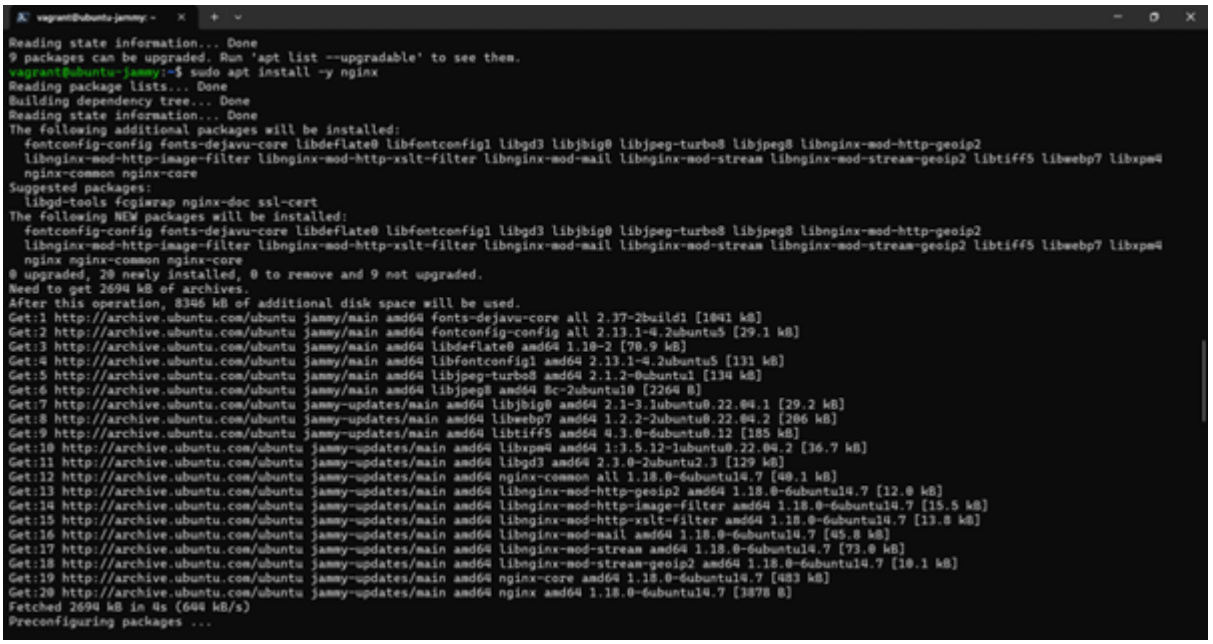
```
vagrant ssh
```



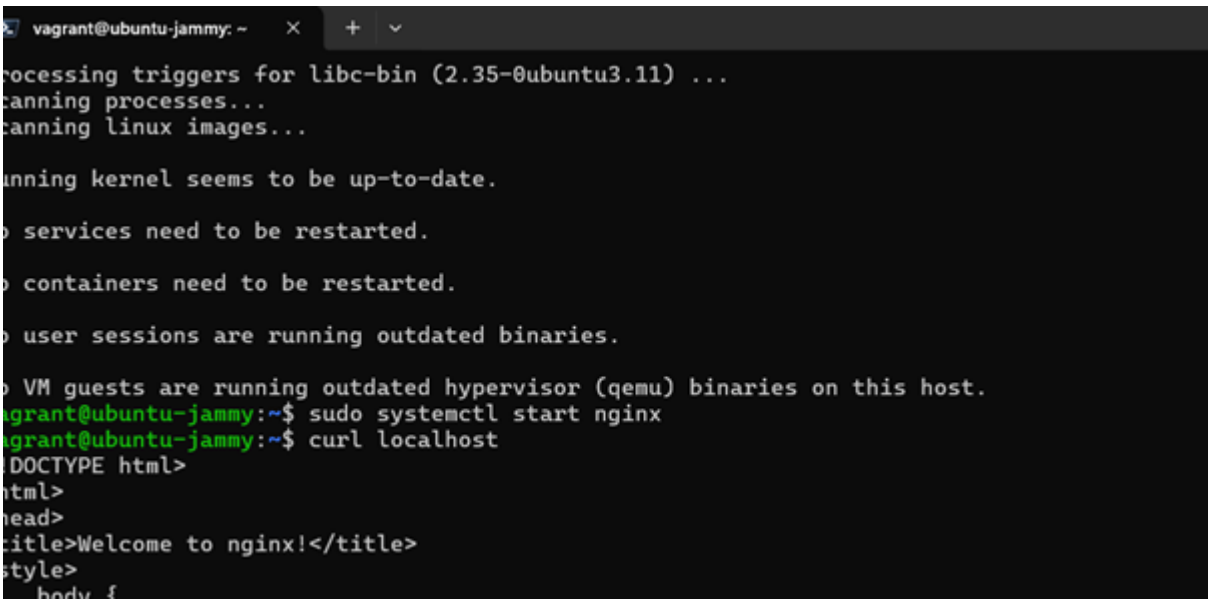
Observation:
Successful login to Ubuntu 22.04.5 LTS environment.

Step 3: Installing Nginx

```
sudo apt update
sudo apt install -y nginx
```



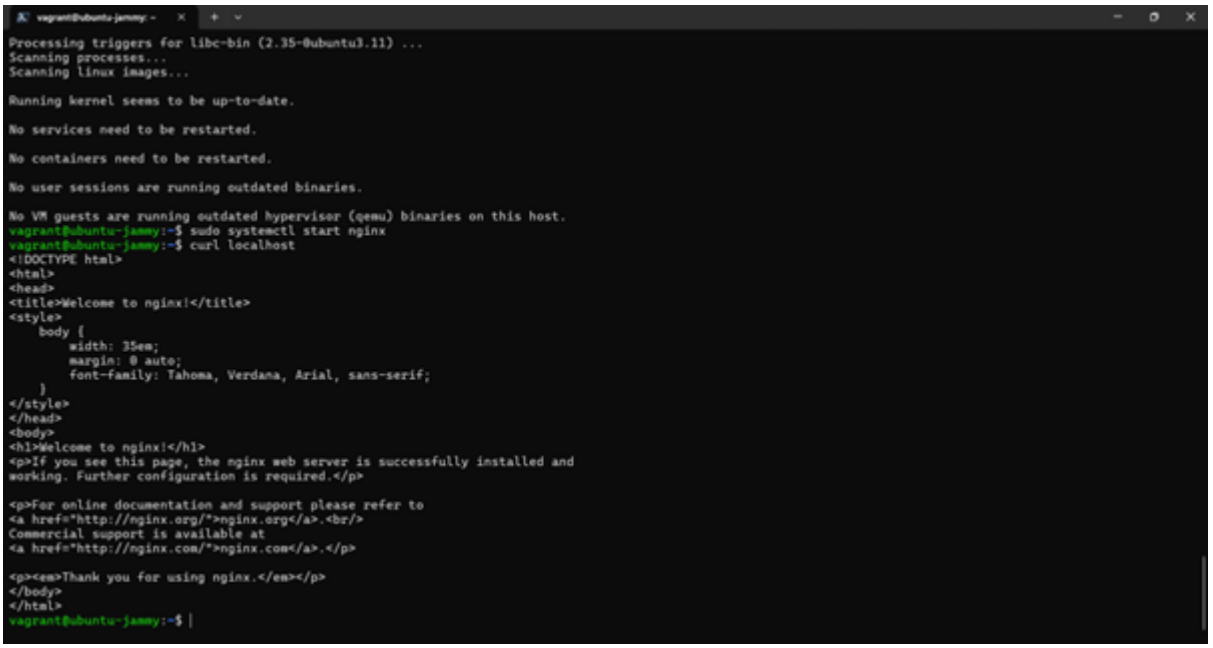
Observation:
The apt package manager installs required dependencies. This process is slower compared to Docker because a full OS environment is involved.



Observation:
Installation completes successfully with triggers processed for man-db and ufw.

Step 4: Verification Inside VM

```
curl localhost
```



Observation:
The command returns the full HTML source of the “Welcome to nginx!” page.

---

5. Experiment Setup – Part B: Containers (Docker)

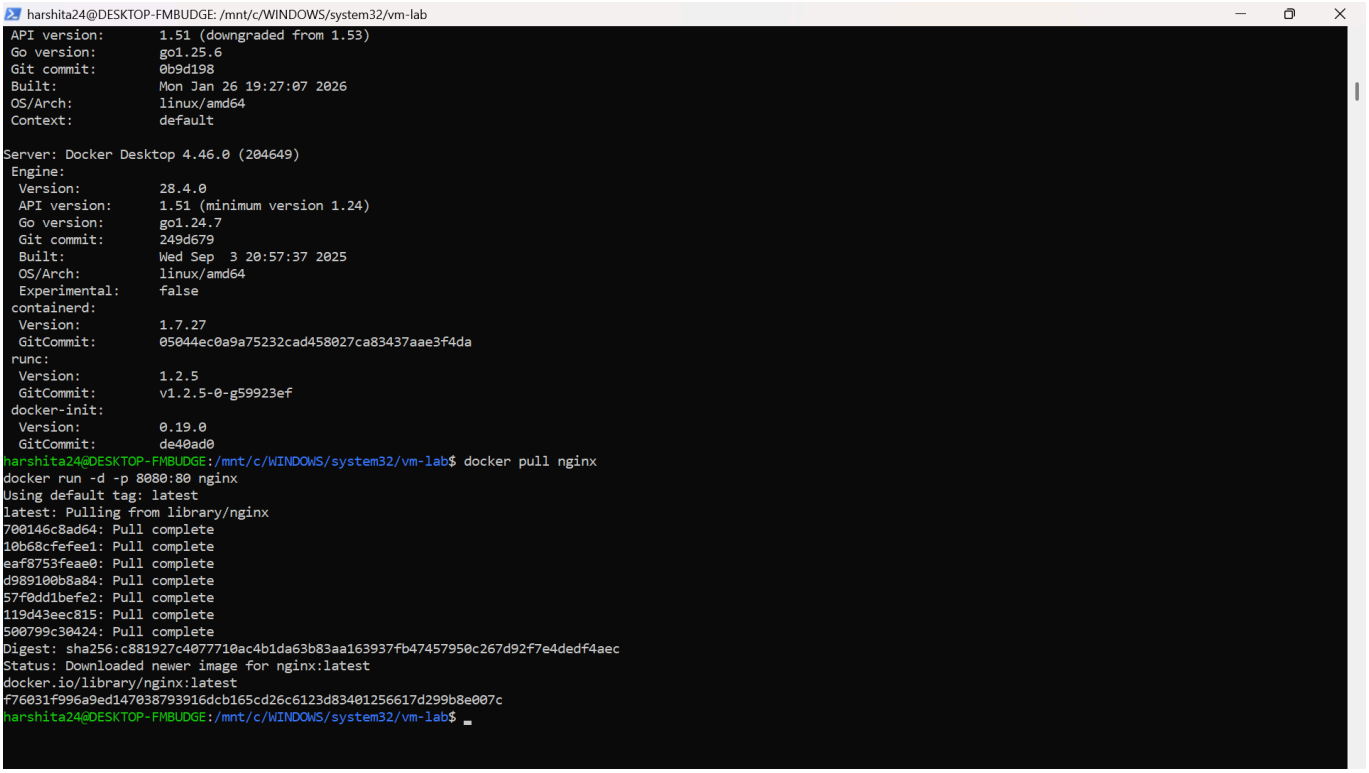
Step 1: Running the Container

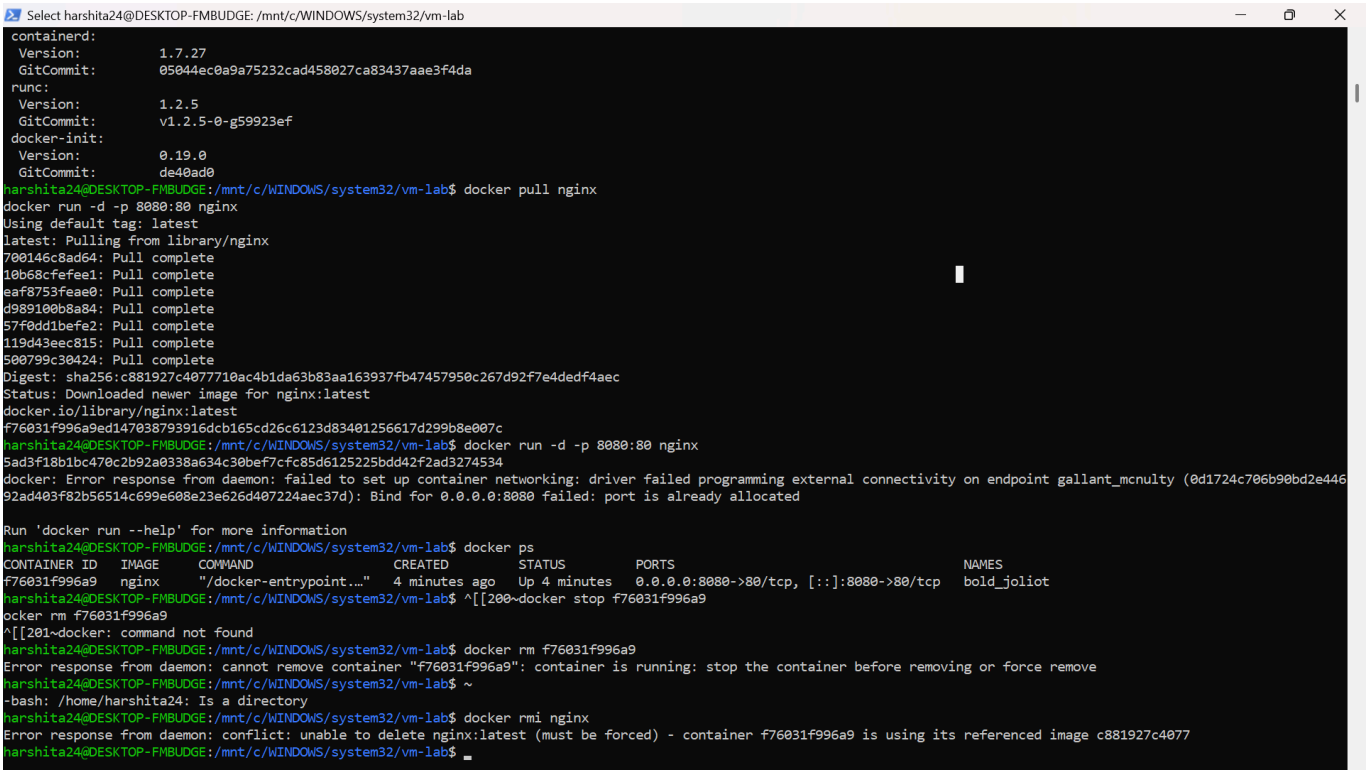
```
docker run -dp 8080:80 --name nginx-container nginx
```

Observation:
Docker pulls image layers and starts the container almost instantly.

Step 2: Verification

```
curl localhost:8080
```





Observation:

The Nginx “Welcome” page is successfully displayed on port 8080.

---

6. Resource Utilization & Comparison

A. Boot Time Analysis

```
systemd-analyze
```

Observation (VM):
The VM took 36.819 seconds to finish startup (6.9s kernel + 29.8s userspace).

Observation (Container):
The container started in less than 1 second.

B. Process Overhead & Isolation

```
htop
```

Observation (VM):
Multiple background services such as systemd, snapd, rsyslogd, polkitd, and sshd are running.

```
docker stats
```

Observation (Container):
The container runs only the Nginx process and minimal dependencies.

C. Memory Usage

```
free -h
```

Observation (VM):
957 Mi total allocated, with ~196 Mi used by OS services.

Observation (Container):
Container consumes approximately 13.22 MiB of RAM.

Comparison Summary Table

Parameter	Virtual Machine (VM)	Container (Docker)
Boot Time	~36.8 seconds	< 1 second
RAM Usage	~196 MiB	~13.22 MiB
Background Processes	High	Low
Disk Usage	Larger	Smaller

---

7. Conclusion

This experiment demonstrates that Containers are significantly more lightweight than Virtual Machines.

- **Speed:** Docker containers start instantly, while VMs require ~37 seconds to boot.
- **Efficiency:** VMs consume ~200MB RAM for OS overhead, whereas containers use ~13MB.
- **Complexity:** VMs run a full operating system stack, creating additional overhead.

Final Conclusion:
Containers are ideal for microservices and rapid scaling, while Virtual Machines are better suited for complete OS isolation and hardware-level simulation.

Lab 2: Docker Installation, Configuration, and Running Images

1. Objective

The objectives of this experiment are:

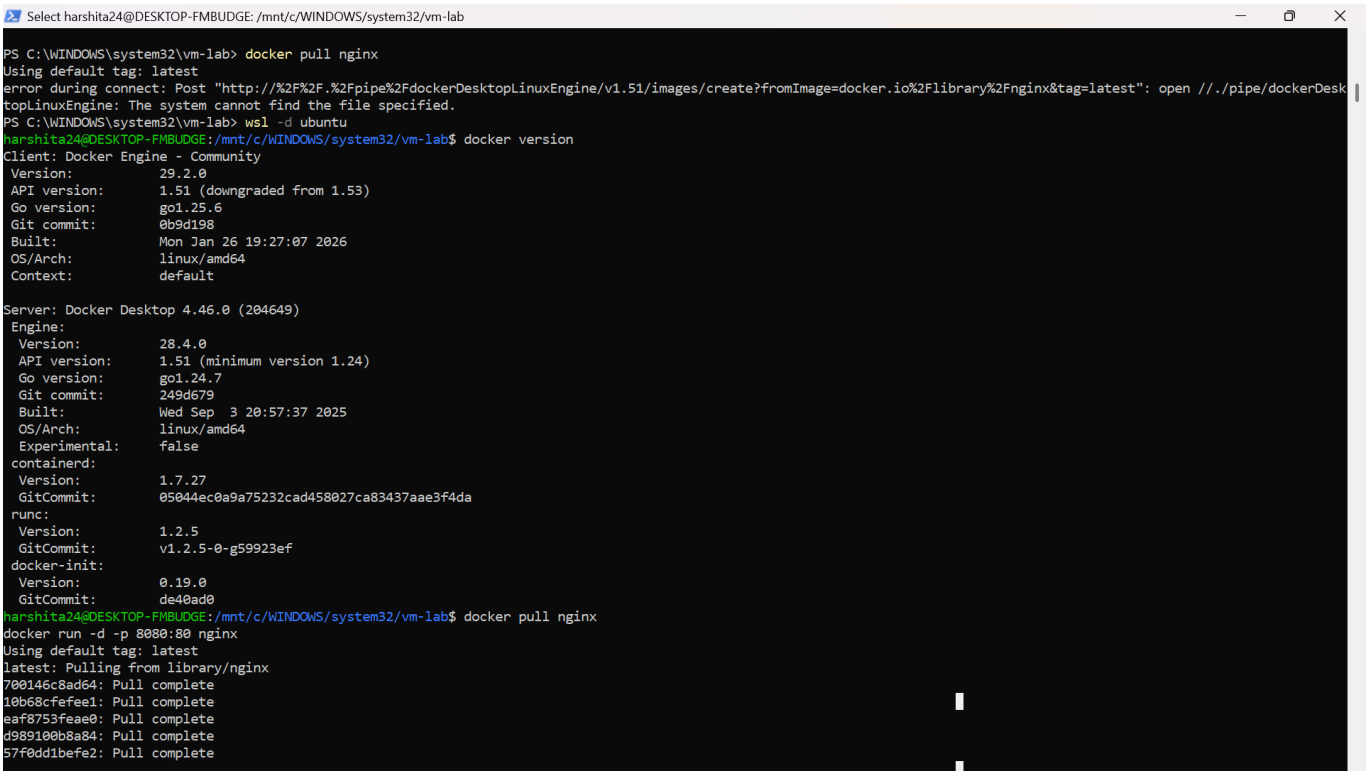
- To pull Docker images from Docker Hub
- To run containers using Docker
- To manage container lifecycle (start, stop, remove)
- To remove Docker images

2. Procedure

Step 1: Pull Docker Image

The nginx image was pulled from Docker Hub using the following command:

```
docker pull nginx
```

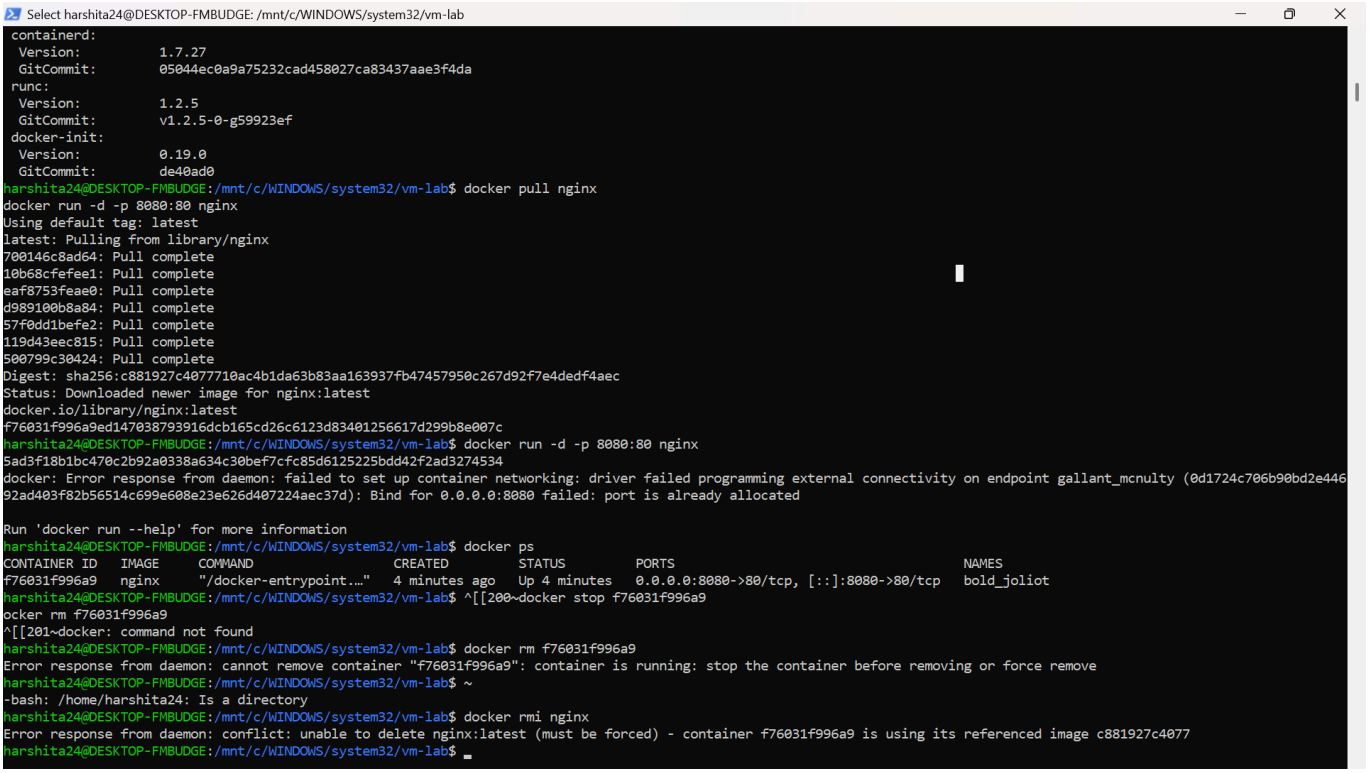


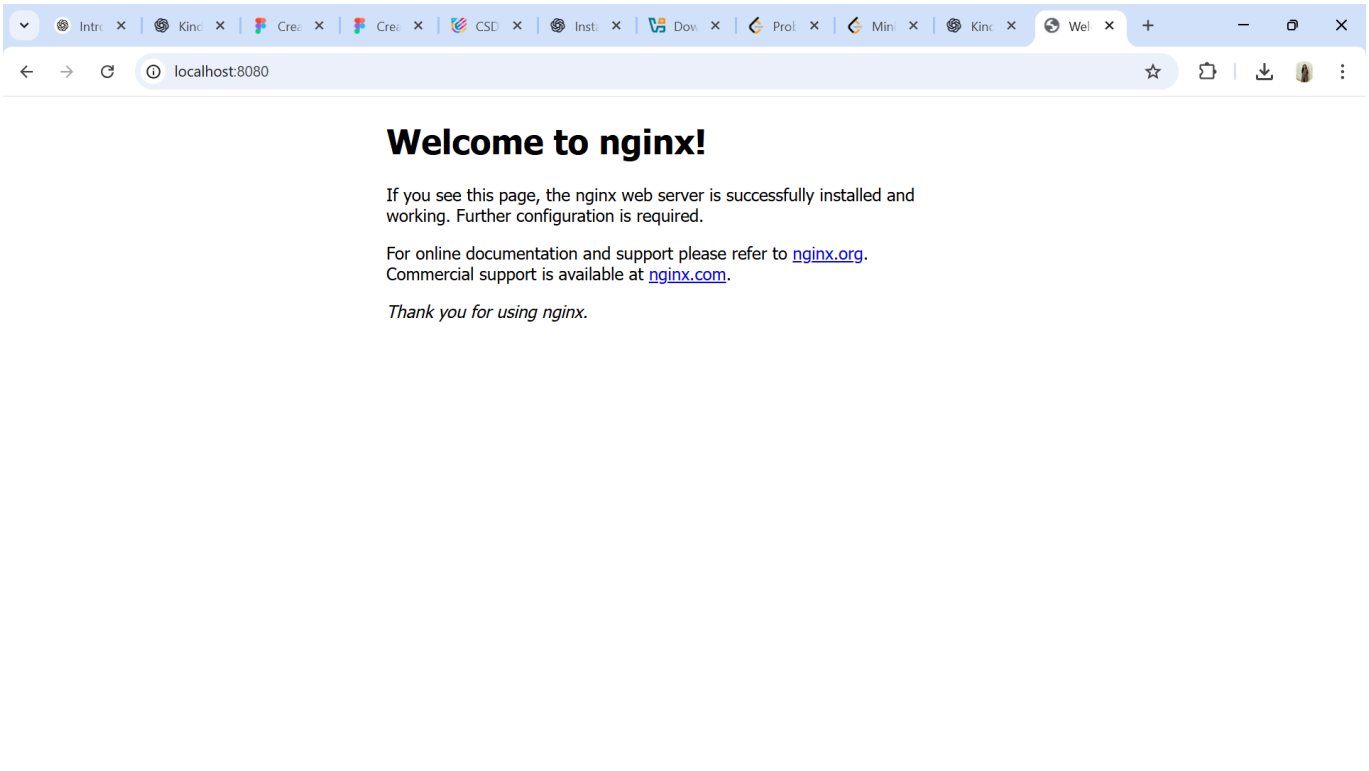
Observation:
Docker successfully downloaded the nginx image layers from Docker Hub and stored them locally.

Step 2: Run Container with Port Mapping

A container was created and run in detached mode with port mapping.

```
docker run -d -p 8080:80 nginx
```





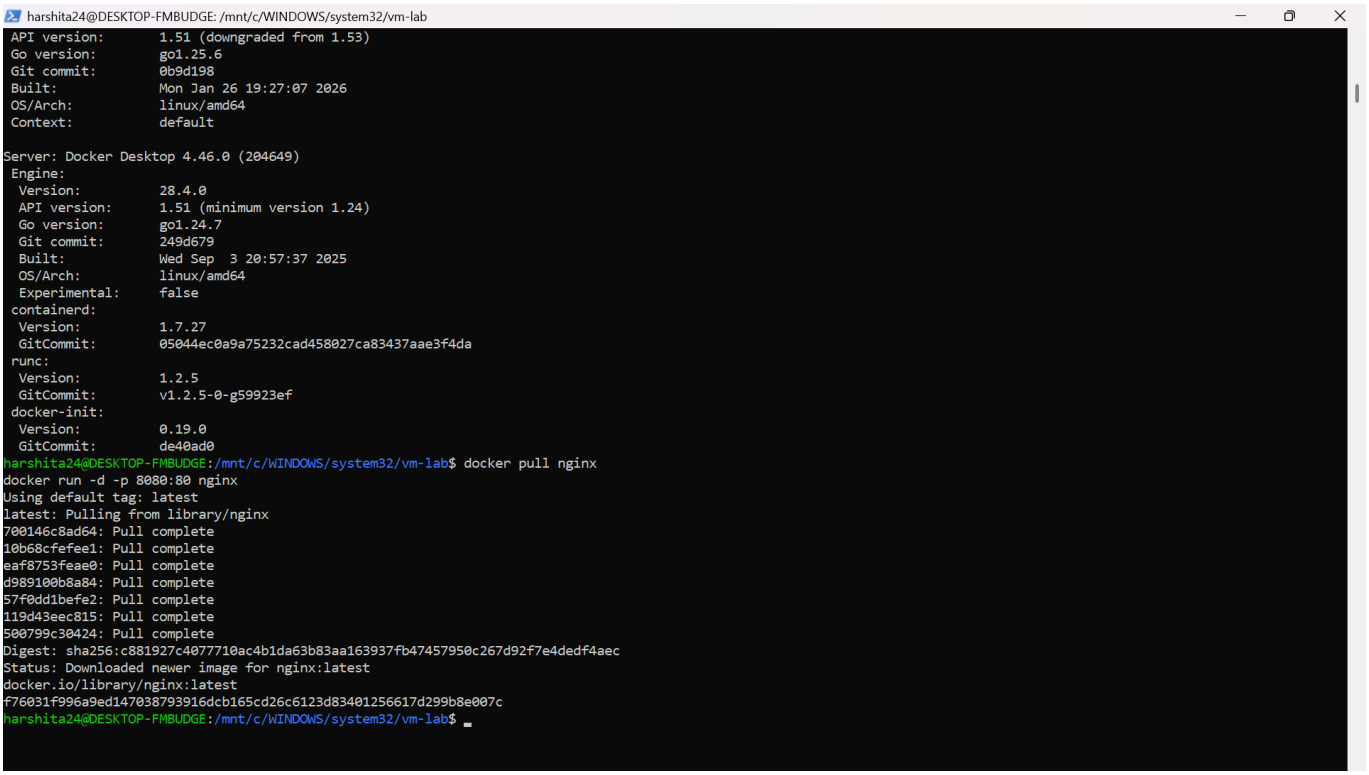
Observation:

The container started successfully in detached mode. Port 8080 on the host machine was mapped to port 80 inside the container.

Step 3: Verify Running Containers

To verify the running containers, the following command was used:

```
docker ps
```



Observation:
The nginx container was listed as running with its container ID, image name, and mapped ports.

Step 4: Stop and Remove Container

To stop the running container:

```
docker stop <container_id>
```

To remove the container:

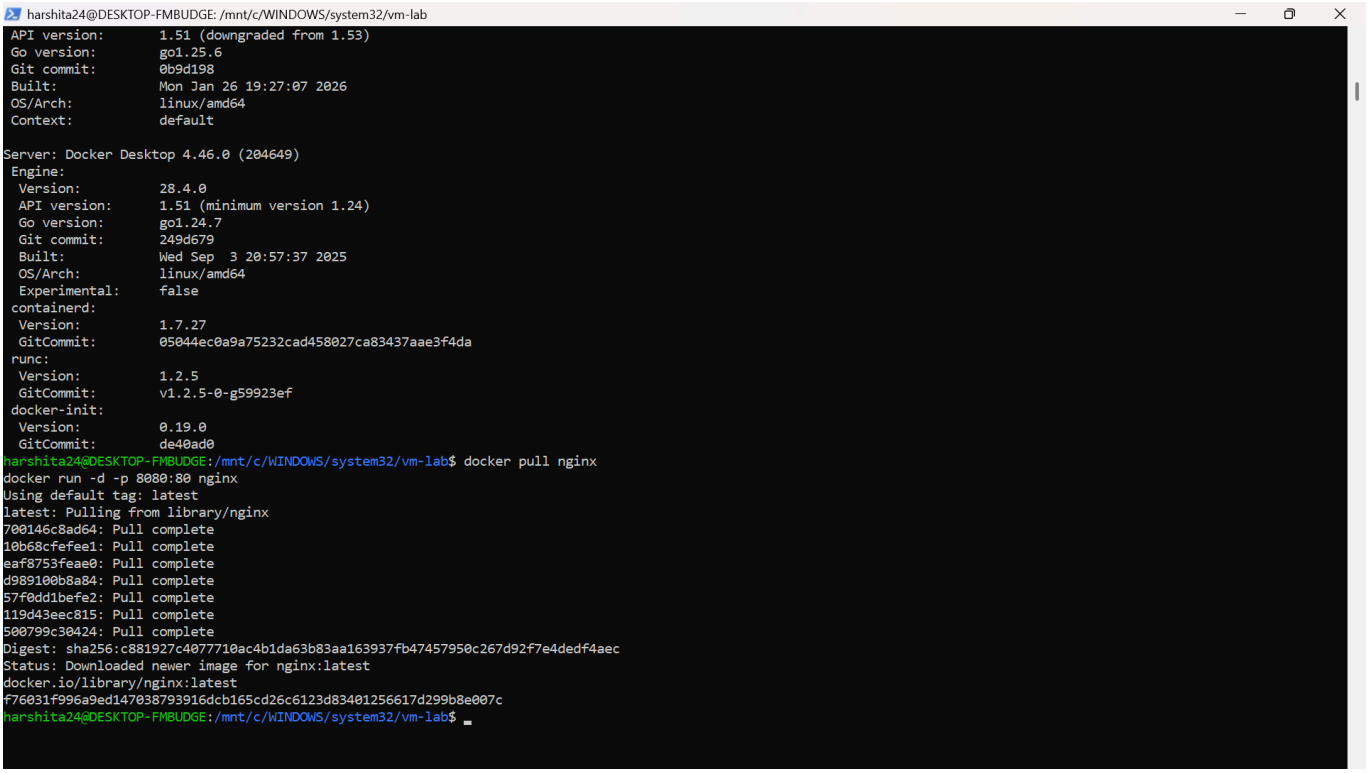
```
docker rm <container_id>
```

Observation:
The container was successfully stopped and removed from the system.

Step 5: Remove Docker Image

To remove the nginx image:

```
docker rmi nginx
```



Observation:

The nginx image was removed from the local Docker image repository.

---

3. Result

Docker images were successfully pulled from Docker Hub.
Containers were created, executed, verified, stopped, and removed.
The container lifecycle management commands were performed successfully.

Lab 3: Deploying NGINX Using Different Base Images and Comparing Image Layers

---

Lab Objectives

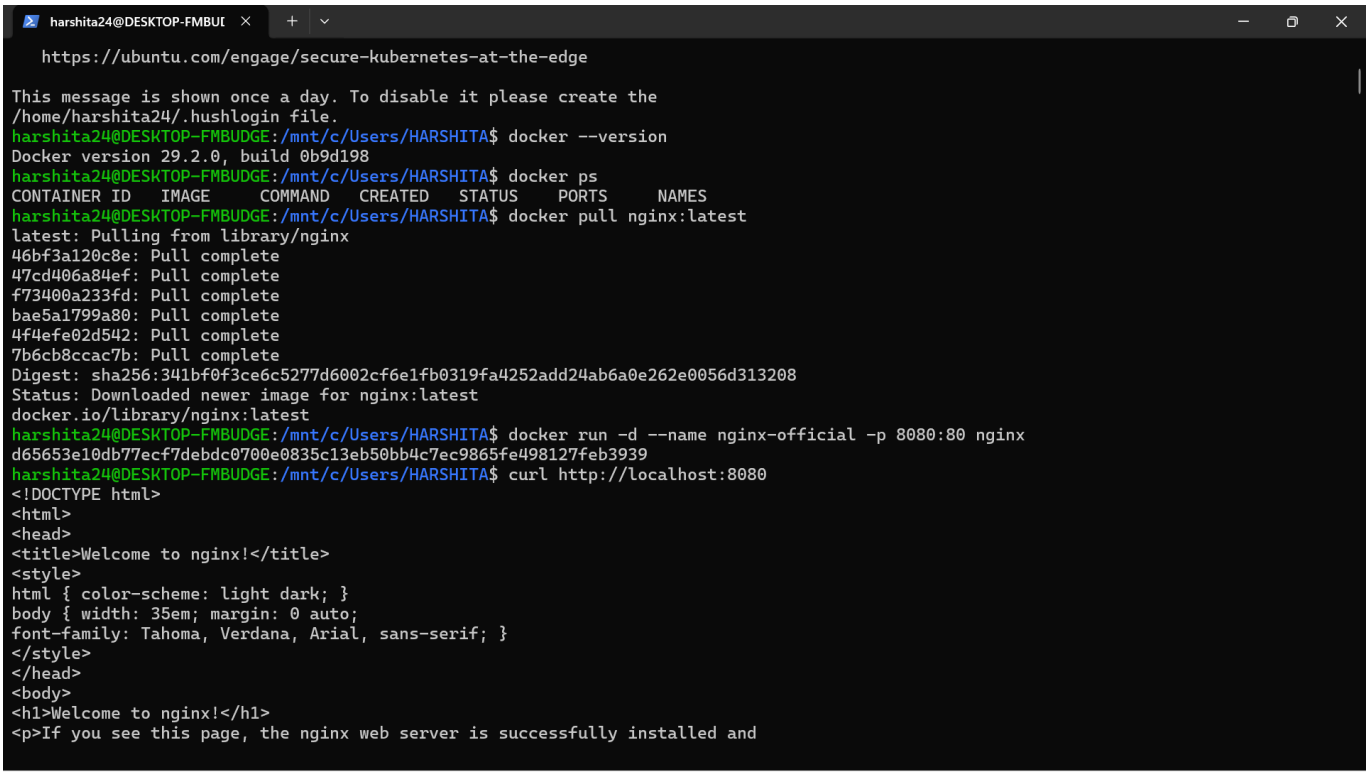
After completing this lab, students will be able to:

- Deploy NGINX using:
  - Official NGINX image
  - Ubuntu-based image
  - Alpine-based image
- Understand Docker image layers and size differences
- Compare performance, security, and use-cases
- Explain real-world use of NGINX in containerized systems

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

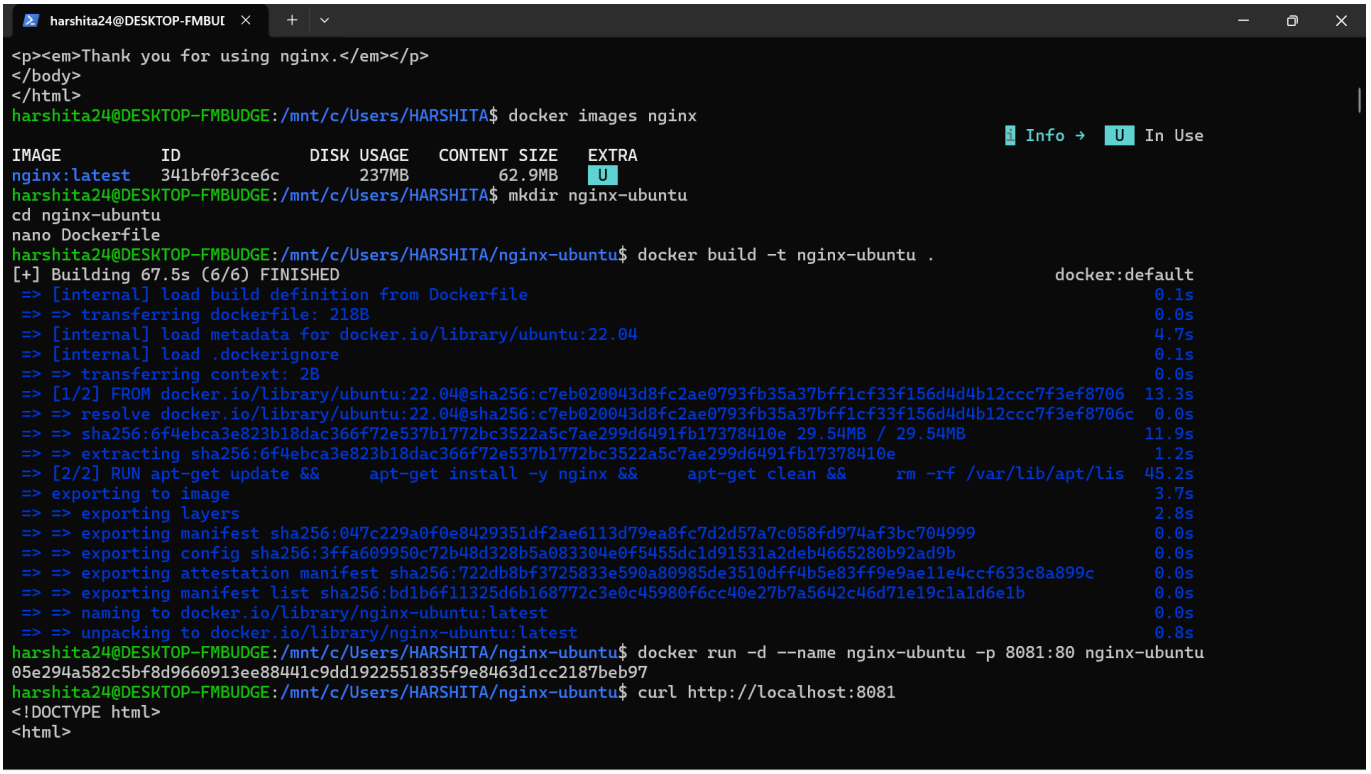
Step 1: Pull the Image

```
docker pull nginx:latest
```



Step 2: Run the Container

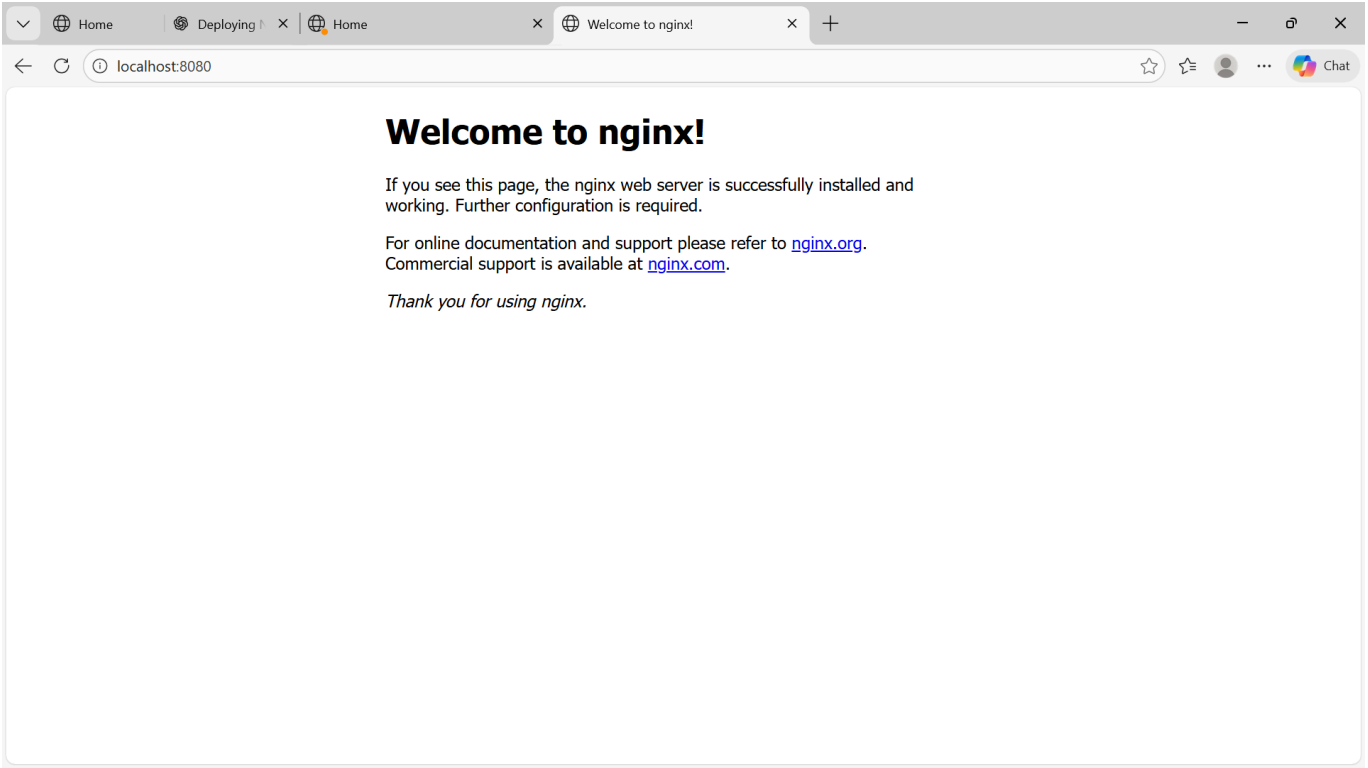
```
docker run -d --name nginx-official -p 8080:80 nginx
```



Step 3: Verify

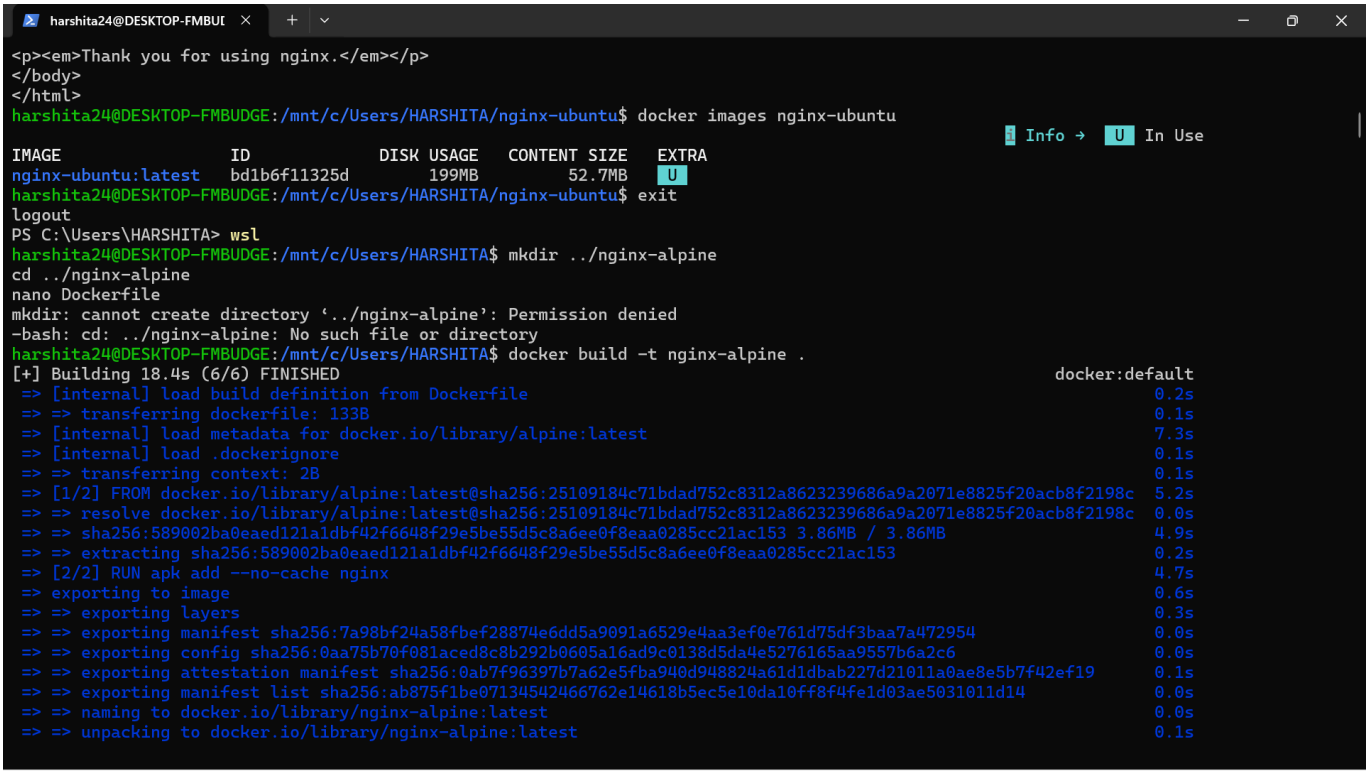
```
curl http://localhost:8080
```

You should see the NGINX welcome page.



Key Observations

```
docker images nginx
```



- Image is pre-optimized
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

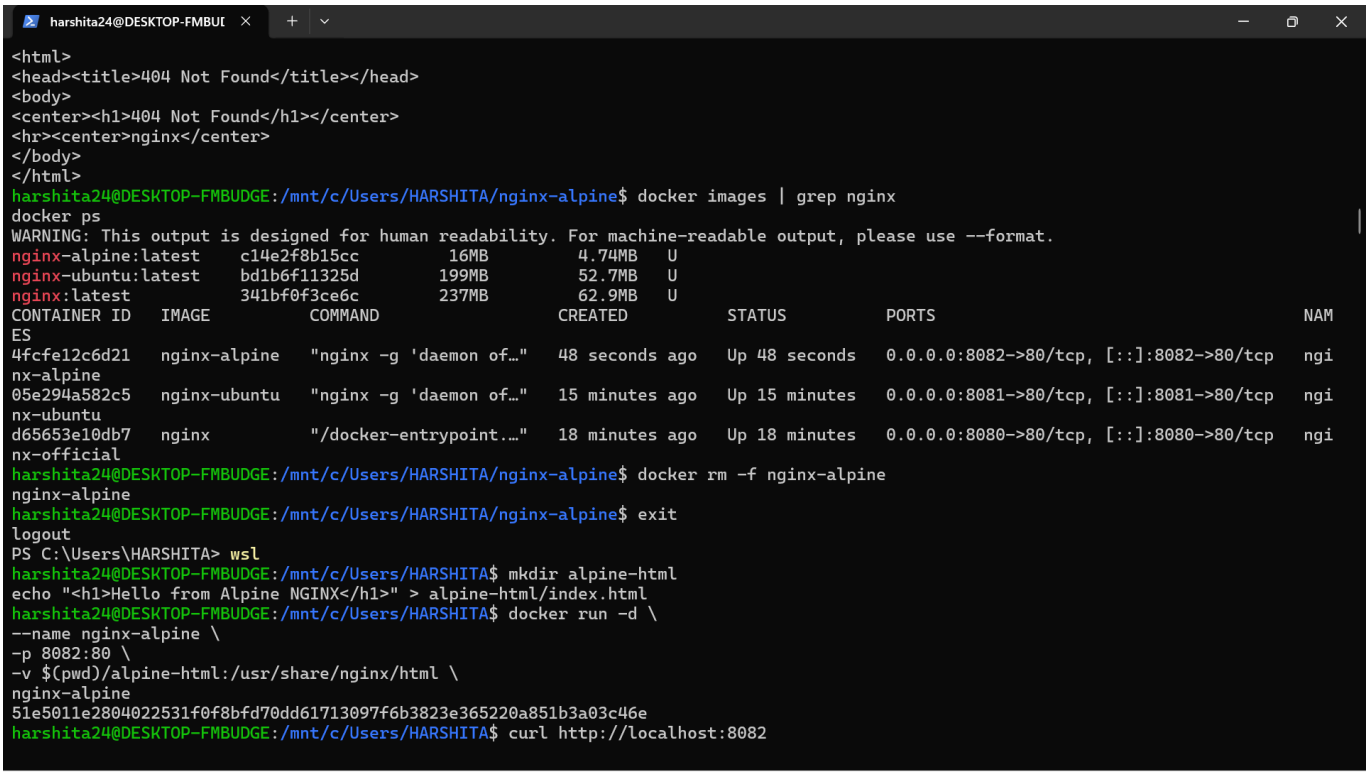
Create a file named `Dockerfile` and add:

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```



Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

Observations

```
docker images nginx-ubuntu
```

- Much larger image size
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

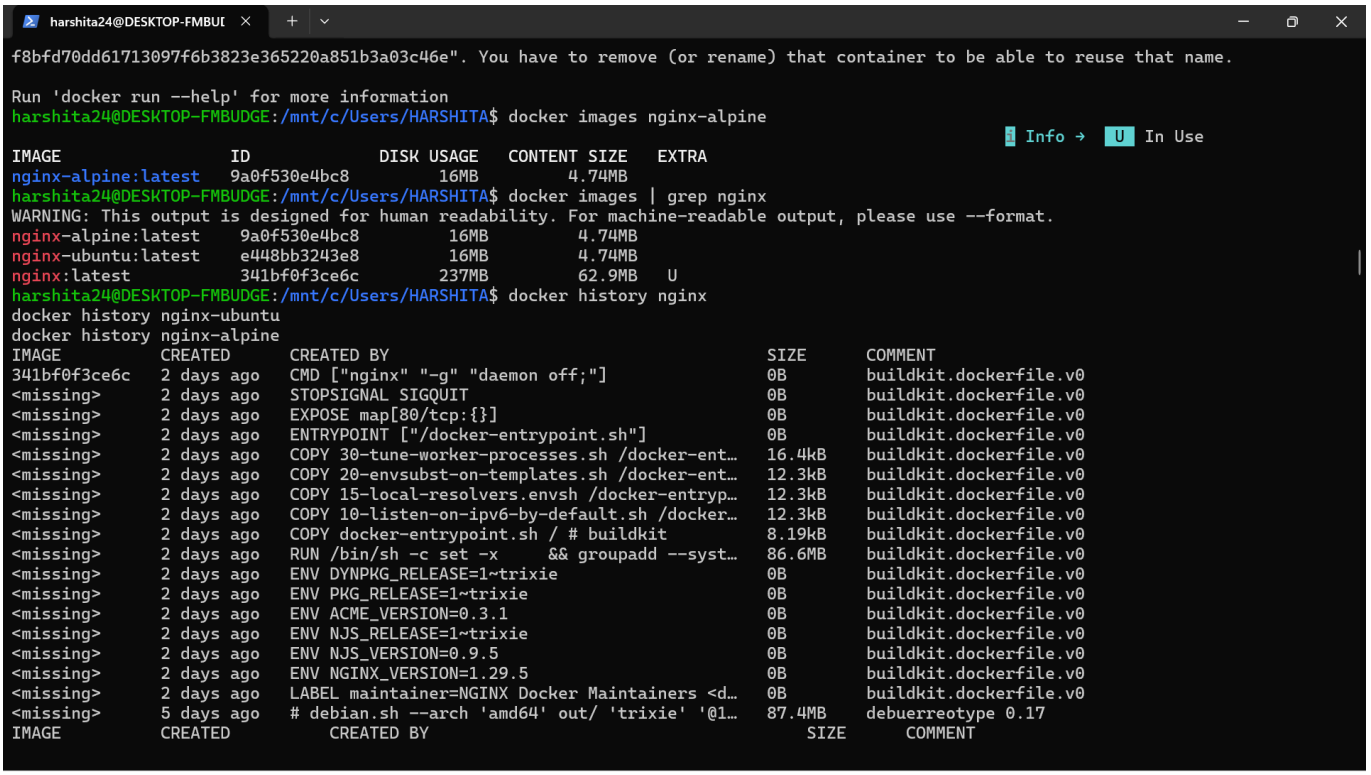
```
docker build -t nginx-alpine .
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

Observations

```
docker images nginx-alpine
```



- Extremely small image
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

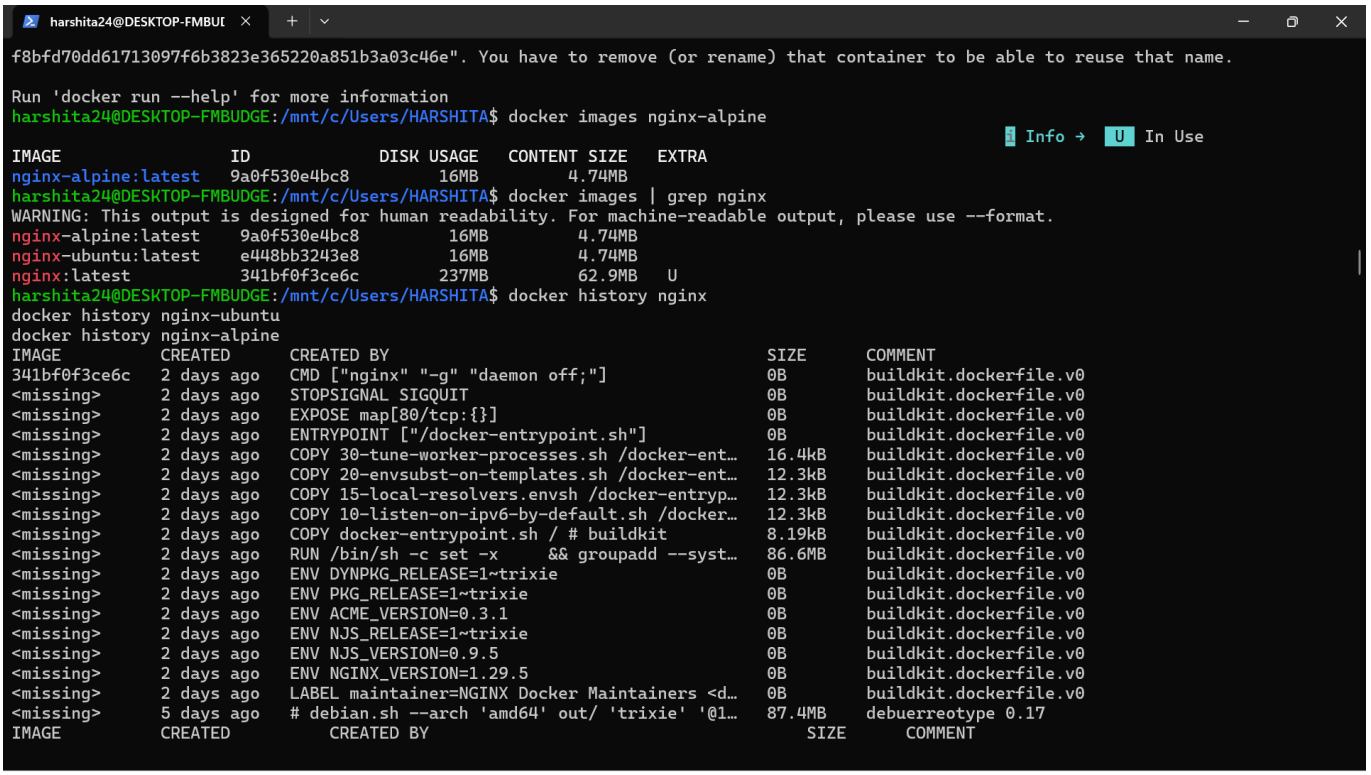
```
docker images | grep nginx
```

Typical Size Comparison

Image Type	Approx Size
nginx:latest	~140 MB
nginx-ubuntu	~220+ MB
nginx-alpine	~25-30 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```



Observations

- Ubuntu image has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Task Using NGINX

Task 1: Serve Custom HTML Page

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

Run container with volume mounting:

```
docker run -d \
-p 8083:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx
```

Result

NGINX was successfully deployed using three different base images.
Image sizes, layers, and performance characteristics were compared.
Custom HTML content was served using volume mounting.

Experiment 3: Deploying Web Applications with Docker

In this experiment, we deploy a sample web application using Docker. This includes writing a custom Dockerfile, building a Docker image, and running the application inside a containerized environment.

This experiment demonstrates the complete lifecycle of containerizing and deploying a web application.

Objectives

By completing this experiment, we will:

1. Prepare a simple web application for containerization.
2. Write a Dockerfile defining the application environment.
3. Build a custom Docker image.
4. Run the web app inside a Docker container.
5. Verify that the application works correctly in the container.

Step 1: Choose a Sample Application

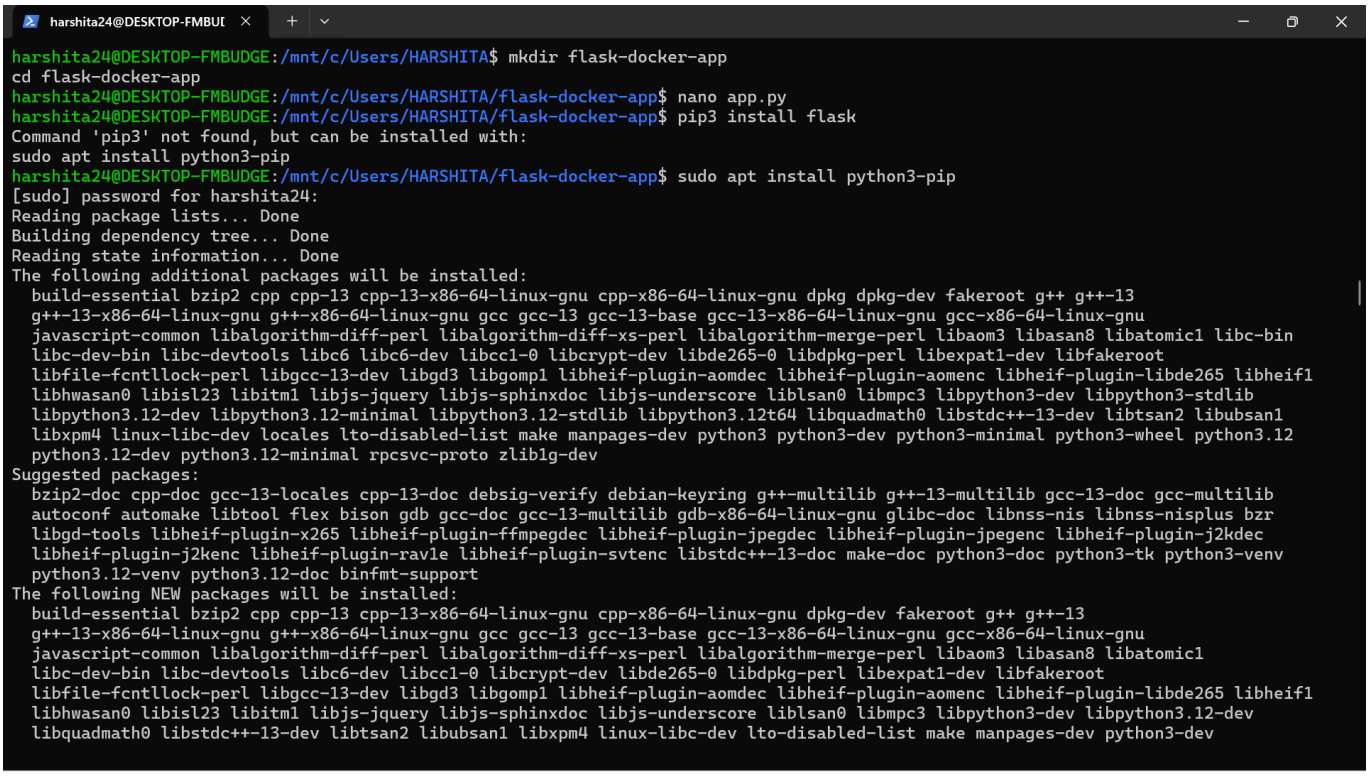
We selected a lightweight Python Flask web application.

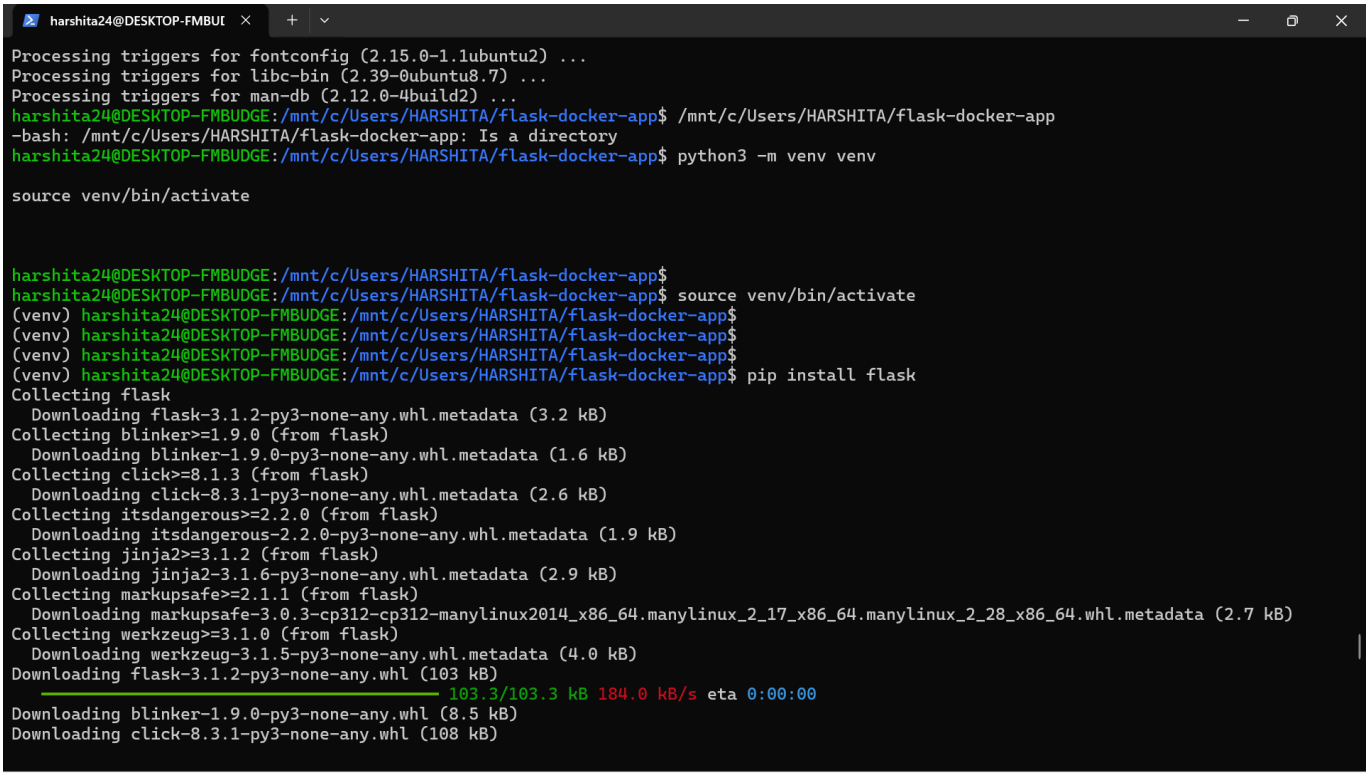
Sample Flask Application (app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, Docker!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```





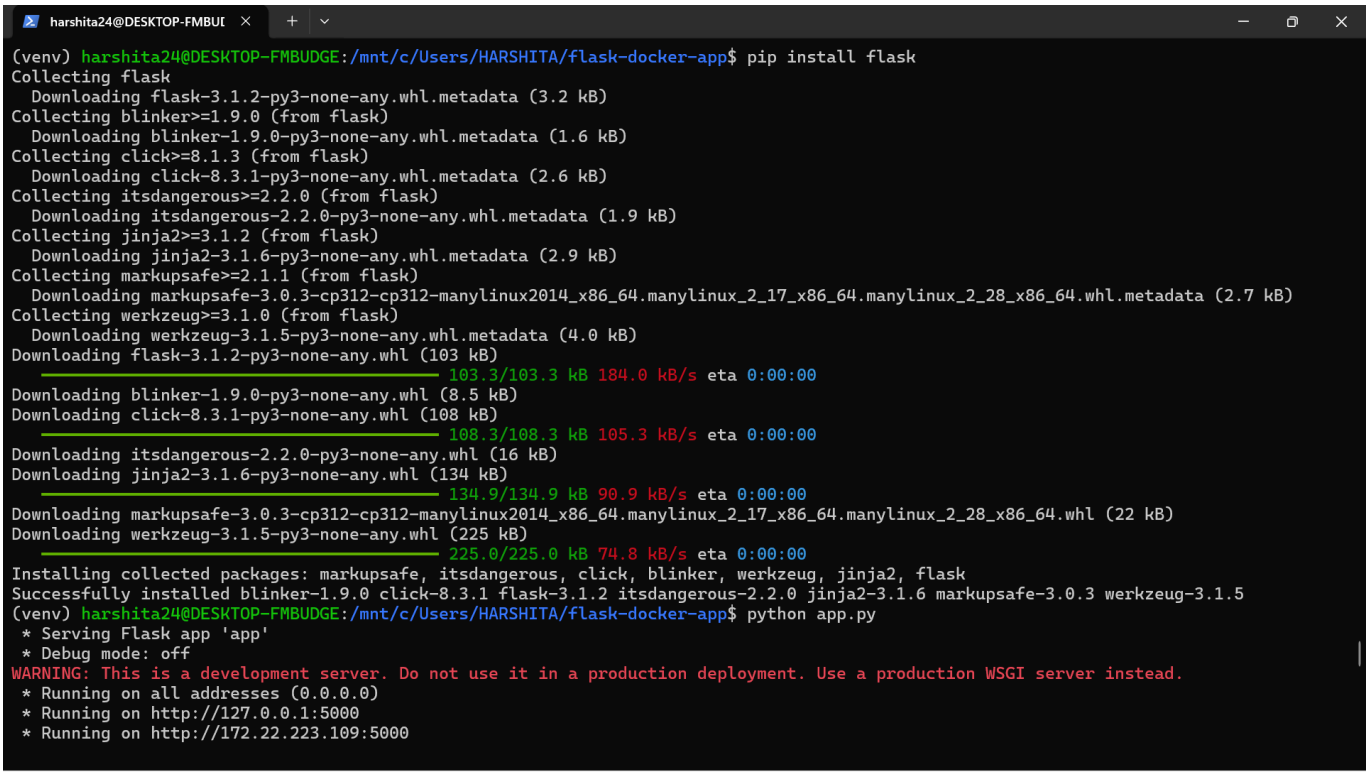
Step 2: Create Application Files

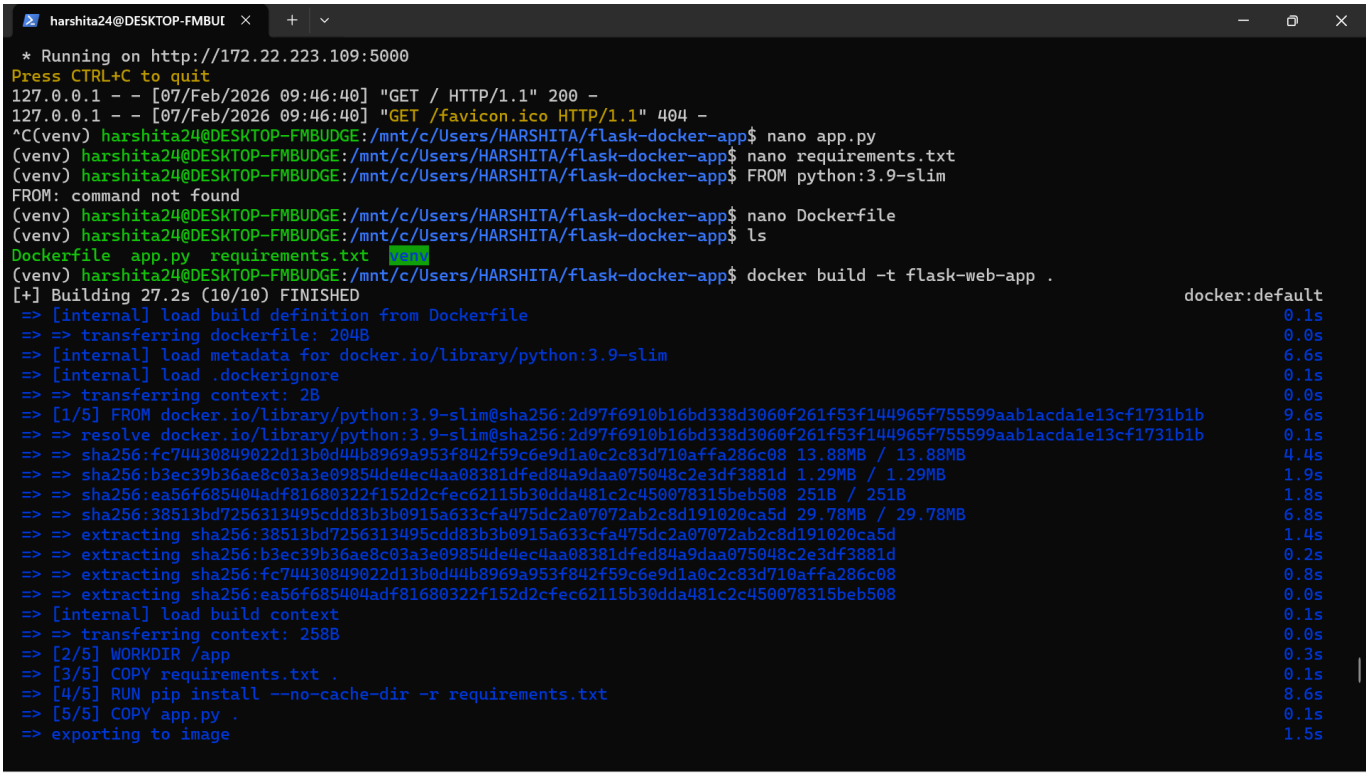
Inside the project directory, create:

- `app.py`
- `requirements.txt`

requirements.txt

```
flask
```





Step 3: Test the Application Locally

Before containerizing, test locally:

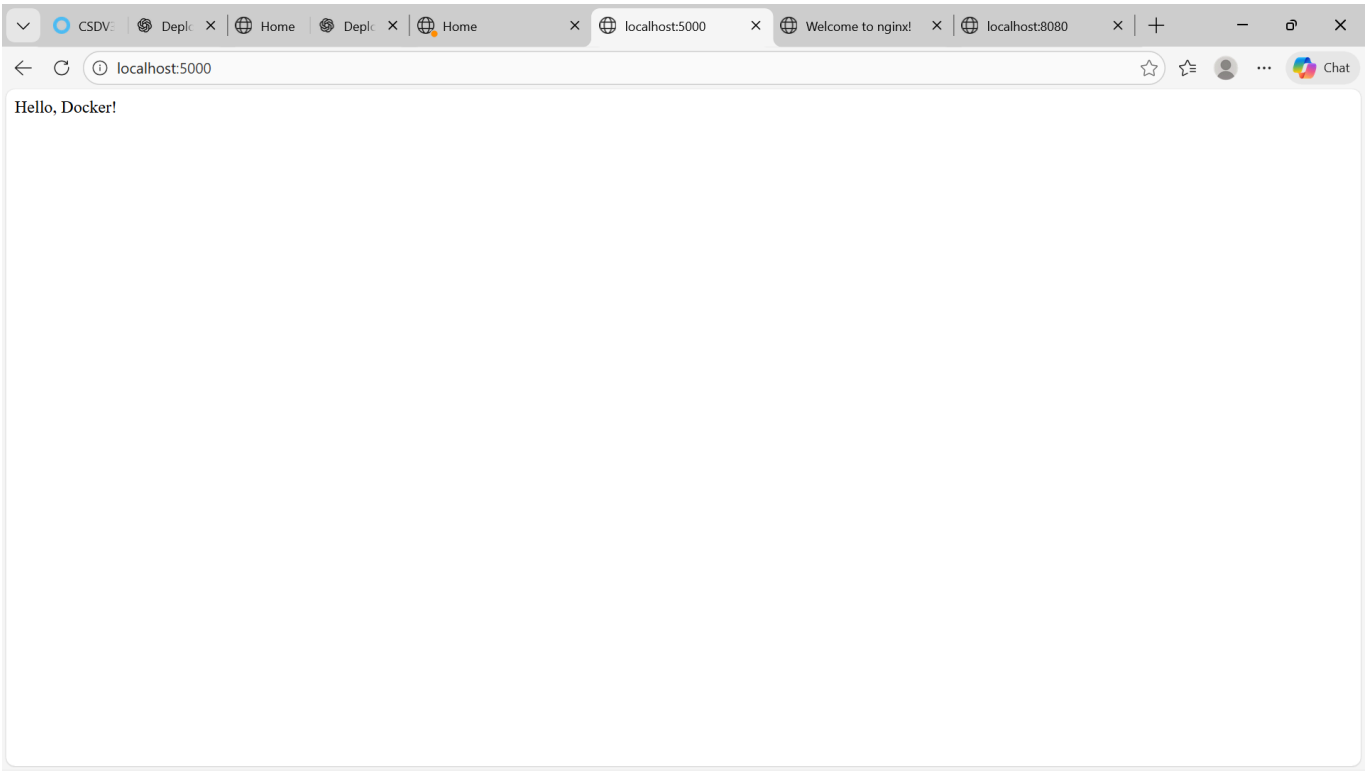
```
python app.py
```

Visit:

```
http://localhost:8080
```

Expected Output:

```
Hello, Docker!
```



Fix any errors before proceeding.

Step 4: Writing the Dockerfile

Create a file named `Dockerfile`.

Dockerfile Content

```
# Base Image
FROM python:3.9-slim

# Set working directory
```

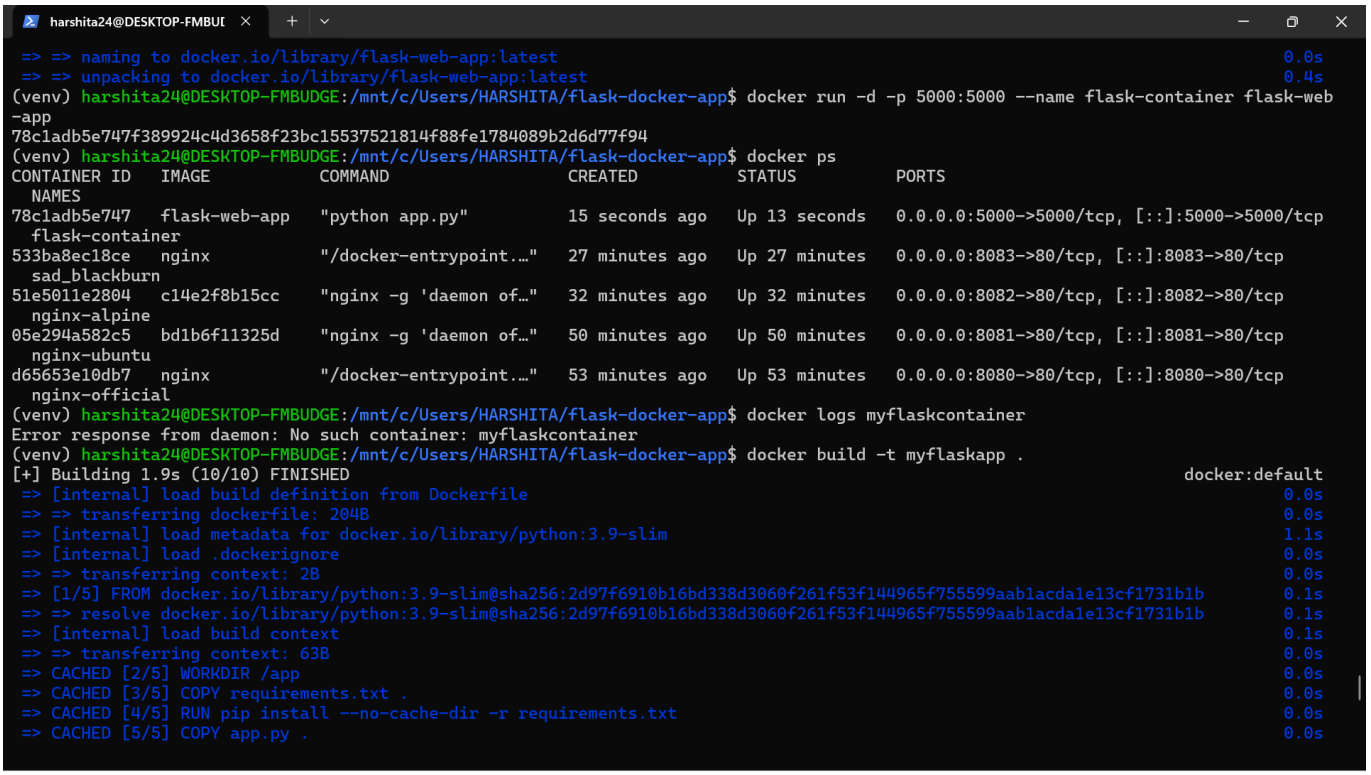
```
WORKDIR /app

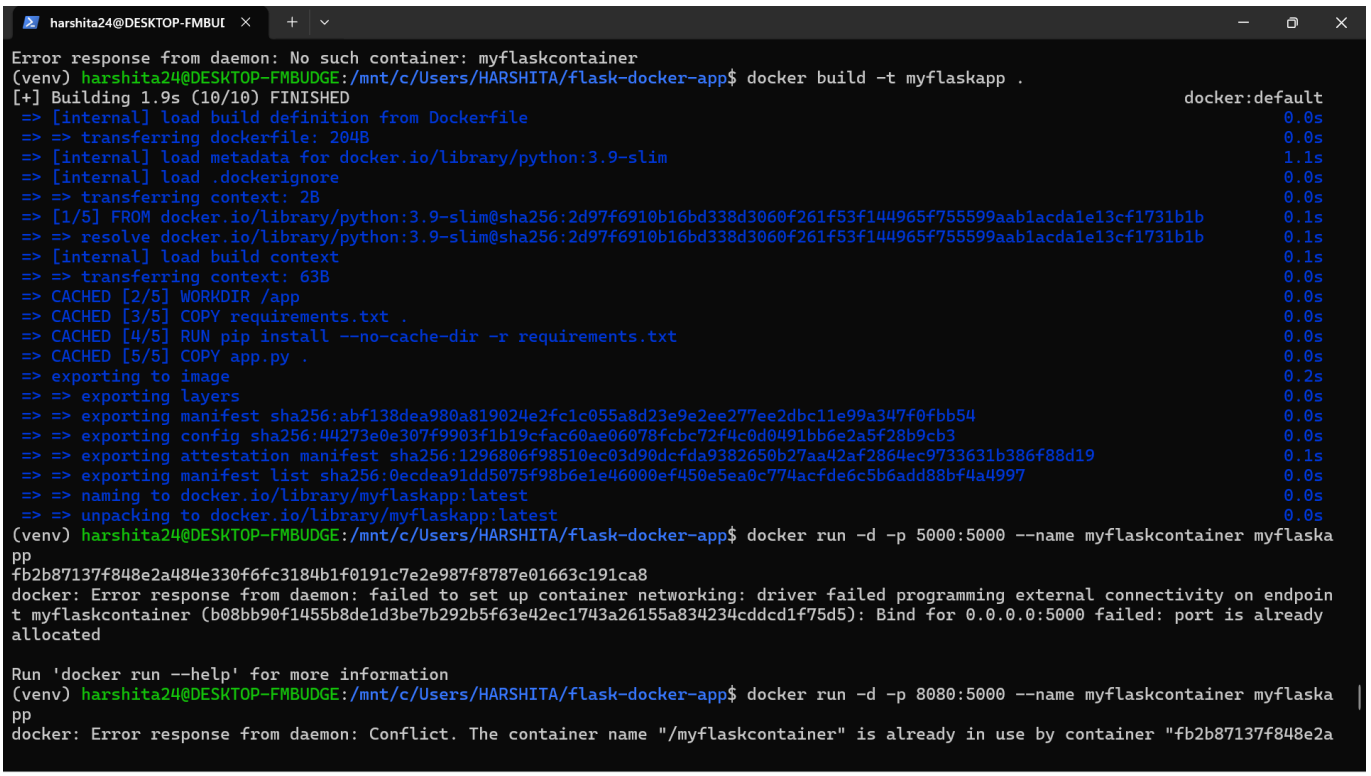
# Copy application files
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose port
EXPOSE 8080

# Start application
CMD ["python", "app.py"]
```





Explanation of Dockerfile Sections

1. Base Image

```
FROM python:3.9-slim
```

This provides a lightweight Python runtime environment.

---

2. Copying Files

```
COPY . /app
```

Copies all application files into the container.

---

3. Installing Dependencies

```
RUN pip install --no-cache-dir -r requirements.txt
```

Installs Flask and other dependencies.

---

4. Entrypoint / Command

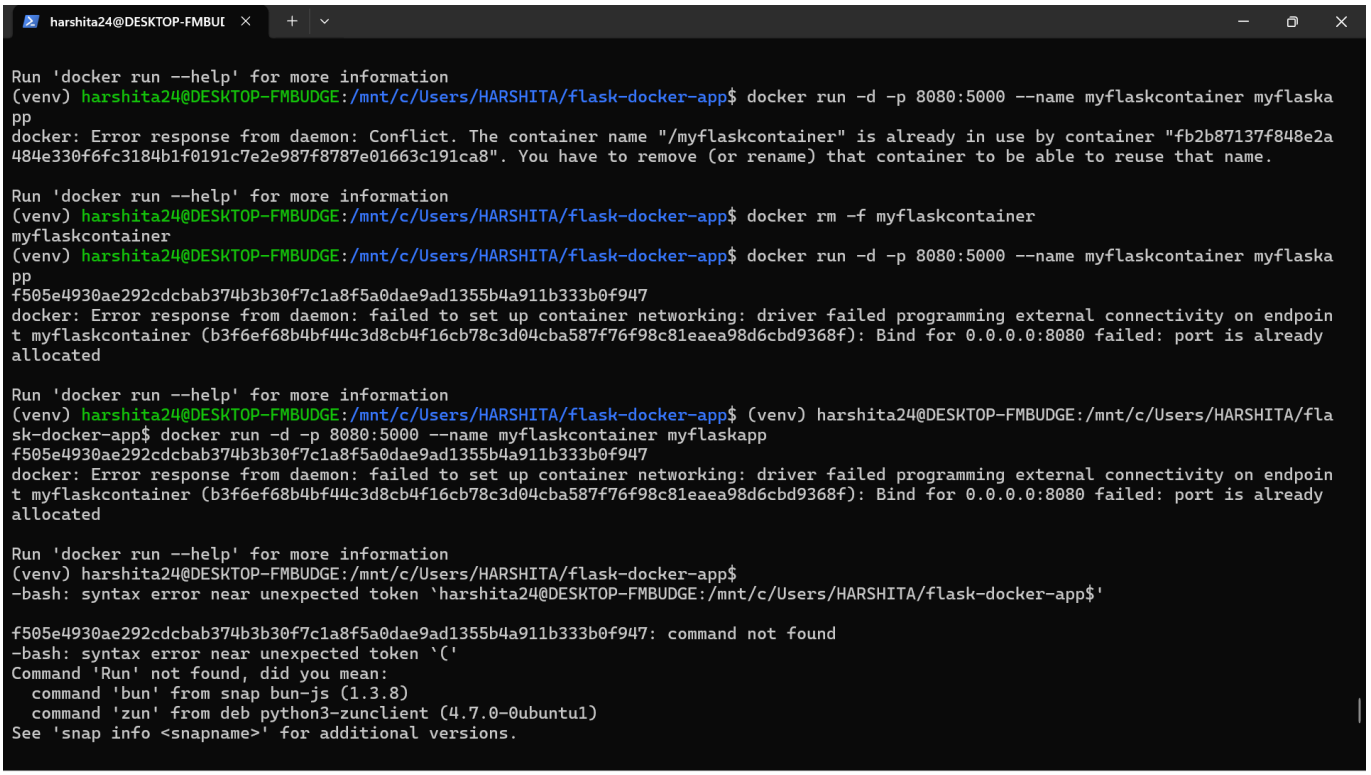
```
CMD ["python", "app.py"]
```

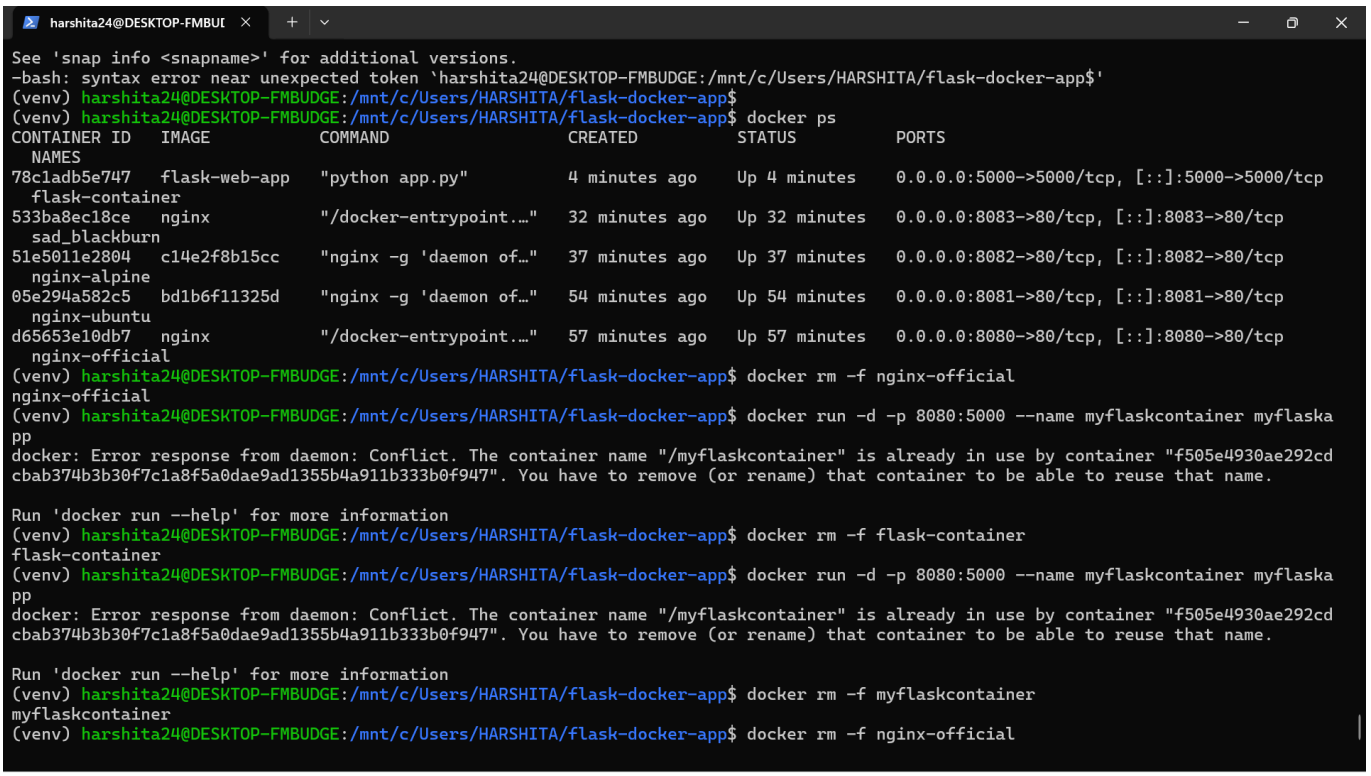
Defines the default command to run the application.

Step 5: Build the Docker Image

Run:

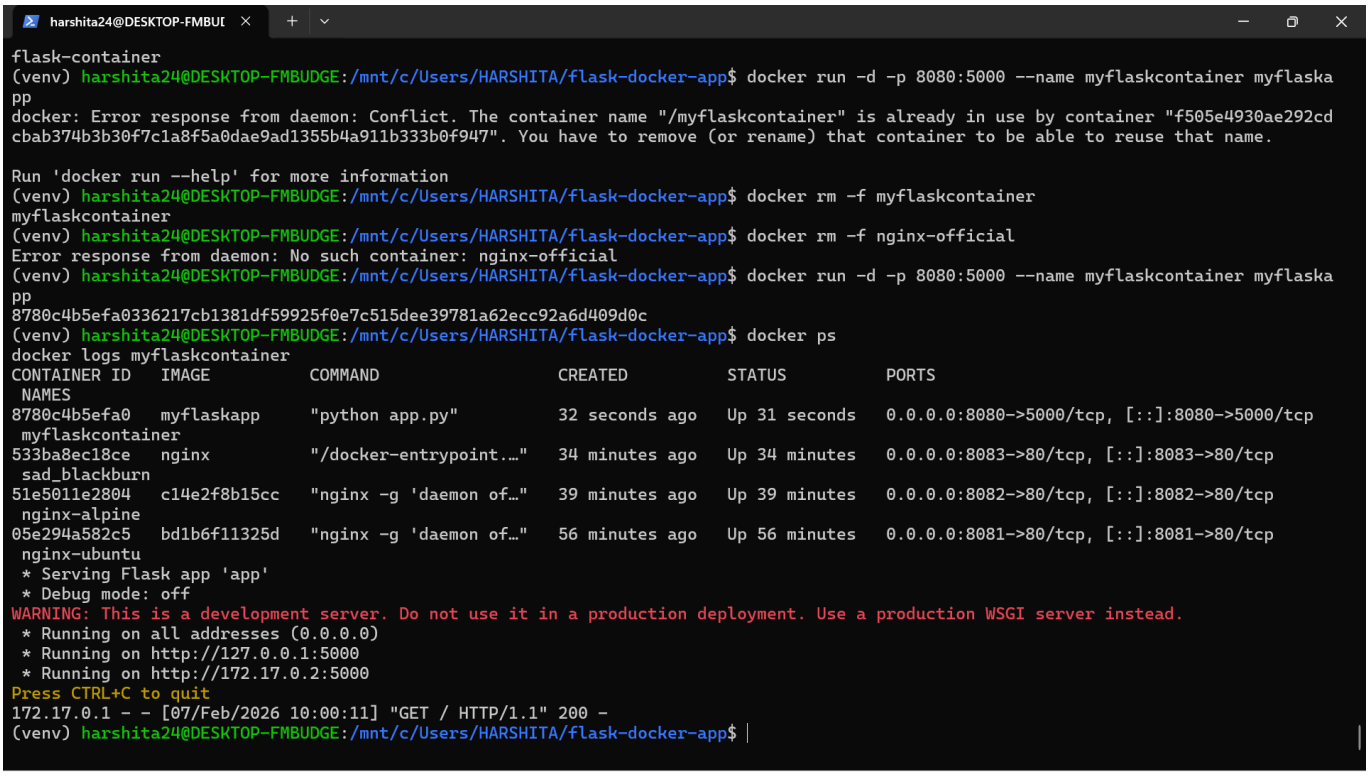
```
docker build -t flask-docker-app .
```





Step 6: Run the Docker Container

```
docker run -d -p 8080:8080 --name flask-container flask-docker-app
```



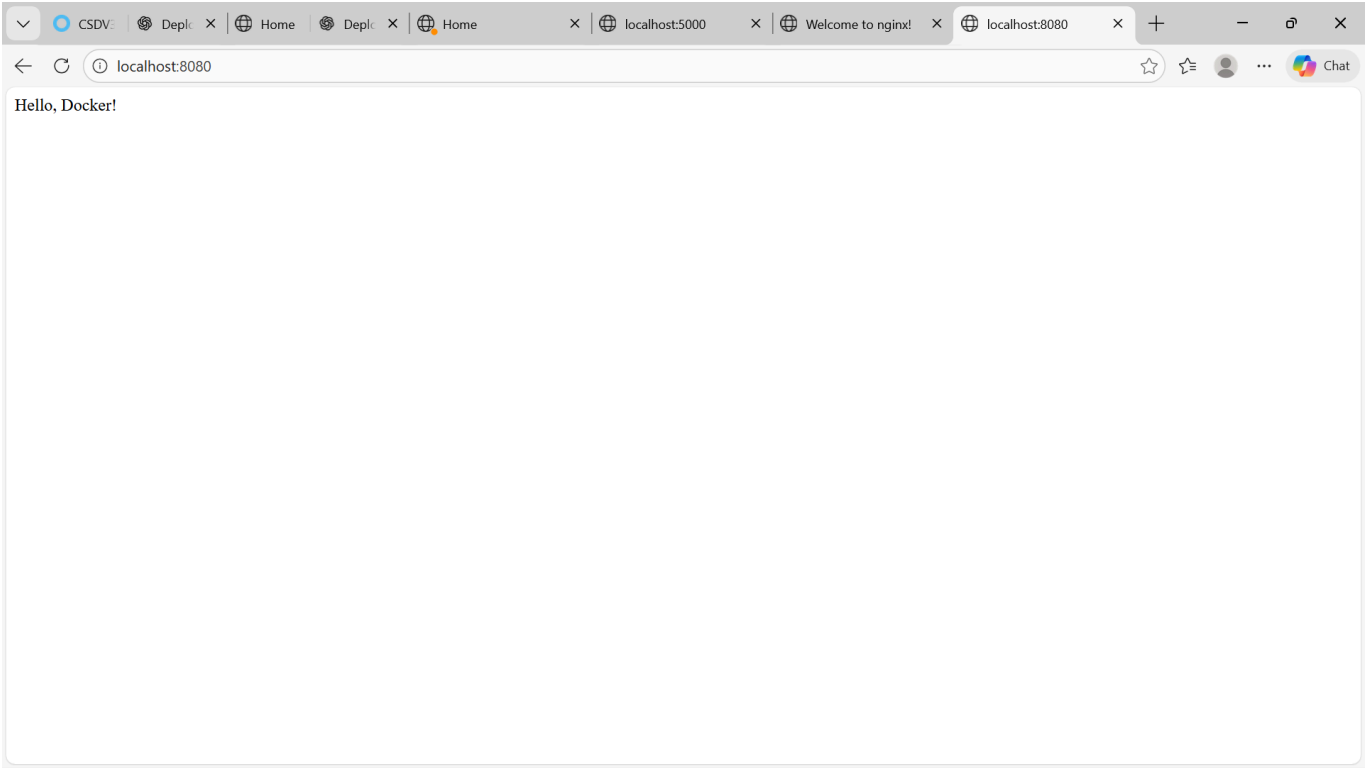
Step 7: Verify the Application

Open in browser:

```
http://localhost:8080
```

Expected Output:

```
Hello, Docker!
```



Troubleshooting (If Issues Occur)

If nothing loads:

- Check container status:

```
docker ps
```

- Check container logs:

```
docker logs flask-container
```

Result

The web application was successfully containerized using Docker.
A custom image was built, the container was launched, and the application was accessed via a web browser.

This experiment demonstrates the complete workflow of deploying a web application in a containerized environment.

. dockerignore

Experiment 4: Docker Essentials

Dockerfile

Part 1: Containerizing Applications with Dockerfile

Step 1: Create a Simple Application

Python Flask App:

tagging

publishing

mkdir my-flask-app cd my-flask-app

app . py from flask import Flask app = Flask(_name

@app.route('/') def hello(): return "Hello from Docker!"

@app.route('/health') def health(): return "OK"

name app.run(host='0.0.0.0', port=5000)

if

main

requirements.txt

Flask == 2.3.3

Step 2: Create Dockerfile Dockerfile

Use Python base image

FROM python: 3.9-slim

Set working directory

WORKDIR /app

Copy requirements file

COPY requirements.txt .

Install dependencies

RUN pip install -- no-cache-dir -r requirements.txt

Copy application code

COPY app.py .

Expose port

EXPOSE 5000

Run the application

CMD ["python", "app.py"] Part 2: Using .dockerignore

Step 1: Create .dockerignore File

. dockerignore

.

Python files

_pycache

- . pyc
- . pyo .pyd

Environment files

. env . venv env/ venv/

IDE files

. vscode/ .idea/

Git files

.git/ .gitignore

OS files

.DS_Store Thumbs.db

Logs

*. log logs/

Test files

tests/ test *. py

Step 2: Why .dockerignore is Important

· Prevents unnecessary files from being copied · Reduces image size . Improves build speed Part 3: Building Docker Images

Step 1: Basic Build Command

Build image from Dockerfile

docker build -t my-flask-app .

Check built images

docker images

Step 2: Tagging Images

Tag with version number

docker build -t my-flask-app:1.0 .

Tag with multiple tags

docker build -t my-flask-app:latest -t my-flask-app:1.0 .

Tao with cuctom nogistey

docker build -t my-flask-app:1.0 .

Tag with multiple tags

docker build -t my-flask-app:latest -t my-flask-app:1.0 .

Tag with custom registry

docker build -t username/my-flask-app:1.0 .

Tag existing image

docker tag my-flask-app:latest my-flask-app:v1.0

Step 3: View Image Details

List all images

docker images

Show image history

docker history my-flask-app

Inspect image details

docker inspect my-flask-app Part 4: Running Containers

Step 1: Run Container

Run container with port mapping

docker run -d -p 5000:5000 -- name flask-container my-flask-app

Test the application

curl http://localhost:5000

View running containers

docker ps

View container logs

docker logs flask-container

Sten 2: Manage Containers Step 2: Manage Containers

Stop container

docker stop flask-container

Start stopped container

docker start flask-container

Remove container

docker rm flask-container

Remove container forcefully

docker rm -f flask-container Part 5: Multi-stage Builds

Step 1: Why Multi-stage Builds?

· Smaller final image size . Better security (remove build tools) · Separate build and runtime environments

Step 2: Simple Multi-stage Dockerfile

Dockerfile.multistage

STAGE 1: Builder stage

FROM python: 3.9-slim AS builder

STAGE 1: Builder stage

FROM python:3.9-slim AS builder

WORKDIR /app

Copy requirements

COPY requirements.txt .

Install dependencies in virtual environment

RUN python -m venv /opt/venv ENV PATH="/opt/venv/bin:$PATH" RUN pip install --no-cache-dir -r requirements.txt

STAGE 2: Runtime stage

FROM python:3.9-slim

WORKDIR /app

Copy virtual environment from builder

COPY --from=builder /opt/venv /opt/venv ENV PATH="/opt/venv/bin:$PATH"

Copy application code

COPY app.py .

Create non-root user

RUN useradd -m -u 1000 appuser USER appuser

Expose port

EXPOSE 5000

Run application

CMD ["python", "app.py"] Step 3: Build and Compare

Build regular image

docker build -t flask-regular .

Build multi-stage image

docker build -f Dockerfile.multistage -t flask-multistage .

Compare sizes

docker images | grep flask-

Expected output:

flask-regular

flask-multistage

~250MB ~150MB (40% smaller!) Part 6: Publishing to Docker Hub

Step 1: Prepare for Publishing

Login to Docker Hub

docker login

Tag image for Docker Hub

docker tag my-flask-app:latest username/my-flask-app:1.0 docker tag my-flask-app:latest username/my-flask-app:latest

Push to Docker Hub

docker push username/my-flask-app:1.0 docker push username/my-flask-app:latest

Step 2: Pull and Run from Docker Hub

Pull from Docker Hub (on another machine)

docker pull username/my-flask-app:latest

Run the pulled image

docker run -d -p 5000:5000 username/my-flask-app:latest

Part 7: Node.js Example (Quick Version)

Step 1: Node.js Application

mkdir my-node-app cd my-node-app

app. js const express = require('express'); const app = express(); const port = 3000;

app.get('/', (req, res) => { res.send('Hello from Node.js Docker!'); });

app.get('/health', (req, res) => { res.json({ status: 'healthy' }); });

app.listen(port, () => { console.log(`Server running on port ${port}`); });

package.json { "name": "node-docker-app", "version": "1.0.0", "main": "app.js", "dependencies": { "express": "^4.18.2" } } Step 2: Node.js Dockerfile

FROM node: 18-alpine

WORKDIR /app

COPY package *. json . / RUN npm install -- only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

Step 3: Build and Run

Build image

docker build -t my-node-app .

Run container

docker run -d -p 3000:3000 --name node-container my-node-app

Test

curl http://localhost:3000

🐳 Containerization and DevOps

Experiment 5: Docker – Volumes, Environment Variables, Monitoring & Networks

---

📌 Objective

To understand and implement core Docker concepts including volumes, environment variables, monitoring, and networking, and to build a real-world multi-container application.

🔹 Part 1: Docker Volumes – Persistent Data Storage

🧪 Lab 1: Understanding Data Persistence

Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

Inside container:

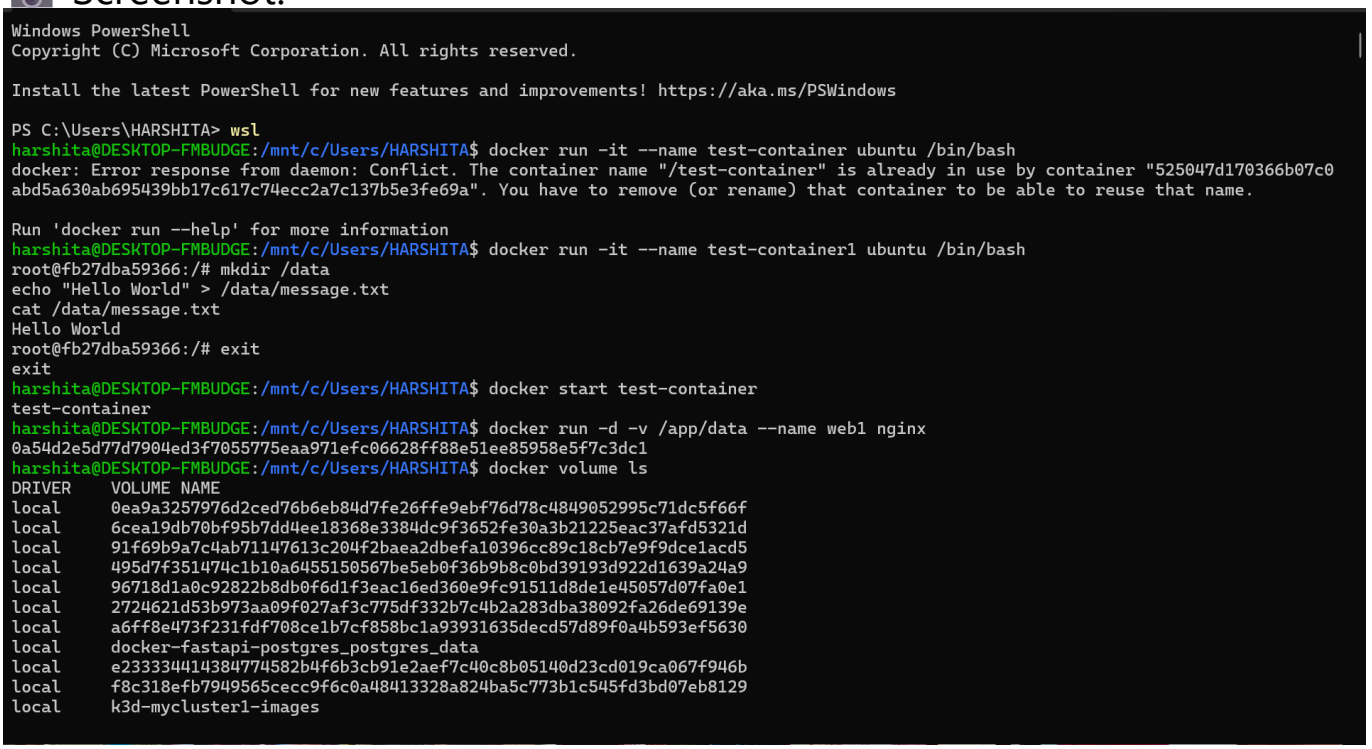
```
echo "Hello World" > /data/message.txt
cat /data/message.txt
exit
```

Restart container:

```
docker start test-container
docker exec test-container cat /data/message.txt
```

👉 Data is lost after restart.

📸 Screenshot:

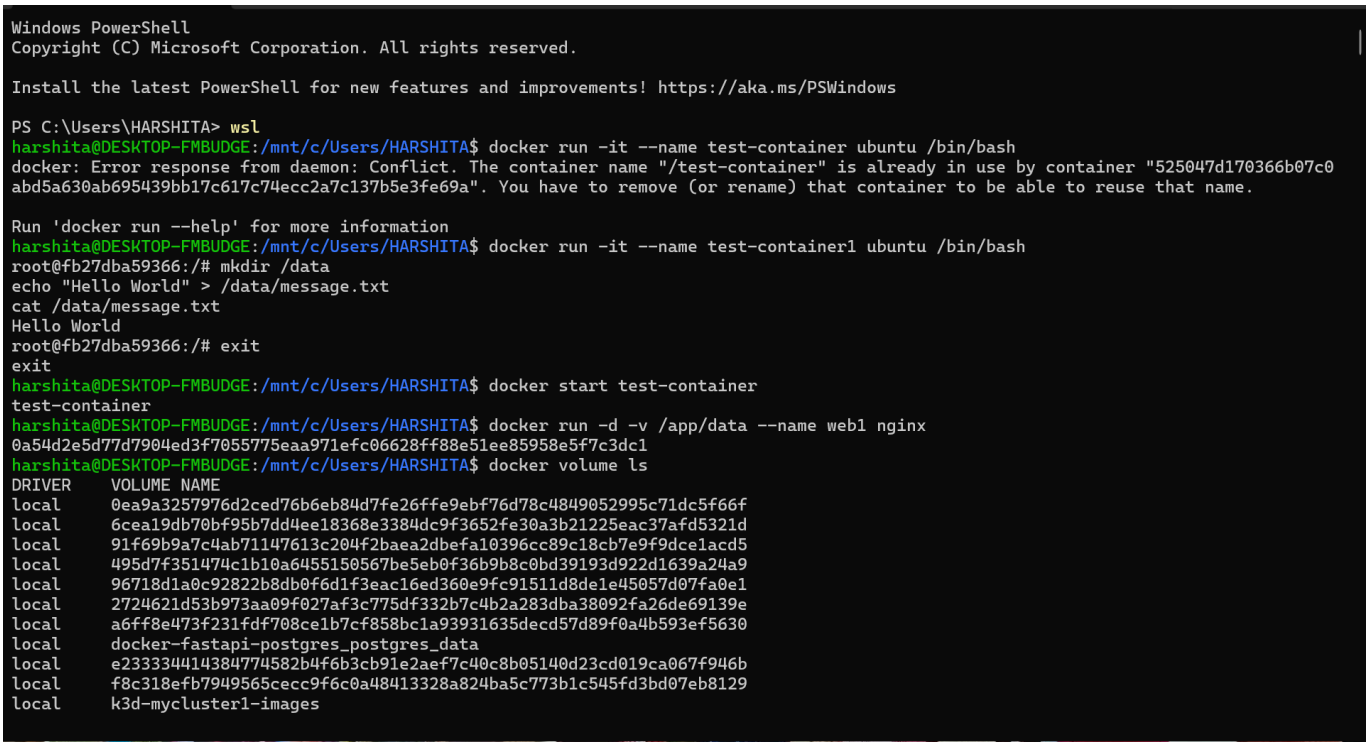


🧪 Lab 2: Volume Types

1. Anonymous Volume

```
docker run -d -v /app/data --name web1 nginx
```

📸 Screenshot:

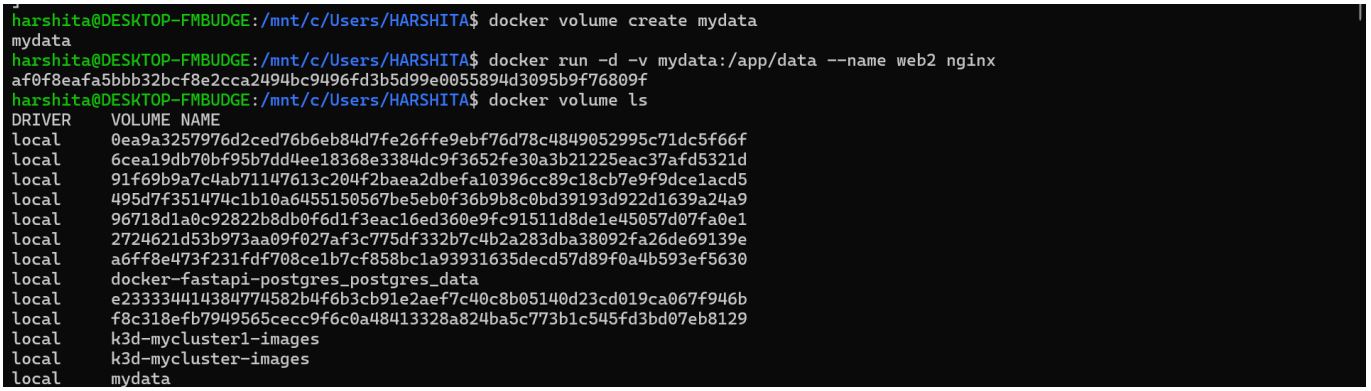


---

2. Named Volume

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
```

📸 Screenshot:

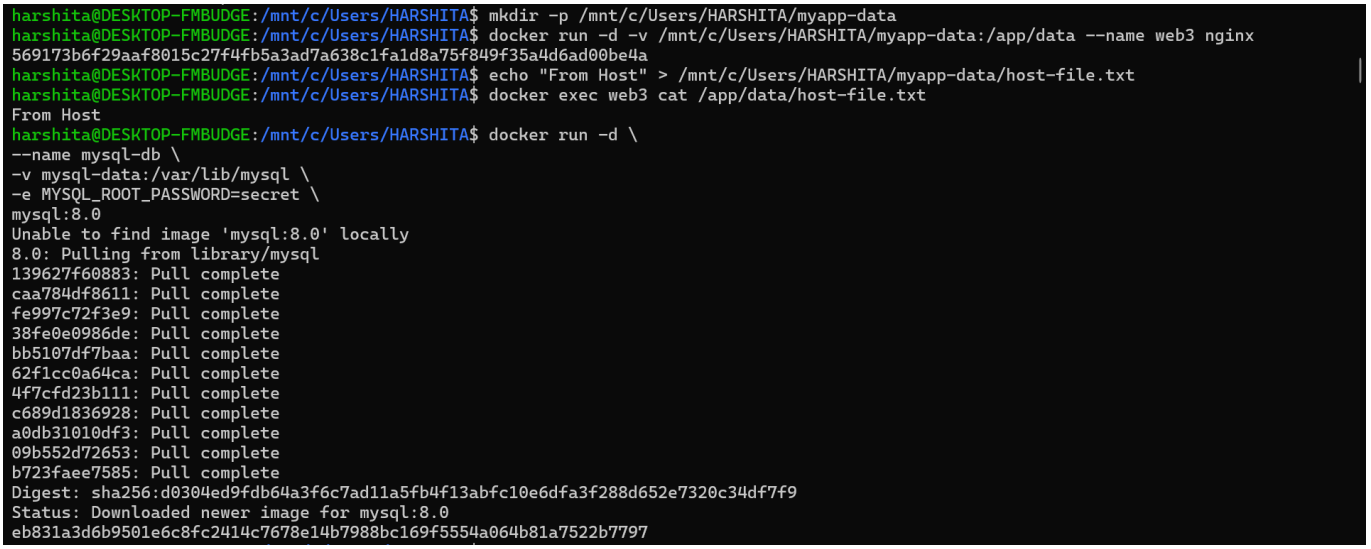


---

3. Bind Mount

```
docker run -d -v /mnt/c/Users/HARSHITA/myapp-data:/app/data --name web3 nginx
```

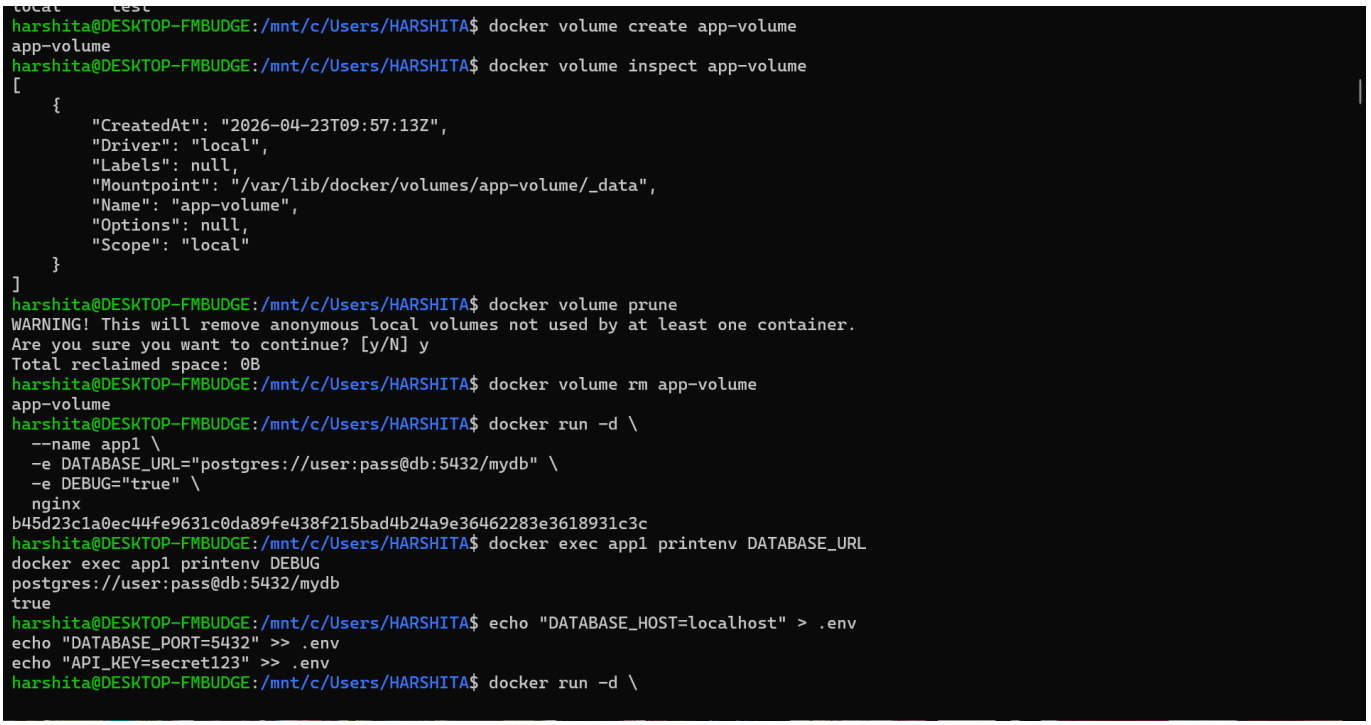
📸 Screenshot:



🧪 Lab 3: Volume Commands

```
docker volume ls
docker volume create app-volume
docker volume inspect app-volume
docker volume prune
docker volume rm app-volume
docker cp local-file.txt web3:/app/data/
```

📸 Screenshot:



🔷 Part 2: Environment Variables

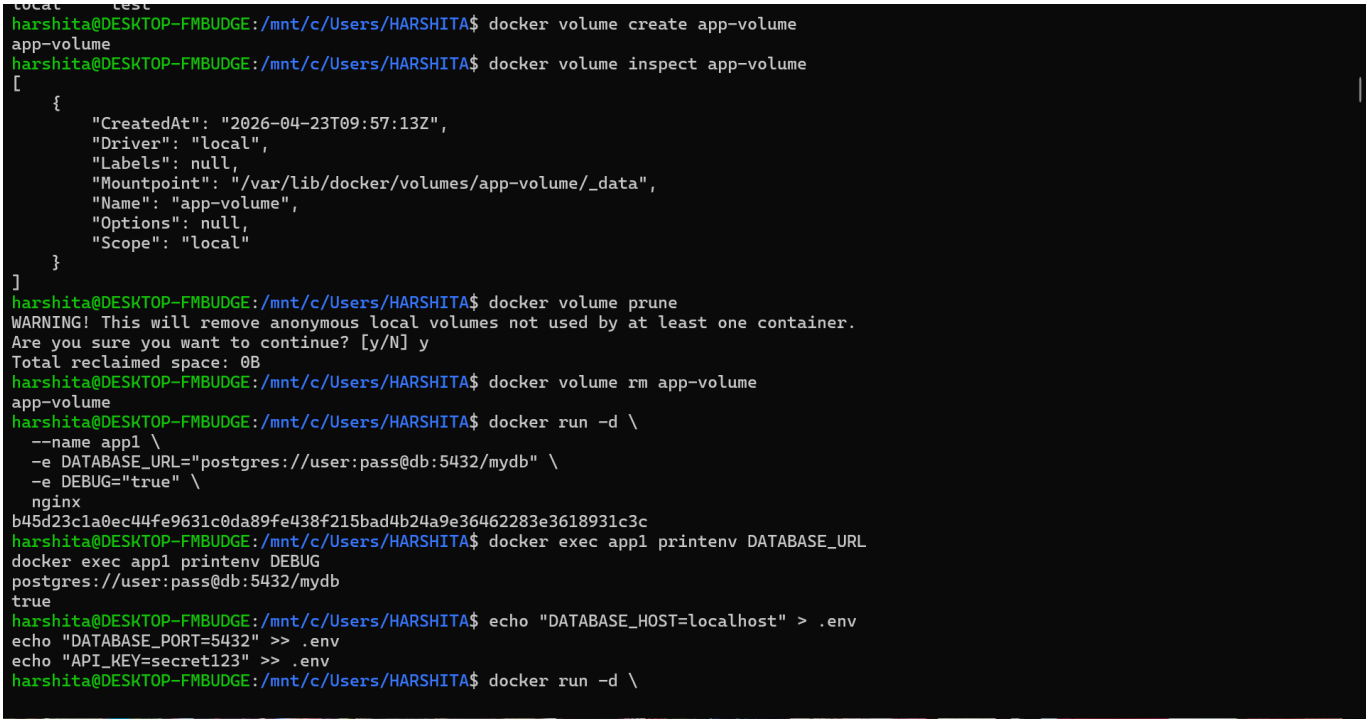
🧪 Lab 1: Using -e Flag

```
docker run -d \
  --name app1 \
  -e DEBUG=true \
  nginx
```

🧪 Lab 2: Using .env File

```
docker run -d --env-file .env --name app2 nginx
```

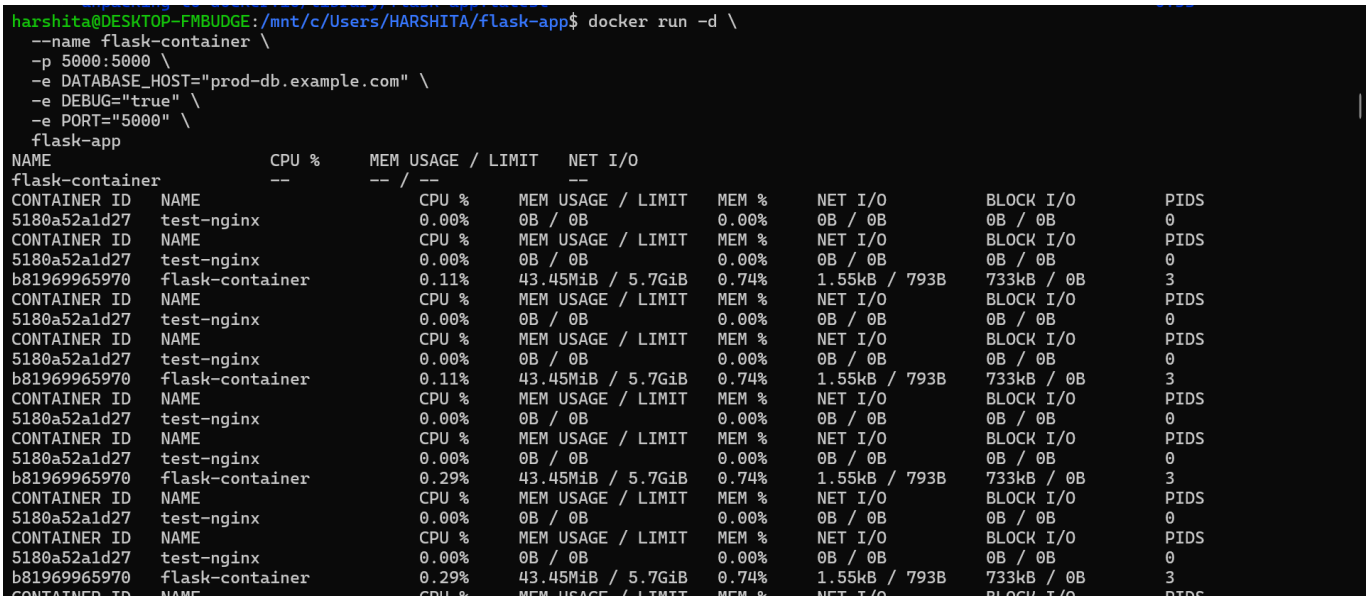
📸 Screenshot:



🧪 Lab 3: Flask App with Environment Variables

```
docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST=prod-db \
  flask-app
```

📸 Screenshot:

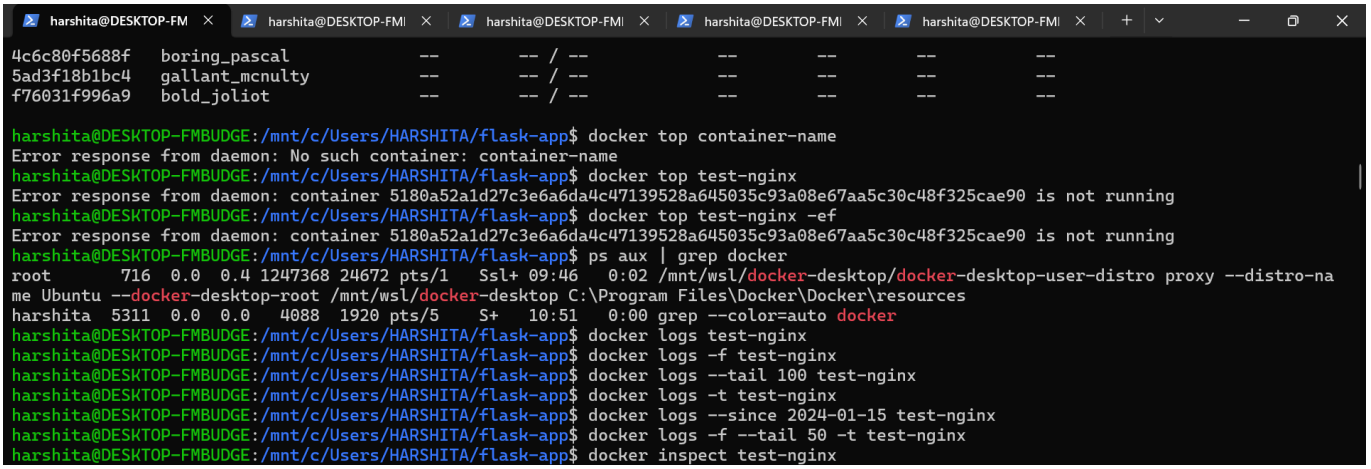


🔷 Part 3: Docker Monitoring

🧪 Lab 1: docker stats

```
docker stats
```

📸 Screenshot:



🧪 Lab 2: docker top

```
docker top flask-app
```

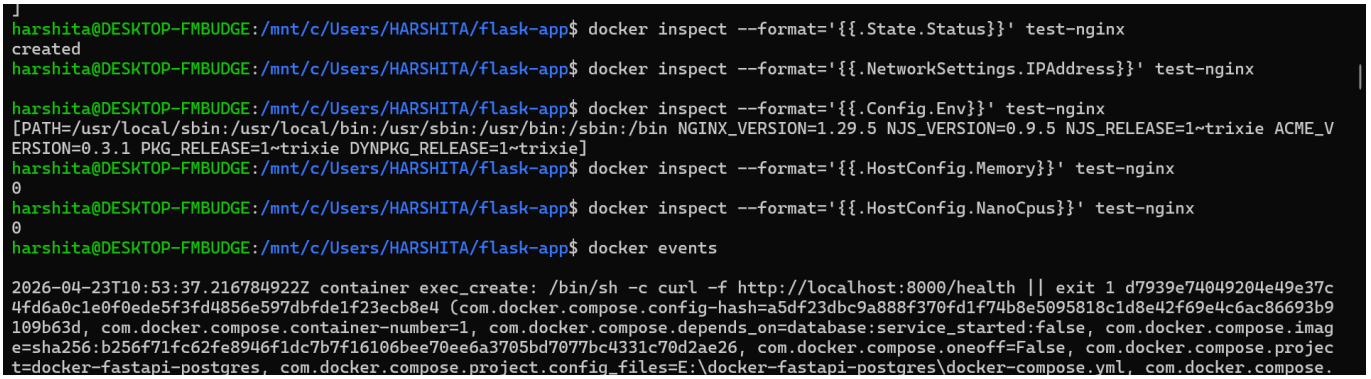
🧪 Lab 3: docker logs

```
docker logs -f flask-app
```

🧪 Lab 4: docker inspect

```
docker inspect flask-app
```

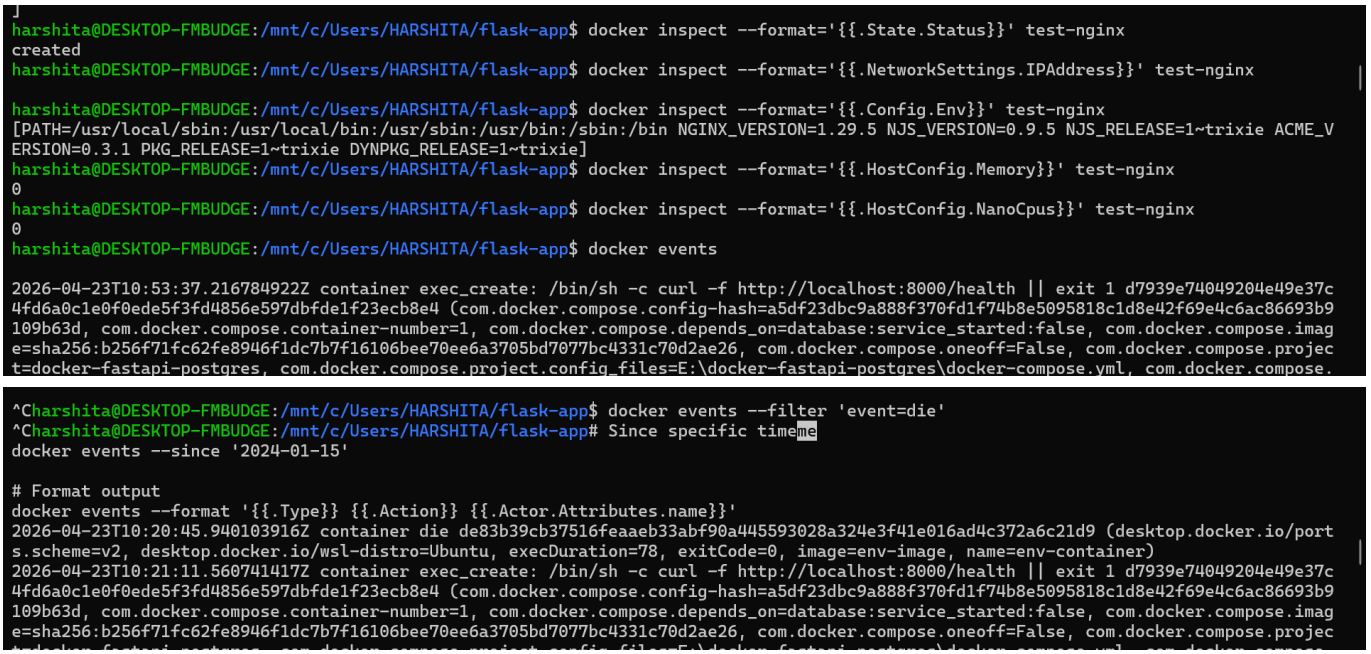
📸 Screenshot:



🧪 Lab 5: docker events

```
docker events
```

📸 Screenshot:

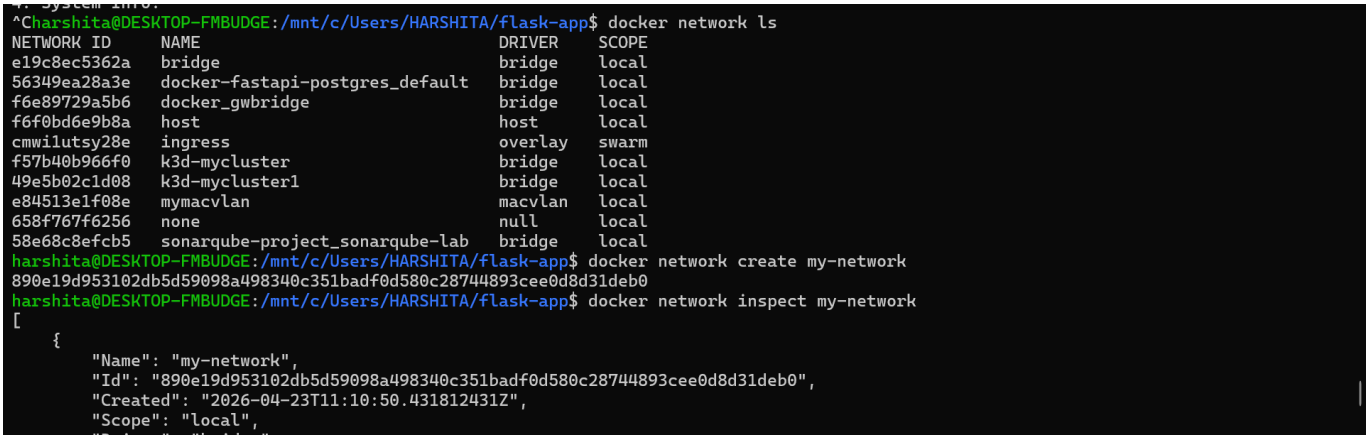


🔷 Part 4: Docker Networks

🧪 Lab 1: List Networks

```
docker network ls
```

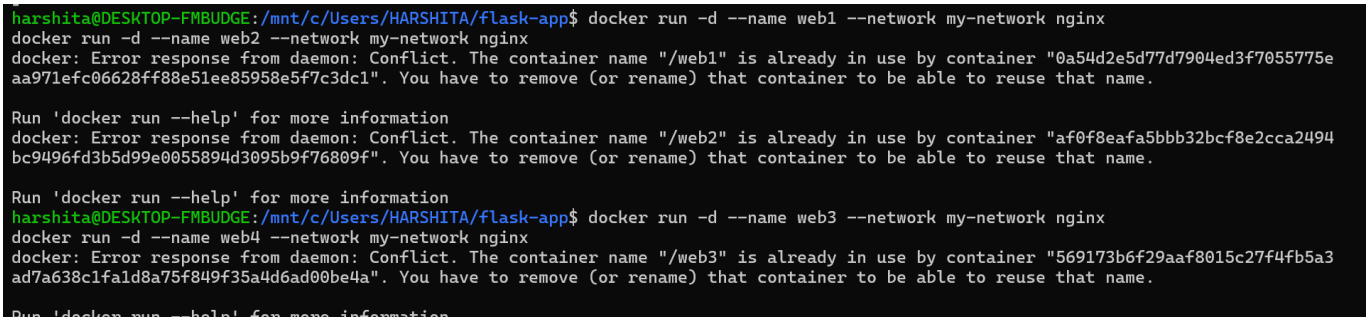
📸 Screenshot:



🧪 Lab 2: Bridge Network

```
docker network create my-network
docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx
```

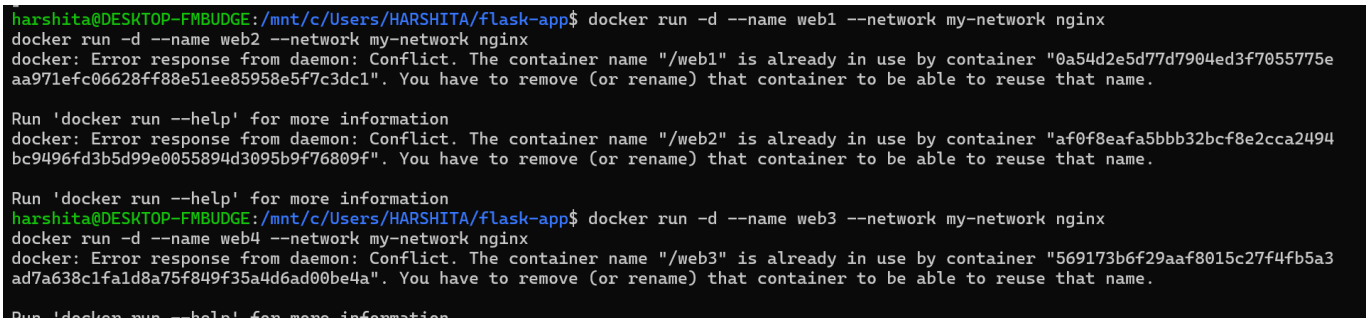
📸 Screenshot:



🧪 Lab 3: Network Commands

```
docker network inspect my-network
docker network connect my-network web1
docker network disconnect my-network web1
```

📸 Screenshot:



🔷 Part 5: Real-World Multi-Container Application

🏗️ Architecture

- Flask App → Port 5000
- PostgreSQL → Port 5432
- Redis → Port 6379
- Custom Network

⚙️ Implementation

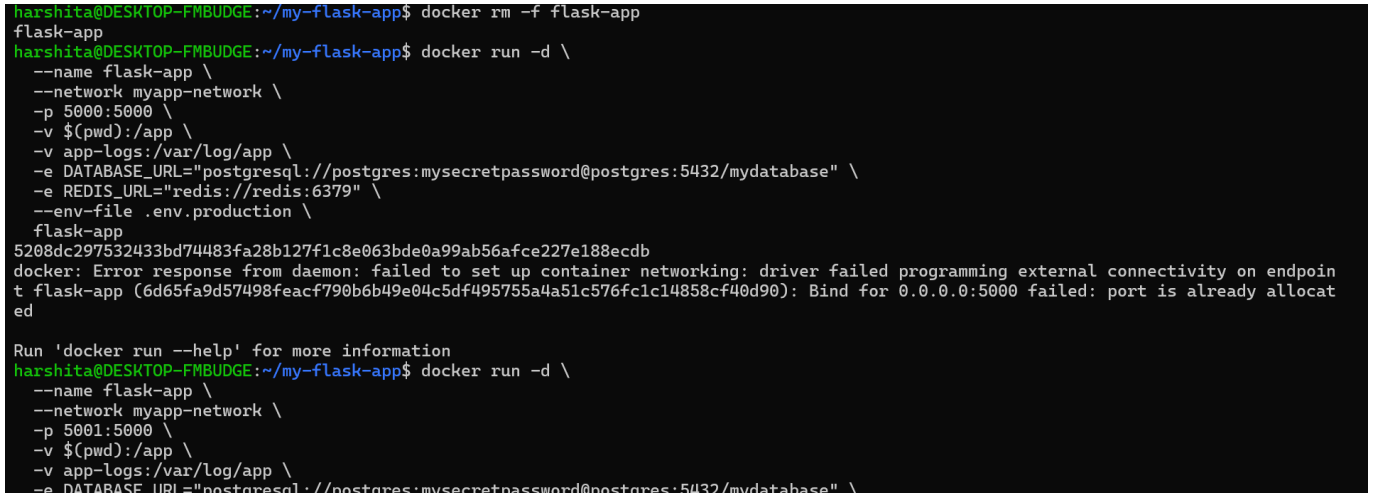
```
docker network create myapp-network
```

```
docker run -d --name postgres --network myapp-network -e
POSTGRES_PASSWORD=mysecretpassword postgres:15
```

```
docker run -d --name redis --network myapp-network redis:7-alpine
```

```
docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5000:5000 \
  -e
DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  flask-app
```

📸 Screenshot:



```
67ed2a107425ab09ed4018f58fd4a579660a37f4887895c5cc9b5bb9d5879724
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ curl http://localhost:5000
{"database":"postgresql://postgres:mysecretpassword@postgres:5432/mydatabase","debug":"false","redis":"redis://redis:6379"}
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED             STATUS                  PORTS
                         NAMES
67ed2a107425   flask-app                "python app.py"          28 seconds ago      Up 27 seconds           0.0.0.0:5000-
>5000/tcp, [::]:5000->5000/tcp   flask-app
bb7e3e02fa5d   redis:7-alpine           "docker-entrypoint.s…"   4 minutes ago       Up 4 minutes            6379/tcp
                         redis
470567fe8bb0   postgres:15              "docker-entrypoint.s…"   5 minutes ago       Up 4 minutes            5432/tcp
                         postgres
5703ce05cca1   postgres:15              "docker-entrypoint.s…"   12 minutes ago      Up 12 minutes           5432/tcp
                         postgres-db
291ded8544e6   alpine                   "sleep 3600"             15 minutes ago      Up 15 minutes
                         isolated-app
bf6d98ca9b8a   nginx                    "/docker-entrypoint.…"   15 minutes ago      Up 15 minutes
                         host-app
CONTAINER ID   NAME        CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
470567fe8bb0   postgres    --        -- / --             --        --        --          --
bb7e3e02fa5d   redis       --        -- / --             --        --        --          --
67ed2a107425   flask-app   --        -- / --             --        --        --          --

^C
got 3 SIGTERM/SIGINTs, forcefully exiting
```

```
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker exec flask-app ping -c 2 postgres
docker exec flask-app ping -c 2 redis
PING postgres (172.24.0.2) 56(84) bytes of data.
64 bytes from postgres.myapp-network (172.24.0.2): icmp_seq=1 ttl=64 time=0.072 ms
64 bytes from postgres.myapp-network (172.24.0.2): icmp_seq=2 ttl=64 time=0.059 ms

--- postgres ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.059/0.065/0.072/0.006 ms
PING redis (172.24.0.3) 56(84) bytes of data.
64 bytes from redis.myapp-network (172.24.0.3): icmp_seq=1 ttl=64 time=0.175 ms
64 bytes from redis.myapp-network (172.24.0.3): icmp_seq=2 ttl=64 time=0.110 ms

--- redis ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.110/0.142/0.175/0.032 ms
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker network inspect myapp-network
[
    {
        "Name": "myapp-network",
        "Id": "dbcfcca2cc92d22e5af318530fd3e061803e0b42e86f0a598e28539f5a848f75",
        "Created": "2026-04-23T11:23:25.279261813Z",
        "Scope": "local",
        "Driver": "bridge"
```

🔍 Verification

```
docker ps
docker stats
docker logs -f flask-app
docker network inspect myapp-network
```

📊 Result

Successfully implemented Docker concepts including volumes, environment variables, monitoring, and networking, and deployed a real-world multi-container application.

📌 Key Takeaways

- Docker volumes ensure data persistence
- Environment variables provide flexible configuration
- Monitoring tools help debug containers
- Networks enable container communication
- Multi-container applications simulate real-world systems

Experiment 6 A

Comparison of Docker Run and Docker Compose

PART B - PRACTICAL TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

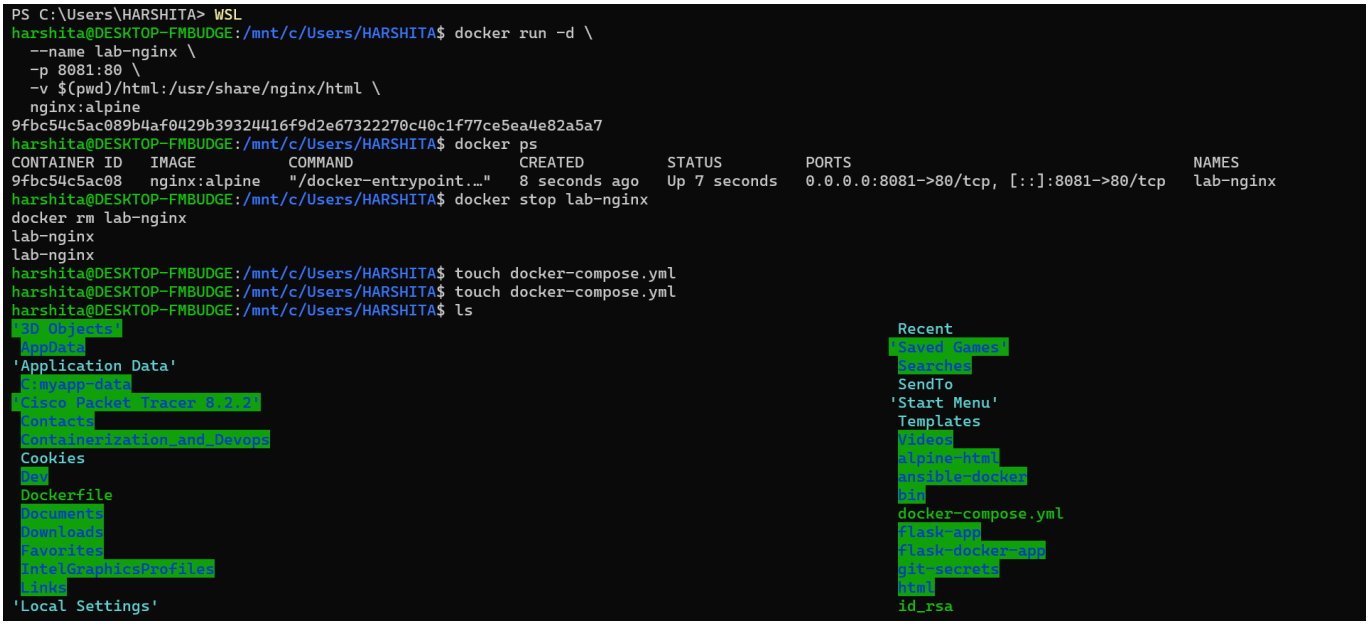
Execute

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

Verify

```
docker ps
```

📸 Screenshot:



Access

http://localhost:8081

Stop & Remove

```
docker stop lab-nginx
docker rm lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml`

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

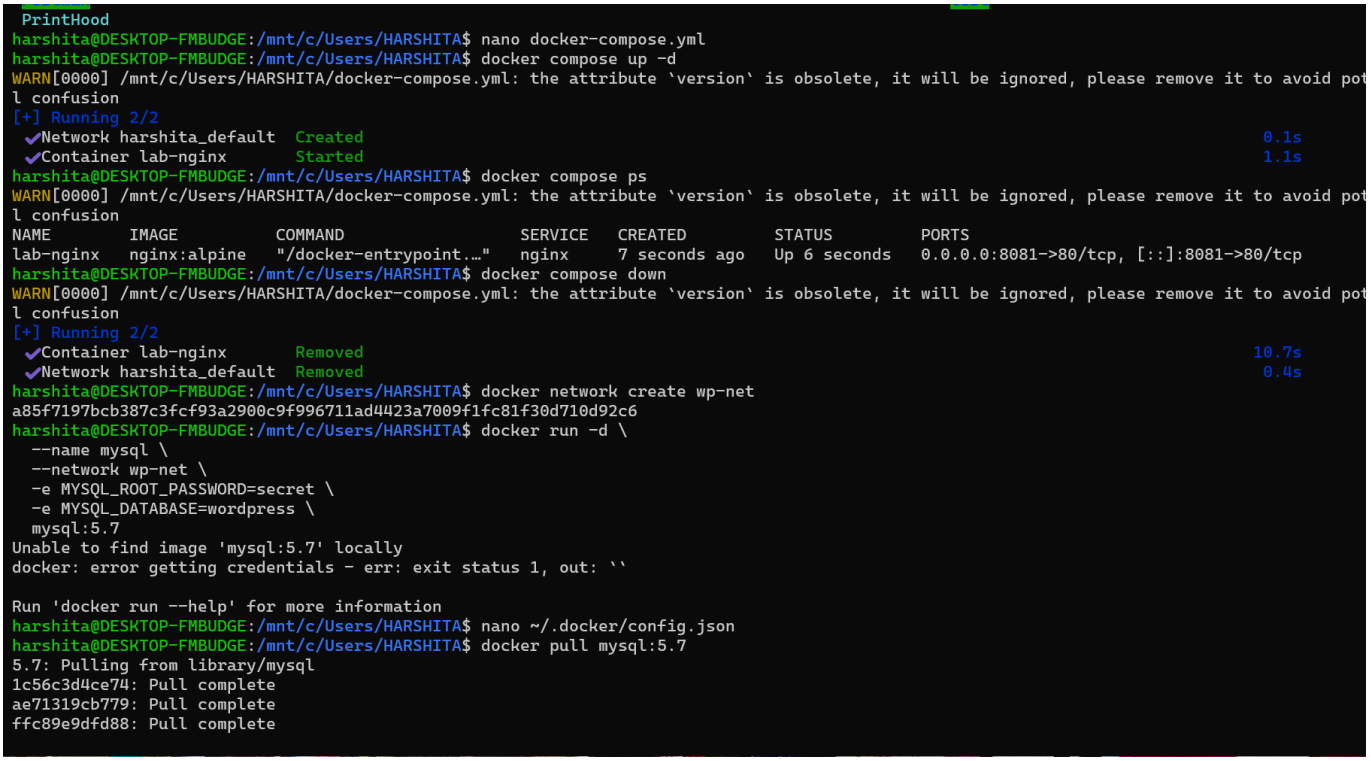
Run

```
docker compose up -d
```

Verify

```
docker compose ps
```

📸 Screenshot: `



Stop

```
docker compose down
```

Task 2: Multi-Container Application

Objective

Deploy WordPress with MySQL using:

- Docker Run (manual way)
- Docker Compose (structured way)

A. Using Docker Run

Step 1: Create Network

```
docker network create wp-net
```

Step 2: Run MySQL

```
docker run -d \
  --name mysql \
```

```
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
```

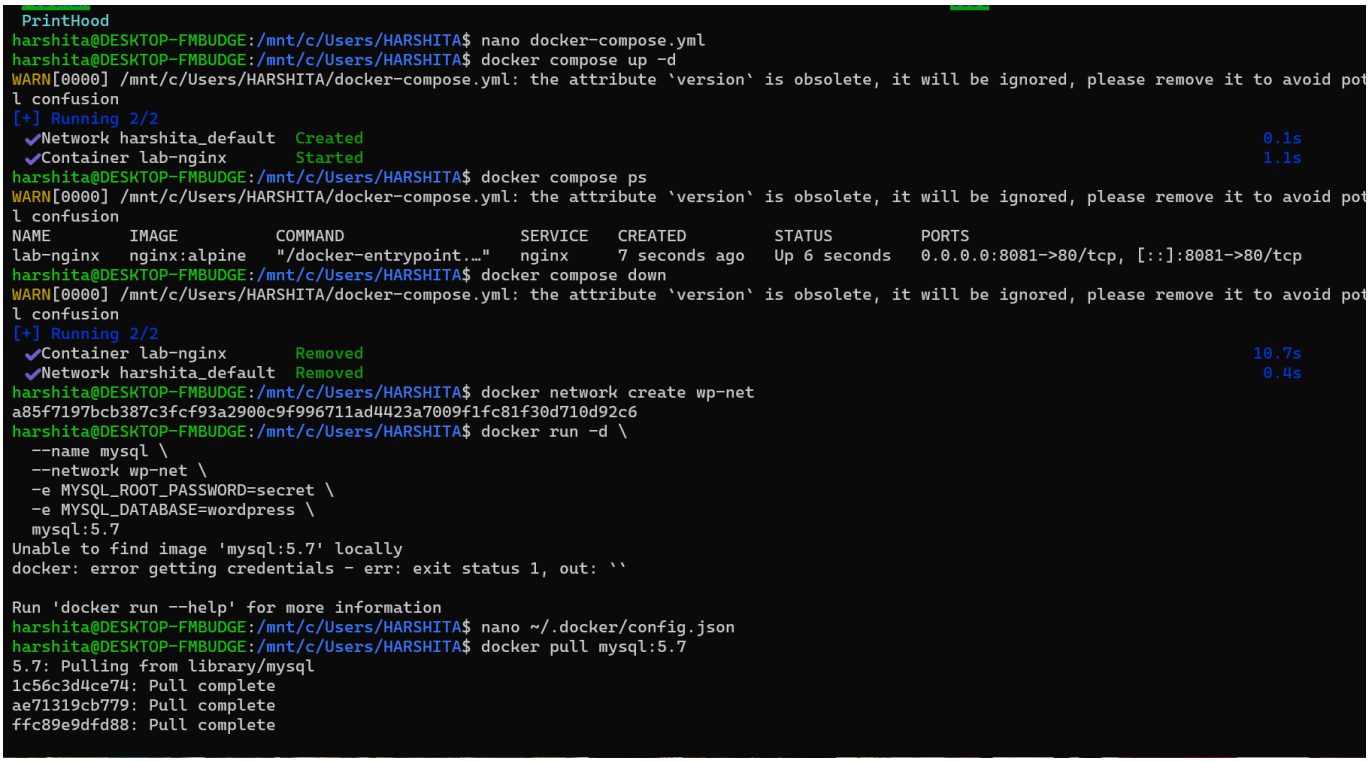
Step 3: Run WordPress

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

Test

http://localhost:8082

📸 Screenshot:



```
docker.io/library/mysql:5.7
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
c6f14eb8dadc699d3970d3b41a55f65b1349054744c4c507460249338db9afe0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
a5f587e05427: Pull complete
899cbc2fdb85: Pull complete
f7417fc69aeb: Pull complete
cac4a54c06df: Pull complete
bd5f94b94043: Pull complete
a923b3cdaa21: Pull complete
```

```
2b5354e1530f829a0f8715426f9c95e691fa1fba609dd525786a27f7783a4b67
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS
2b5354e1530f   wordpress:latest   "docker-entrypoint.s…"   28 seconds ago   Up 24 seconds   0.0.0.0:80
c6f14eb8dadc   mysql:5.7          "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes    3306/tcp,
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker stop wordpress mysql
docker rm wordpress mysql
docker network rm wp-net
wordpress
mysql
wordpress
mysql
wp-net
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be i
l confusion
```

B. Using Docker Compose

Create ````bash

docker compose down

```
---

## Task 2: Multi-Container Application

### Objective

Deploy WordPress with MySQL using:

* Docker Run (manual way)
* Docker Compose (structured way)
```

````
---

### A. Using Docker Run

#### Step 1: Create Network

```bash
docker network create wp-net
````

Step 2: Run MySQL

```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
```

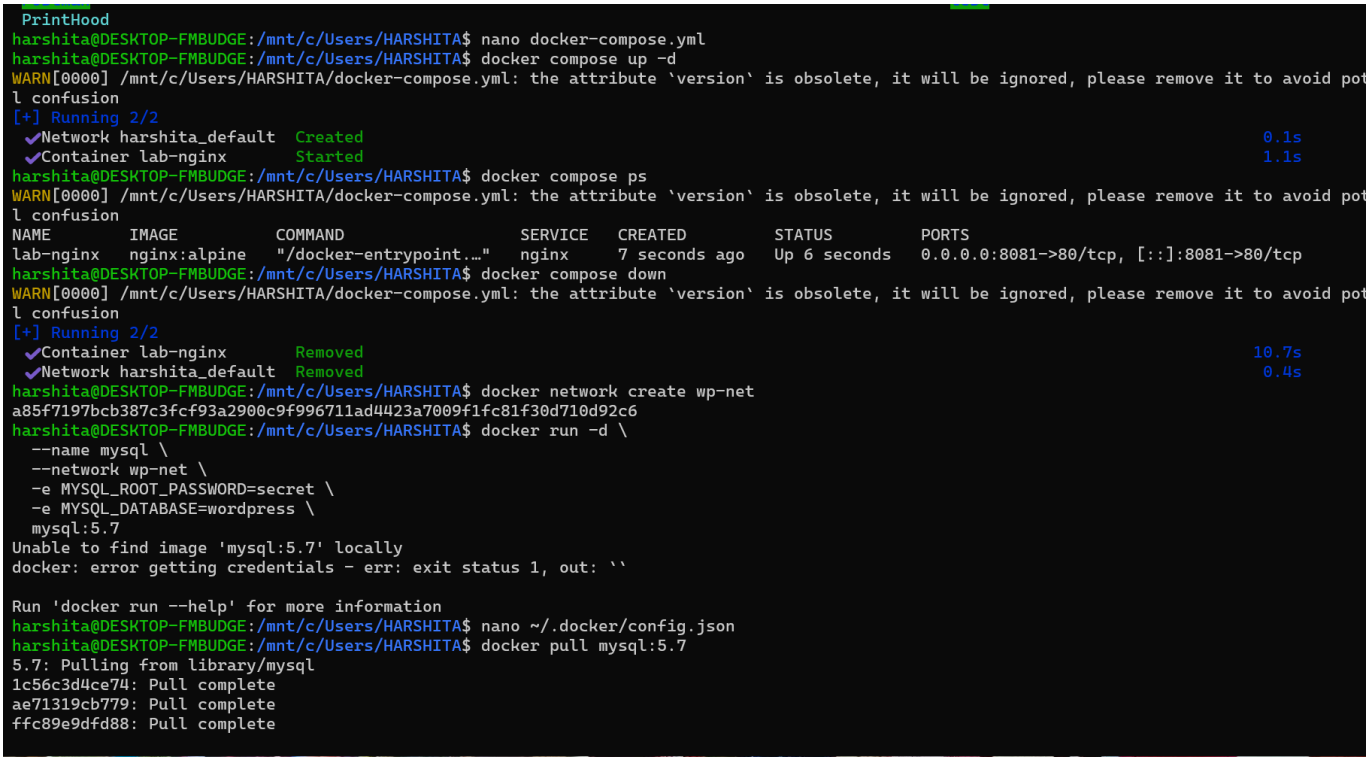
Step 3: Run WordPress

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

Test

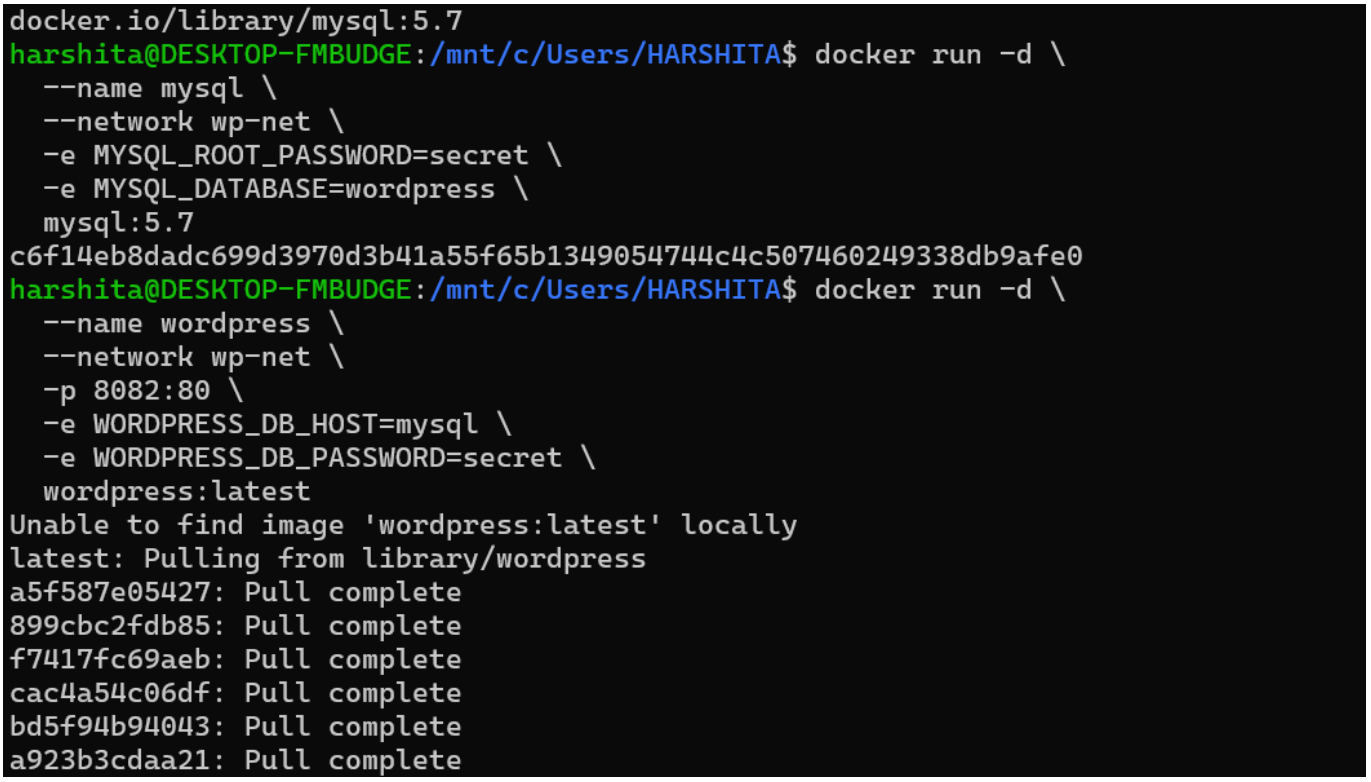
http://localhost:8082
```

📸 Screenshot:

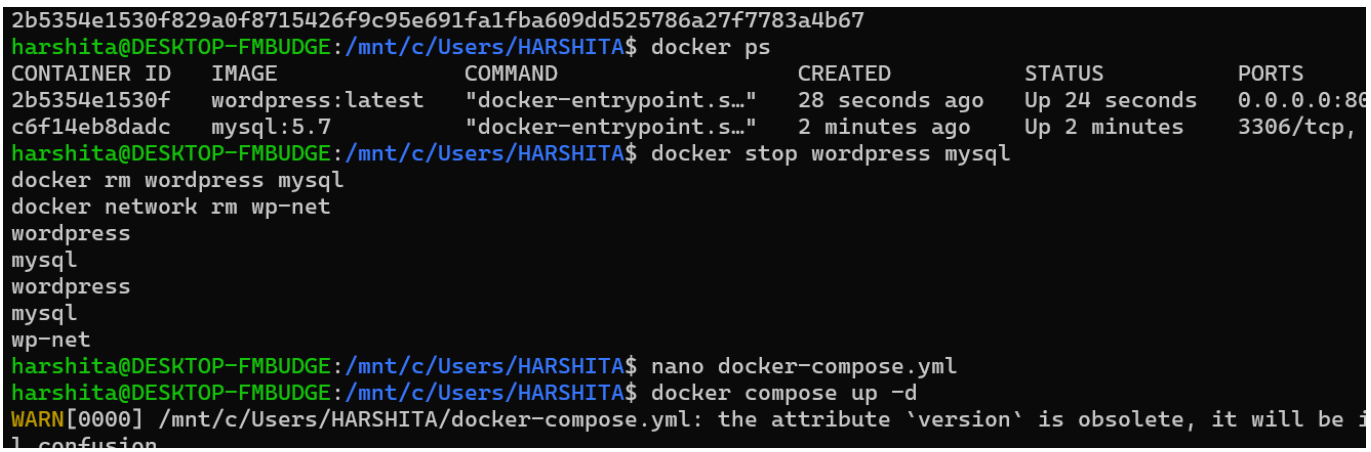


```
 PrintHood
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
 ✔Network harshita_default  Created                                                                                    0.1s
 ✔Container lab-nginx       Started                                                                                    1.1s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
NAME        IMAGE          COMMAND                  SERVICE   CREATED         STATUS         PORTS
lab-nginx   nginx:alpine   "/docker-entrypoint.…"   nginx     7 seconds ago   Up 6 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose down
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
 ✔Container lab-nginx       Removed                                                                                   10.7s
 ✔Network harshita_default  Removed                                                                                    0.4s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker network create wp-net
a85f7197bcb387c3fcf93a2900c9f996711ad4423a7009f1fc81f30d710d92c6
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
Unable to find image 'mysql:5.7' locally
docker: error getting credentials - err: exit status 1, out: ``

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano ~/.docker/config.json
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker pull mysql:5.7
5.7: Pulling from library/mysql
1c56c3d4ce74: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
```



```
docker.io/library/mysql:5.7
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
c6f14eb8dadc699d3970d3b41a55f65b1349054744c4c507460249338db9afe0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
a5f587e05427: Pull complete
899cbc2fdb85: Pull complete
f7417fc69aeb: Pull complete
cac4a54c06df: Pull complete
bd5f94b94043: Pull complete
a923b3cdaa21: Pull complete
```



```
2b5354e1530f829a0f8715426f9c95e691fa1fba609dd525786a27f7783a4b67
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS
2b5354e1530f   wordpress:latest   "docker-entrypoint.s…"   28 seconds ago   Up 24 seconds   0.0.0.0:80
c6f14eb8dadc   mysql:5.7          "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes    3306/tcp,
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker stop wordpress mysql
docker rm wordpress mysql
docker network rm wp-net
wordpress
mysql
wordpress
mysql
wp-net
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be i
l confusion
```

B. Using Docker Compose

Create `docker-compose.yml`

```
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

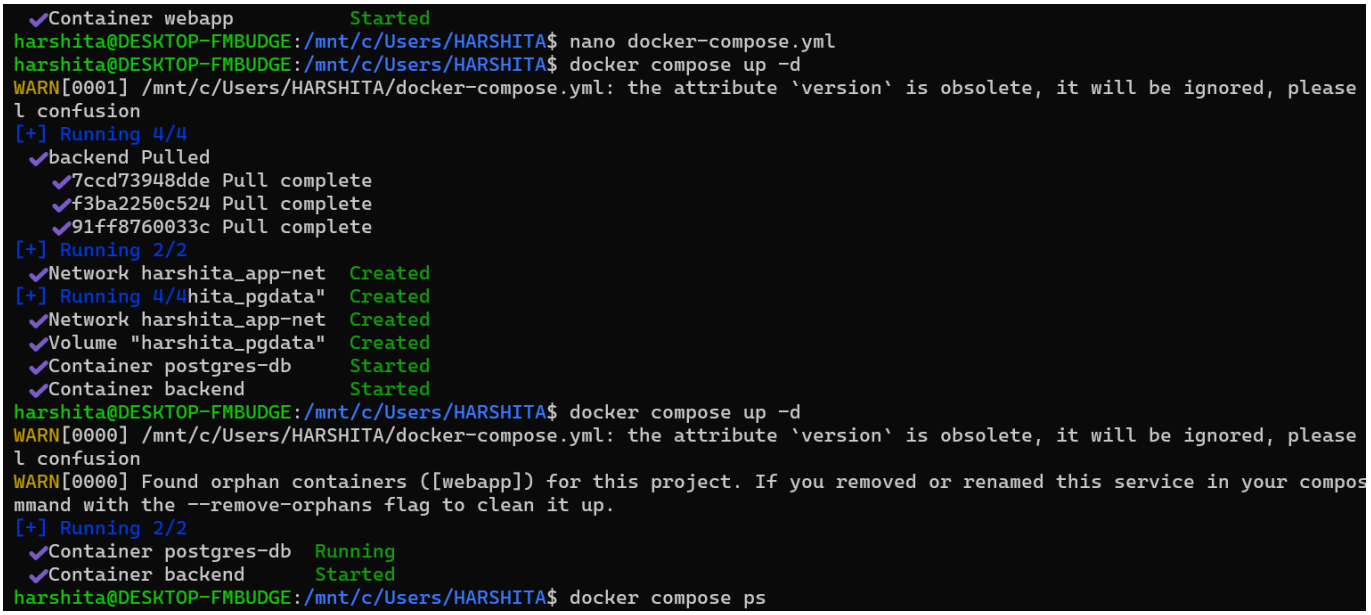
Run

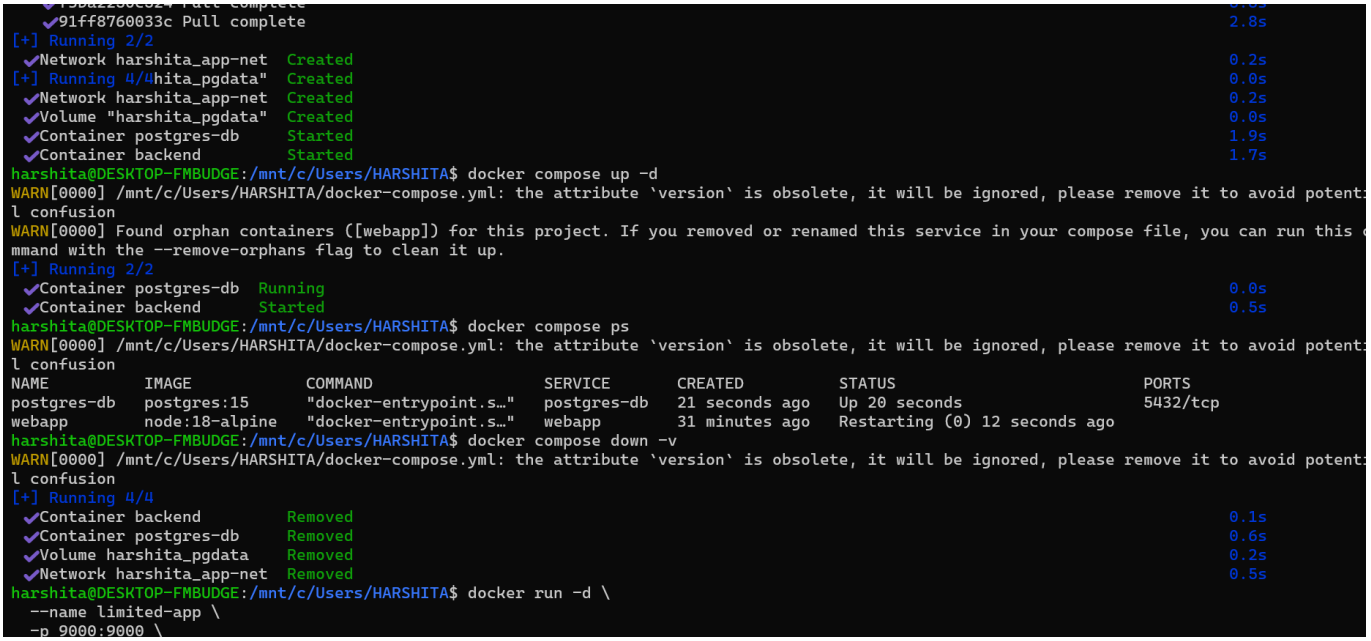
```
docker compose up -d
```

Stop

```
docker compose down -v
```

📸 Screenshot:





PART C - CONVERSION & BUILD-BASED TASKS

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application

Given Docker Run Command

```
docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
```

```
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
```

Docker Compose Equivalent

```
version: '3.8'

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

Run & Verify

```
docker compose up -d
docker compose ps
```

📸 Screenshot:

```
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql
```

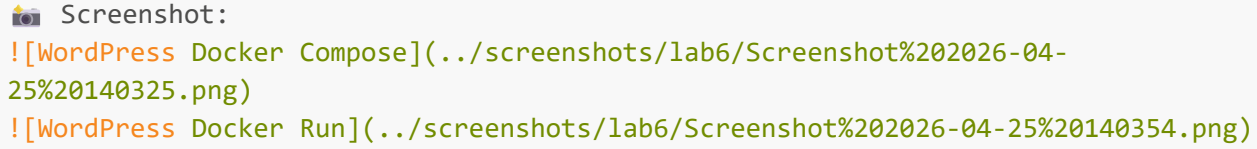
```
volumes:
  mysql_data:
```

Run

```bash
docker compose up -d
```

Stop

```bash
docker compose down -v
```

📸 Screenshot:


PART C - CONVERSION & BUILD-BASED TASKS

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application

Given Docker Run Command

```bash
docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
```

Docker Compose Equivalent

```yaml
version: '3.8'

services:
  webapp:
```

```
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

Run & Verify

```bash
docker compose up -d
docker compose ps
```

📸 Screenshot:
`![WebApp Compose](../screenshots/lab6/Screenshot%202026-04-25%20140405.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140412.png)

Problem 2: Volume + Network Configuration

Docker Compose File

```yaml
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
```

```
      - postgres-db
    networks:
      - app-net

volumes:
  pgdata:

networks:
  app-net:
```

Run

```bash
docker compose up -d
```

Stop

```bash
docker compose down -v
```

📸 Screenshot:
`![Backend + DB](../screenshots/lab6/Screenshot%202026-04-25%20140422.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140429.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140238.png)

Task 4: Resource Limits Conversion

Docker Compose

```yaml
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
```

Explanation

````
* ```yaml
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
```

#### Explanation

* `deploy` works only in Docker Swarm mode
* In normal Compose → ignored
* In Swarm → enforced

📸 Screenshot:
`![Resource Limits](../screenshots/lab6/Screenshot%202026-04-25%20213901.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214338.png)


---

# PART D - USING DOCKERFILE

---

## Task 5: Replace Standard Image with Dockerfile

### Step 1: Create `app.js`

```javascript
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

---

### Step 2: Create Dockerfile

```dockerfile
FROM node:18-alpine
````

````
WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

---

### Step 3: Create docker-compose.yml

```yaml
version: '3.8'

services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

---

#### Run

```bash
docker compose up --build -d
```

#### Verify

http://localhost:3000

📸 Screenshot:
`![Node App](../screenshots/lab6/Screenshot%202026-04-25%20214106.png)

---

### Difference: image vs build

| Feature       | image      | build            |
| ------------- | ---------- | ---------------- |
| Source        | Docker Hub | Local Dockerfile |
| Control       | Low        | High             |
| Customization | No         | Yes              |

---

## Task 6: Multi-Stage Dockerfile
````
````

Requirements

* Multi-stage build
* Smaller image
* Use Compose
* Add environment variables
* Add volume mount

Verify Image Size

```bash
docker images
```

📸 Screenshot:

Experiment 6 B

Multi-Container Application (WordPress + MySQL)

Objective

* Deploy WordPress + MySQL
* Understand networking & volumes
* Learn scaling
* Compare with Swarm

Architecture

```
User → WordPress → MySQL → Volume
```

Step 1: Create Directory

```bash
mkdir wp-compose-lab
cd wp-compose-lab
```

Step 2: Create docker-compose.yml

```yaml
version: '3.9'
```

```
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Step 3: Start Application

```bash
docker-compose up -d
```

📸 Screenshot:
````bash
docker images
```
📸 Screenshot:
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140437.png)
---

# Experiment 6 B

## Multi-Container Application (WordPress + MySQL)

````

Objective

* Deploy WordPress + MySQL
* Understand networking & volumes
* Learn scaling
* Compare with Swarm

Architecture

```
User → WordPress → MySQL → Volume
```

Step 1: Create Directory

```bash
mkdir wp-compose-lab
cd wp-compose-lab
```

Step 2: Create docker-compose.yml

```yaml
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

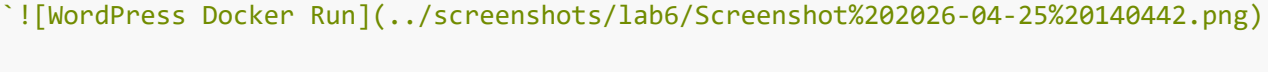
  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
````
````

```
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Step 3: Start Application

```bash
docker-compose up -d
```

📸 Screenshot:


Step 4: Verify

```bash
docker ps
```

Step 5: Access WordPress

http://localhost:8080

📸 Screenshot:


Step 6: Check Volumes

```bash
docker volume ls
```

Step 7: Stop

```bash
docker-compose down
```

Scaling in Docker Compose

```bash
docker-compose up --scale wordpress=3
```

Issues

* Port conflict
* No load balancing

Solution

Use Nginx Reverse Proxy

Docker Swarm

Initialize

```bash
docker swarm init
```

Deploy Stack

```bash
docker stack deploy -c docker-compose.yml wpstack
```

Scale

```bash
docker service scale wpstack_wordpress=3
```

📸 Screenshot:
`![Swarm](../screenshots/lab6/Screenshot%202026-04-25%20214439.png)
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140437.png)
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214707.png)
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214721.png)

Benefits of Docker Swarm

* Load balancing

```
* Auto-healing
* Scaling
* Rolling updates

---

## Limitations

* Less popular than Kubernetes
* Limited ecosystem

---

## Key Learning Outcomes

* Multi-container apps need orchestration
* Docker Compose is ideal for:

  * Development
  * Testing
  * Learning

---
```

Experiment 7

CI/CD using Jenkins, GitHub and Docker Hub

Name: Harshita Chabaria

Subject: Containerization and DevOps

Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application and Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build and push process using webhook triggers
- Use same host (Docker) as Jenkins agent

Theory

1. What is Jenkins?

Jenkins is a web-based GUI automation server used to build, test, and deploy applications. It supports plugins (GitHub, Docker, etc.) and uses **Pipeline as Code (Jenkinsfile)**.

2. What is CI/CD?

- **Continuous Integration (CI):** Automatically builds and tests code after each commit
- **Continuous Deployment (CD):** Automatically delivers built artifacts (Docker images)

3. Workflow Overview

```
Developer → GitHub → Webhook → Jenkins → Build → Docker Hub
```

4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux knowledge

Part A: GitHub Repository Setup

Project Structure

```
My-App/
├── app.py
├── requirements.txt
├── Dockerfile
└── Jenkinsfile
```

Application Code

app.py

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.10-slim
```

```
WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80

CMD ["python", "app.py"]
```

Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "har4049/myapp"
    }

    stages {

        stage('Clone Source') {
            steps {
                git branch: 'main', url:
'https://github.com/harshitachabaria24/My-App'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token',
variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u jasman1201 --
password-stdin'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}
```

Part B: Jenkins Setup using Docker

docker-compose.yml

```
version: '3.8'

services:
  jenkins:
```

```
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Steps

Run Jenkins

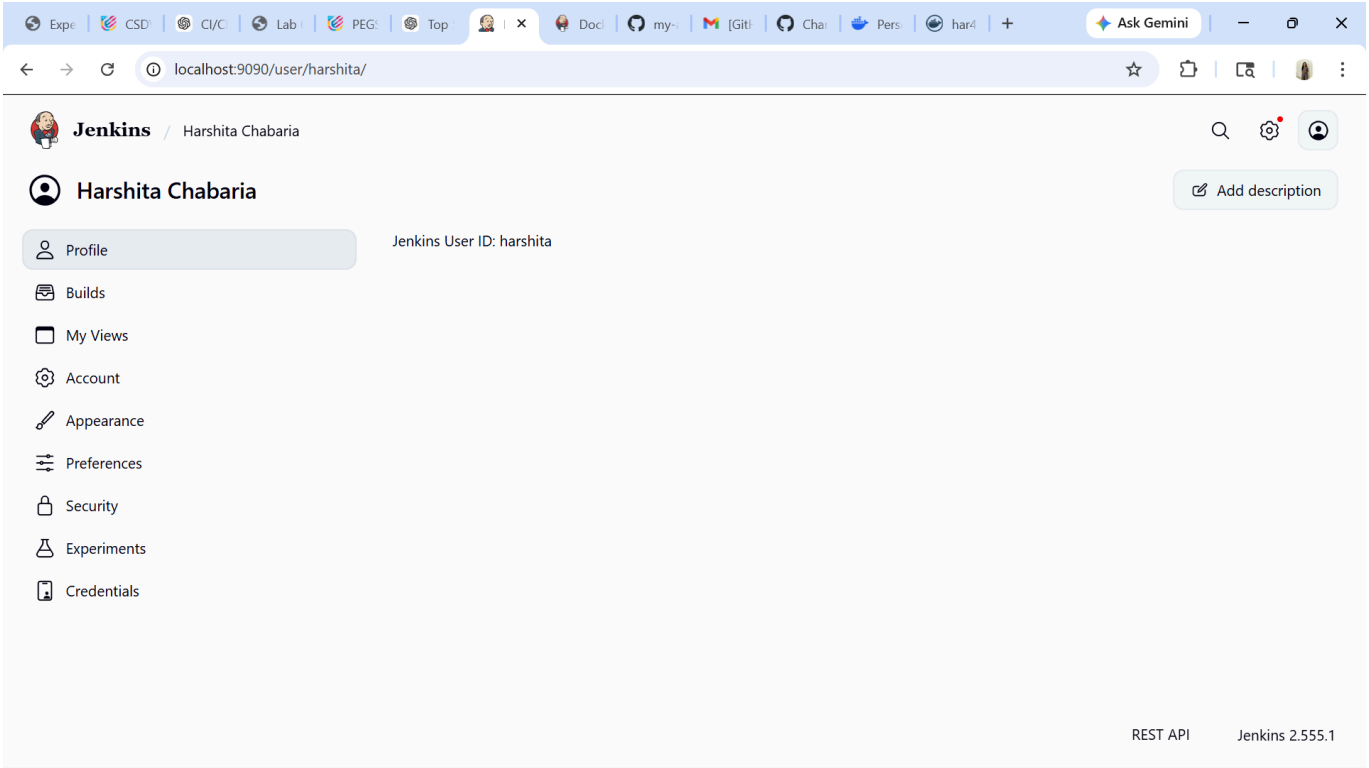
```
docker compose up -d
```

📸 Screenshot: `

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  -p 9090:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
b0d0a7a02241de0d0c50ba4cfb6767c175851621d12d8a5feef04a7ba90f9d00
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
```

Access Jenkins

http://localhost:8080 ![Jenkins Started](../screenshots/lab7/Screenshot%202026-04-25%20225057.png)



Get Admin Password

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Install Plugins

- Click **Install Suggested Plugins**

Install Docker inside Jenkins

```
apt-get update && apt-get install -y docker.io
```

📸 Screenshot:

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Install Plugins

* Click **Install Suggested Plugins**

Install Docker inside Jenkins

```bash id="dockerinstall"
apt-get update && apt-get install -y docker.io
```

📸 Screenshot:
`![Docker Installed](../screenshots/lab7/Screenshot%20(693).png)

Verify Docker

```bash id="dockerps"
docker ps
```

📸 Screenshot:
`![Docker PS](../screenshots/lab7/Screenshot%20(695).png)
`![Console Output](../screenshots/lab7/Screenshot%20(694).png)

Part C: Jenkins Configuration

Add Docker Hub Credentials

Path:
Manage Jenkins → Credentials → Global → Add Credentials

📸 Screenshot:
`![Credentials](../screenshots/lab7/Screenshot%20(688).png)

Configure Pipeline Job

* New Item → Pipeline
* Pipeline script from SCM
* Git Repo URL
* Branch: main
* Script Path: Jenkinsfile

📸 Screenshot:
`![Pipeline Config](../screenshots/lab7/Screenshot%20(690).png)

Pipeline Execution

📸 Screenshot:
`![Console Output](../screenshots/lab7/Screenshot%20(687).png)

````
`![Console Output](../screenshots/lab7/Screenshot%20(697).png)

---

## Execution Flow

| Stage              | Action                | Status    |
| ------------------ | --------------------- | --------- |
| Clone Source       | Pull code from GitHub | ✅ Success |
| Build Docker Image | Build image           | ✅ Success |
| Login Docker Hub   | Authenticate          | ✅ Success |
| Push Image         | Push to Docker Hub    | ✅ Success |

---

## Role of Same Host Agent

Jenkins runs inside Docker with socket mounted:

```id="socket"
/var/run/docker.sock → /var/run/docker.sock
```

➡️ This allows Jenkins to control host Docker directly

---

# Observations

* Jenkins GUI simplifies CI/CD
* GitHub stores code + pipeline
* Docker ensures consistency
* Credentials stored securely
* Docker socket enables direct control

---

# Result

Successfully implemented CI/CD pipeline where:

* Code + Jenkinsfile in GitHub
* Jenkins auto-builds on changes
* Docker image built
* Image pushed to Docker Hub securely

---
````

Experiment 9

Ansible

Theory

Problem Statement

Managing infrastructure manually across multiple servers leads to:

- Configuration drift
- Inconsistent environments
- Time-consuming repetitive tasks

Scaling from one server to hundreds becomes difficult using manual SSH-based administration.

What is Ansible?

- Open-source automation tool for configuration management, deployment, and orchestration
- Agentless architecture (uses SSH for Linux, WinRM for Windows)
- Uses YAML-based playbooks
- Industry-standard automation tool

How Ansible Solves the Problem

- **Agentless** → No installation on target machines
- **Idempotent** → Same result every time
- **Declarative** → Define desired state
- **Push-based** → Control node sends instructions

Key Concepts

Component	Description
Control Node	Machine with Ansible installed
Managed Nodes	Target servers
Inventory	List of servers
Playbooks	YAML automation files
Tasks	Individual actions
Modules	Built-in functions

How Ansible Works

- Control node connects to managed nodes via SSH
- Executes modules through tasks
- Tasks combine into playbooks
- Inventory defines servers

Benefits

- Free & open-source
- Easy to learn
- No agents required
- Reusable and modular

Part A: Installation

Install using pip

```
pip install ansible
ansible --version
```

📸 Screenshot: `

```
harshita@DESKTOP-FMBUDGE:~/ansible-docker$ sudo apt update
[sudo] password for harshita:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1620 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1919 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [345 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.1 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1685 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [259 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:14 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7356 B]
Get:15 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.9 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.0 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1182 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Fetched 8095 kB in 8s (965 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
harshita@DESKTOP-FMBUDGE:~/ansible-docker$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Install using apt

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```

📸 Screenshot:

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```

📸 Screenshot:

Test Installation

```bash id="pingtest"
ansible localhost -m ping
```

Expected Output

```bash id="expected"
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

📸 Screenshot:

Part B: Docker + SSH Setup

Step 1: Create SSH Key Pair

```bash id="sshkey"
ssh-keygen -t rsa -b 4096
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

📸 Screenshot:
````bash id="sshkey"
ssh-keygen -t rsa -b 4096
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

📸 Screenshot:
`![SSH Key](../screenshots/lab9/Screenshot%202026-04-26%20012738.png)

---

## Step 2: Create Dockerfile

```dockerfile id="dockerfile"
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
/etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/'
/etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/'
/etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\\s*required\\s*pam_loginuid.so@session optional
pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

📸 Screenshot:
`![Dockerfile](../screenshots/lab9/Screenshot%202026-04-26%20012753.png)

---

## Step 3: Build Image

```bash id="buildimg"
```

````
docker build -t ubuntu-server .
```

📸 Screenshot:
````bash id="buildimg"
docker build -t ubuntu-server .
```

📸 Screenshot:
`![Docker Build](../screenshots/lab9/Screenshot%202026-04-26%20012753.png)

---

## Step 4: Run Container

```bash id="runcontainer"
docker run -d -p 2222:22 --name ssh-test-server ubuntu-server
```

📸 Screenshot:
`![Docker Run](../screenshots/lab9/Screenshot%202026-04-26%20012812.png)

---

## Step 5: Get Container IP

```bash id="getip"
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-
test-server
```

📸 Screenshot:
````bash id="getip"
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-
test-server
```

📸 Screenshot:
`![Container IP](../screenshots/lab9/Screenshot%202026-04-26%20012827.png)

---

## Step 6: Test SSH

### Password Login

```bash id="sshpass"
ssh root@localhost -p 2222
```

### Key-based Login

```bash id="sshkeylogin"
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
````

```
```

📸 Screenshot:
`![SSH Login](../screenshots/lab9/Screenshot%202026-04-26%20012827.png)

Cleanup

```bash id="cleanup"
docker stop ssh-test-server
docker rm ssh-test-server
```
📸 Screenshot:
````bash id="cleanup"
docker stop ssh-test-server
docker rm ssh-test-server
```
📸 Screenshot:
`![SSH Login](../screenshots/lab9/Screenshot%202026-04-26%20012904.png)


---

# Part C: Ansible with Docker

---

## Step 1: Start Multiple Containers

```bash id="multicontainer"
for i in {1..4}; do
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
done
```

📸 Screenshot:
`![Multiple Servers](../screenshots/lab9/Screenshot%202026-04-30%20224342.png)

---

## Stop All Servers

```bash id="stopservers"
for i in {1..4}; do
  docker stop server${i}
done
```

---

## Step 2: Create Inventory

```bash id="inventory"
```

```
echo "[servers]" > inventory.ini

for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
server${i} >> inventory.ini
done

cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

---

## Step 3: View Inventory

```bash id="viewinventory"
cat inventory.ini
```

---

## Step 4: Test Connectivity

```bash id="ansibleping"
ansible all -i inventory.ini -m ping
```
📸 Screenshot:
````bash id="stopservers"
for i in {1..4}; do
  docker stop server${i}
done
```

---

## Step 2: Create Inventory

```bash id="inventory"
echo "[servers]" > inventory.ini

for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
server${i} >> inventory.ini
done

cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
```

```
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

---

## Step 3: View Inventory

```bash id="viewinventory"
cat inventory.ini
```

---

## Step 4: Test Connectivity

```bash id="ansibleping"
ansible all -i inventory.ini -m ping
```
📸 Screenshot:


---

## Step 5: Create Playbook

```yaml id="playbook"
---
- name: Update servers
  hosts: all
  become: yes

  tasks:
    - name: Update packages
      apt:
        update_cache: yes
```

---

## Step 6: Run Playbook

```bash id="runplaybook"
ansible-playbook -i inventory.ini playbook1.yml
```
📸 Screenshot:


---

## Step 7: Verify

```bash id="verify"
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

📸 Screenshot:
````bash id="verify"
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

📸 Screenshot:
`![Verification](../screenshots/lab9/Screenshot%202026-04-30%20224557.png)

---

# Conclusion

Ansible simplifies server management by automating tasks and ensuring consistency across multiple systems.

---

# Key Takeaways

* Ansible is agentless
* Uses SSH
* Playbooks automate tasks
* Docker simulates servers

---

# Experiment 10

## Working with SonarQube

# Introduction

## What is SonarQube?

SonarQube is an open-source platform for continuous inspection of code quality. It performs static code analysis to detect:

- Bugs
- Code smells
- Security vulnerabilities

## How SonarQube Solves the Problem

- **Continuous Inspection** → Scans code on every commit
- **Quality Gates** → Defines pass/fail criteria
- **Technical Debt** → Measures effort to fix issues
- **Multi-language Support** → Supports 20+ languages
- **Visual Dashboard** → Displays code metrics

## Key Terms

| Term | Meaning |
|---|---|
| Quality Gate | Criteria to pass before deployment |
| Bug | Code that may break |
| Vulnerability | Security issue |
| Code Smell | Poorly written code |
| Technical Debt | Time required to fix issues |
| Coverage | % of tested code |
| Duplication | Repeated code |

# Practical

## Step 1: Create `docker-compose.yml`

```yaml
version: '3.8'

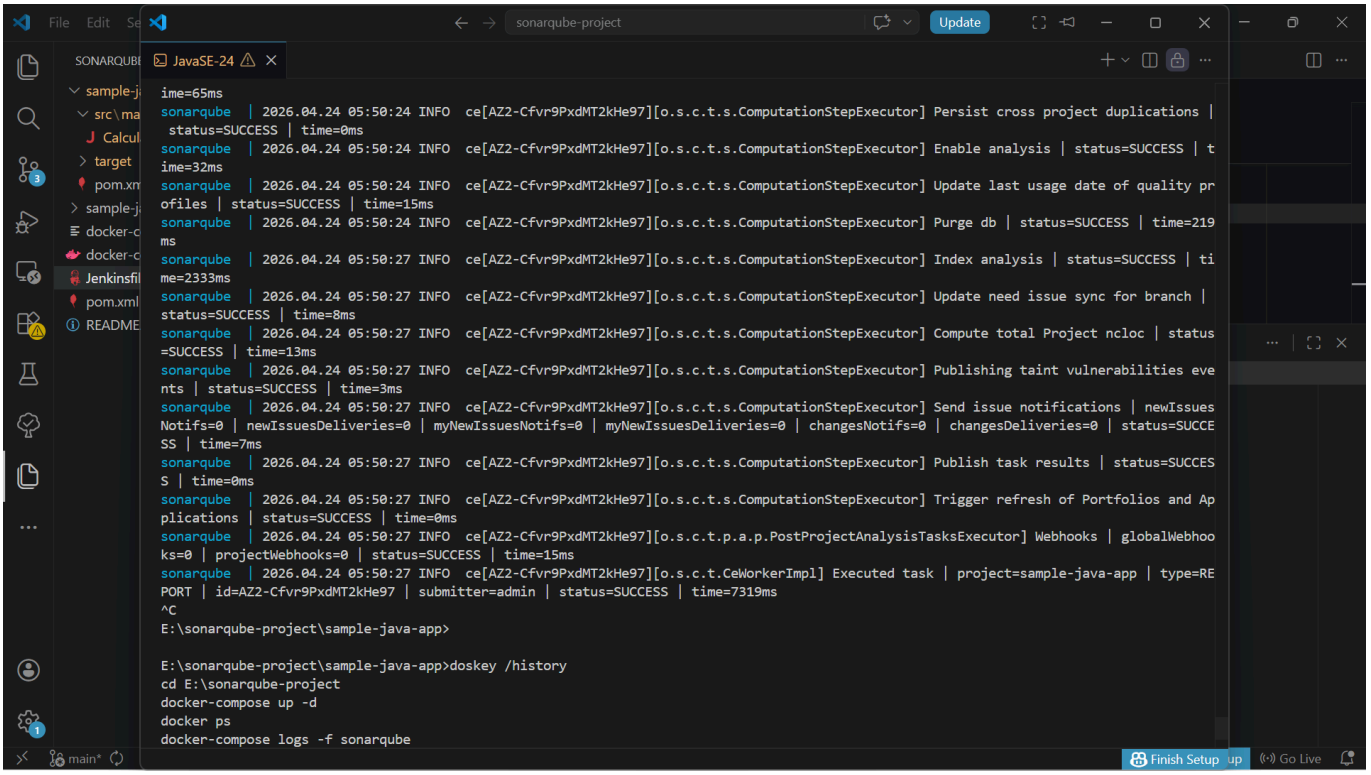
services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge
```

📸 Screenshot: `




---

# Step 2: Start SonarQube

```
docker-compose up -d
```

📸 Screenshot:

```
docker-compose up -d
```

📸 Screenshot:
`![Docker Compose Up](../screenshots/lab10/Screenshot%20(710).png)

---

## Step 3: Verify Using Logs

```bash id="logs"
docker-compose logs -f sonarqube
```

---

## Step 4: Login to SonarQube

👉 Open in browser:
http://localhost:9000

* Username: `admin`
* Password: `admin`

📸 Screenshot:
`![Login](../screenshots/lab10/Screenshot%202026-04-30%20232121.png)

---

## Step 5: Home Page

📸 Screenshot:
`![Homepage](../screenshots/lab10/Screenshot%202026-04-30%20232136.png)

---

## Step 6: Create Sample Java App

```bash id="mkdir"
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

## Step 7: Create `Calculator.java`

```java id="java"
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
```

```
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public String getName(String name) {
        return name.toUpperCase();
    }

    public void riskyOperation() {
        try {
            int x = 10 / 0;
        } catch (Exception e) {
        }
    }
}
```

📸 Screenshot:
`![Java File](../screenshots/lab10/Screenshot%202026-04-30%20232232.png)

---

## Step 8: Create `pom.xml`

```xml id="pom"
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN_HERE</sonar.login>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
```

```
        <artifactId>sonar-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

## Step 9: Generate Token

* Go to: **Profile → My Account → Security**
* Create token: ````xml id="pom"
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN_HERE</sonar.login>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

## Step 9: Generate Token

* Go to: **Profile → My Account → Security**
* Create token: `scanner-token`
---

## Step 10: Run Scanner
```

```bash id="scanner"
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

📸 Screenshot:
````bash id="scanner"
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

📸 Screenshot:
`![Scanner Run](../screenshots/lab10/Screenshot%20(707).png)

---

## Step 11: Verify Build Success

📸 Screenshot:
`![Build Success](../screenshots/lab10/Screenshot%202026-04-30%20232343.png)

---

## Step 12: View Dashboard

👉 Open:
http://localhost:9000/dashboard?id=sample-java-app

📸 Screenshot:
`![Dashboard](../screenshots/lab10/Screenshot%20(706).png)
📸 Screenshot:
`![Dashboard](../screenshots/lab10/Screenshot%202026-04-20%20114053.png)
📸 Screenshot:
`![Report](../screenshots/lab10/Screenshot%202026-04-20%20103310.png)
📸 Screenshot:
`![Report](../screenshots/lab10/Screenshot%202026-04-18%20151951.png)
---

## Step 13: View Detailed Report

* Click issues
* View exact line and reason

📸 Screenshot:
`![Report](../screenshots/lab10/Screenshot%202026-04-18%20151930.png)

---

# Conclusion

SonarQube helps in maintaining code quality by detecting bugs, vulnerabilities, and improving maintainability through continuous inspection.

---

# Key Takeaways

```
* Continuous code inspection
* Improves code quality
* Detects bugs & vulnerabilities
* Provides visual reports
* Integrates with CI/CD pipelines

---
```

# Experiment 11

## Container Orchestration with Docker Stack

# Objective

To understand and implement container orchestration using Docker Stack and Docker Swarm for deploying and managing multi-container applications across a cluster.

# Theory

## Docker Stack

Docker Stack is a collection of services that can be deployed together using a `docker-compose.yml` file (version 3+). It enables orchestration and scaling on Docker Swarm.

## Docker Swarm

Docker Swarm is Docker's native clustering tool that turns multiple Docker hosts into a single virtual system.

### Components

- **Manager Node** → Manages cluster and scheduling
- **Worker Node** → Executes tasks
- **Service** → Definition of containers
- **Task** → Running container instance

## Key Concepts

| Concept | Description |
|---|---|
| Stack | Group of services |
| Service | Container definition |
| Replica | Number of instances |
| Overlay Network | Communication between nodes |
| Volume | Persistent storage |

# Prerequisites

- Docker installed
- Docker Compose v3+
- Basic Docker knowledge
- Terminal access

---

# Environment Setup

## Check Docker Version

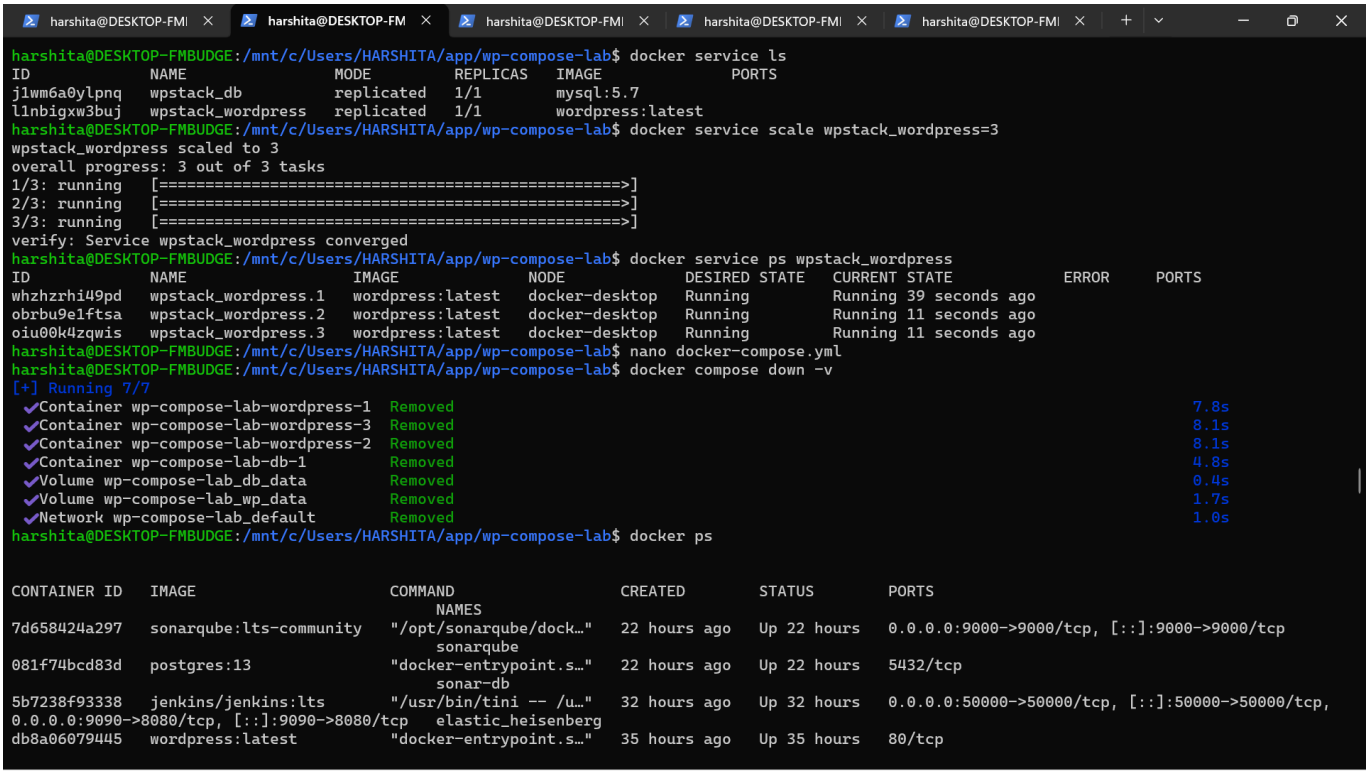
```
docker --version
docker-compose --version
```

---

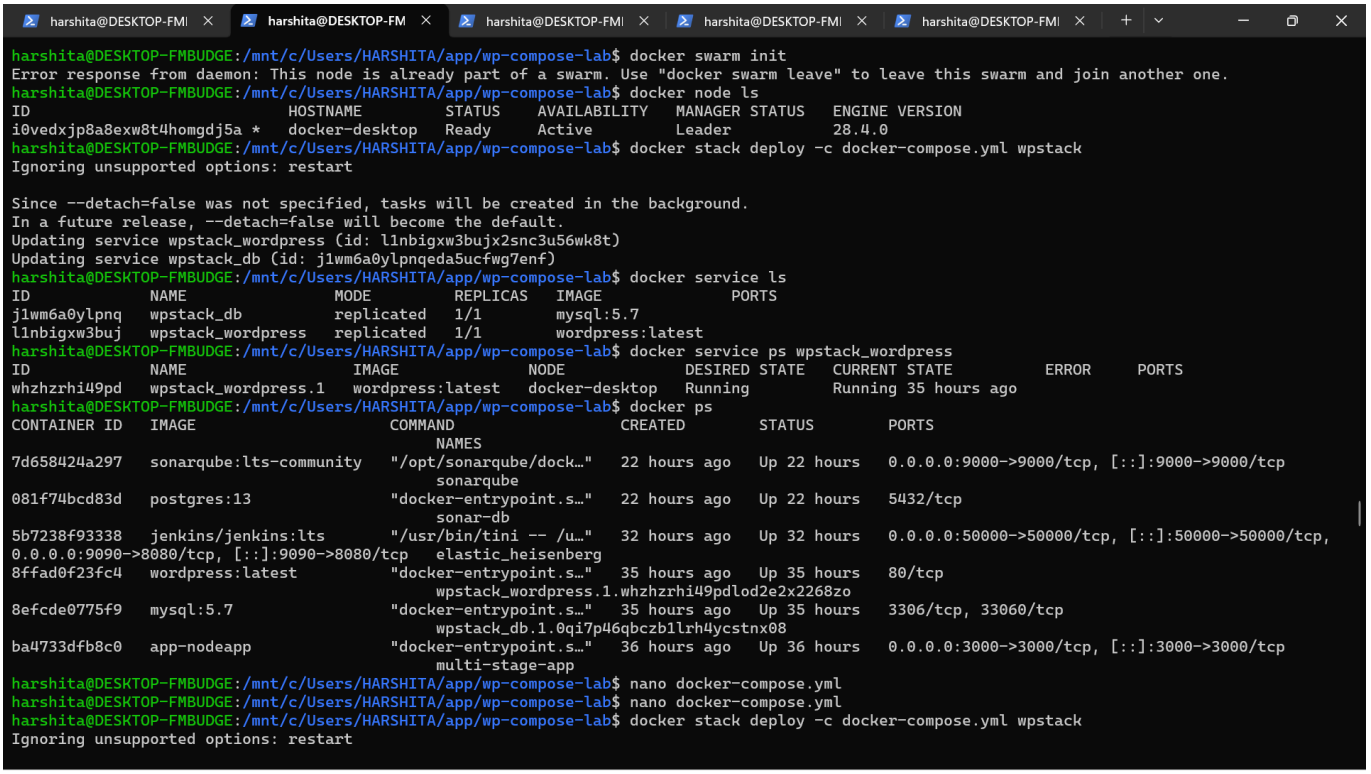
## Initialize Swarm

```
docker swarm init
```

Screenshot: `



`

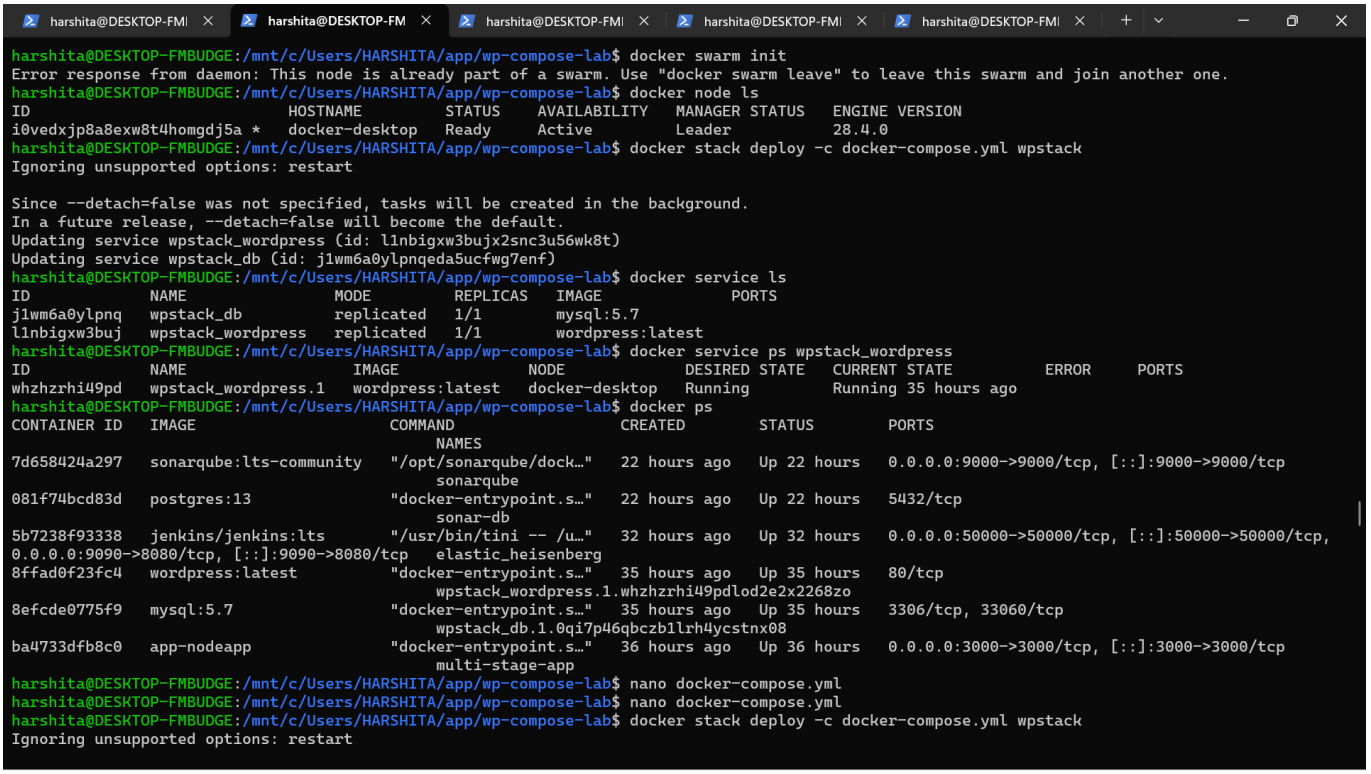



---

# Verify Swarm

```
docker info | grep -i swarm
docker node ls
```

📸 Screenshot: `

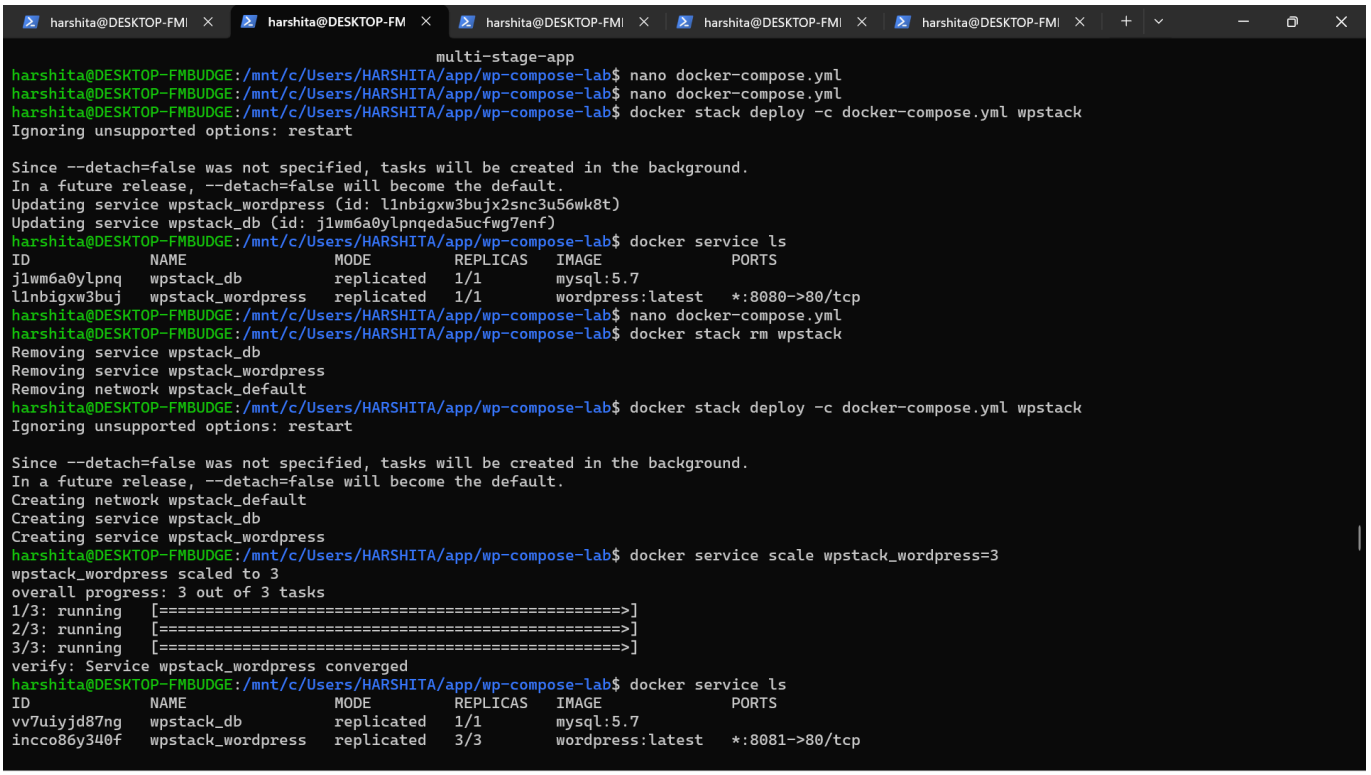



---

# Experiment Steps

## Step 0: docker-compose.yml

📸 Screenshot: `




## Step 1: Stop Existing Containers

```
docker compose down
docker ps
```

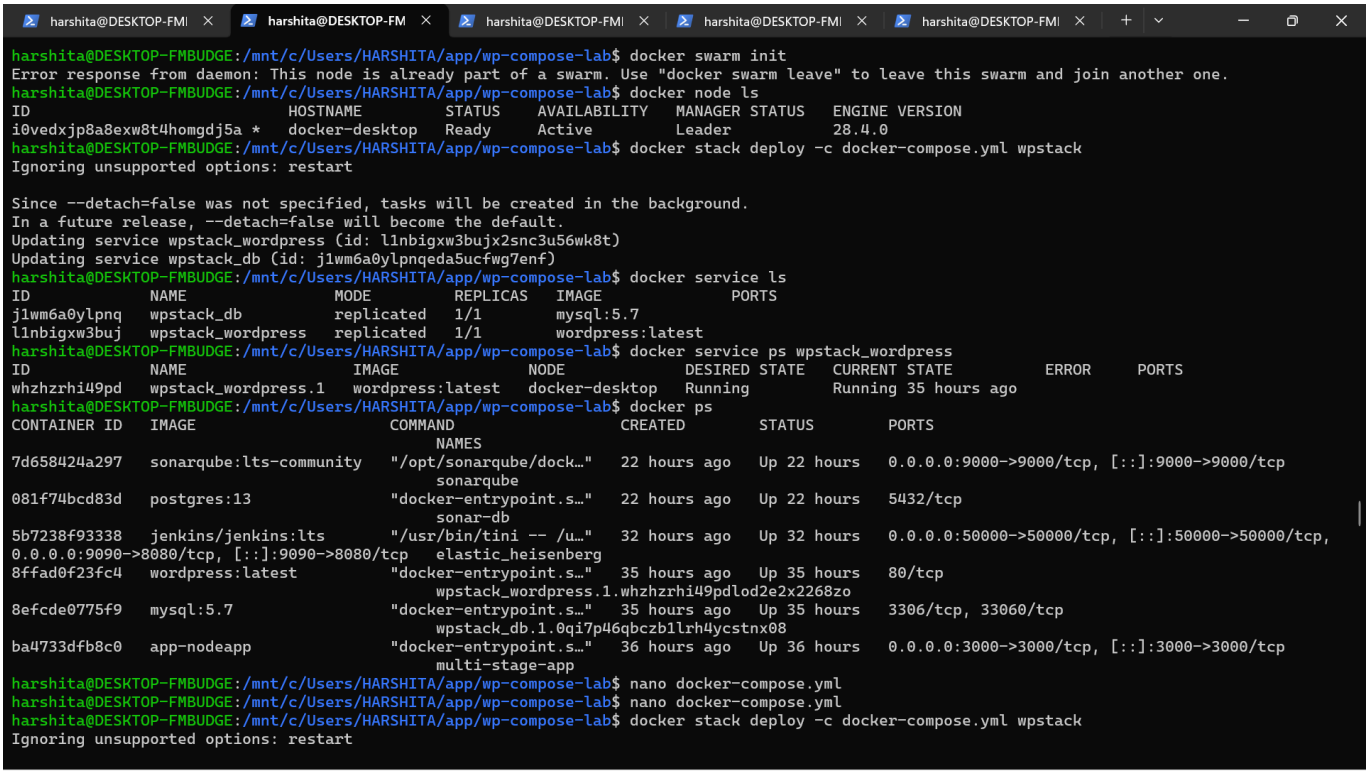
## Step 2: Initialize Swarm

```
docker swarm init
```

## Step 3: Verify Nodes

```
docker node ls
```

📸 Screenshot: `




---

# Step 4: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

📸 Screenshot:

````
docker stack deploy -c docker-compose.yml wpstack
```

📸 Screenshot:
`![Stack Deploy](../screenshots/lab11/Screenshot%20(750).png)

Step 5: List Services

```bash id="services"
docker service ls
```

📸 Screenshot:
`![Service List](../screenshots/lab11/Screenshot%20(750).png)

Step 6: Inspect Service Tasks
````
````

```bash id="tasks"
docker service ps wpstack_wordpress
```

📸 Screenshot:
````bash id="tasks"
docker service ps wpstack_wordpress
```

📸 Screenshot:
`![Service Tasks](../screenshots/lab11/Screenshot%20(751).png)

---

## Step 7: Check Running Containers

```bash id="containers"
docker ps
```
---

## Step 8: Scale Service

```bash id="scale"
docker service scale wpstack_wordpress=3
```

---

## Step 9: Filter WordPress Containers

```bash id="filter"
docker ps | grep wordpress
```

📸 Screenshot:
`![Filtered Containers](../screenshots/lab11/Screenshot%20(751).png)

---

## Step 10: Remove Stack

```bash id="remove"
docker stack rm wpstack
```

📸 Screenshot:
````bash id="remove"
docker stack rm wpstack
```

📸 Screenshot:
`![Stack Remove](../screenshots/lab11/Screenshot%20(752).png)

```
---

# Observations

| Parameter              | Observed Value |
| ---------------------- | -------------- |
| Stack Name             | wpstack        |
| Services Deployed      | ___            |
| Initial Replicas       | ___            |
| Replicas After Scaling | 3              |
| WordPress Port         | ___            |
| MySQL Service Name     | ___            |
| Overlay Network        | ___            |

---

# Expected Output

* `docker stack ls` → shows stack
* `docker service ls` → shows services
* WordPress accessible on browser

---

# Docker Stack vs Docker Compose

| Feature        | Docker Compose | Docker Stack |
| -------------- | -------------- | ------------ |
| Scope          | Single host    | Multi-node   |
| Swarm Required | No             | Yes          |
| Scaling        | Manual         | Declarative  |
| Load Balancing | No             | Yes          |
| Updates        | No             | Rolling      |

---

# Common Commands

```bash id="commands"
docker stack deploy -c <file> <name>
docker stack ls
docker stack services <name>
docker stack ps <name>
docker stack rm <name>
docker service scale <service>=<replicas>
docker service logs <service>
```

---

# Troubleshooting

| Problem                | Cause              | Solution    |
| ---------------------- | ------------------ | ----------- |
```

```
| Service not starting  | Image missing     | Build image |
| Swarm not initialized | Not in swarm mode | Run init    |
| Port conflict         | Port busy         | Change port |
| Pending replicas      | Resource issue    | Check nodes |

---

# Viva Questions

* Difference between Docker Compose and Stack?
* What is Docker Swarm?
* What is overlay network?
* How scaling works?
* What is service vs task?

---

# Conclusion

* Swarm cluster initialized
* Stack deployed successfully
* Services scaled
* Monitoring performed

Docker Stack provides production-ready orchestration with scaling, load balancing,
and fault tolerance.

---

# References

* Docker Stack Docs
* Docker Swarm Docs
* Compose v3 Docs

---
```

# Experiment 12

## Container Orchestration using Kubernetes

# Objective

To learn Kubernetes basics by deploying a WordPress application using Minikube, including:

- Deployments
- Services
- Scaling Pods
- Self-healing

# Tools & Environment

- Minikube
- kubectl
- Ubuntu 22.04
- Docker

# Theory

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

## Key Concepts

| Term | Description |
|---|---|
| Pod | Smallest unit (contains container) |
| Deployment | Manages replicas of pods |
| Service | Exposes pods |
| NodePort | External access to service |
| ReplicaSet | Maintains pod count |
| kubectl | CLI tool |

# Steps Performed

## Step 1: Create Deployment YAML

```
nano wordpress-deployment.yaml
```

wordpress-deployment.yaml

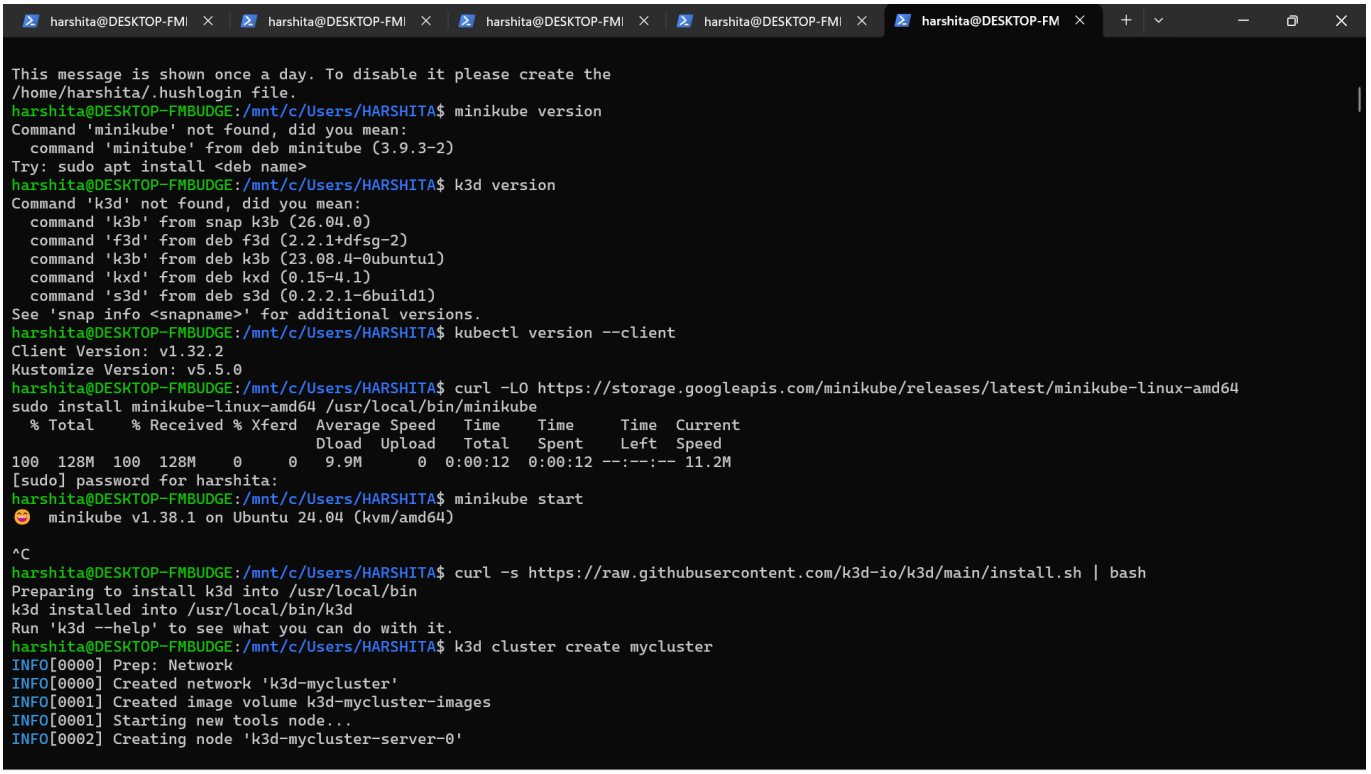
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress

spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress

  template:
    metadata:
      labels:
        app: wordpress

    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

📸 Screenshot: `




---

# Step 2: Start Minikube & Apply Deployment

```
minikube start
kubectl apply -f wordpress-deployment.yaml
```

📸 Screenshot:

```
minikube start
kubectl apply -f wordpress-deployment.yaml
```

📸 Screenshot:
`![Minikube Start](../screenshots/lab12/Screenshot%20(725).png)

---

## Step 3: Create Service YAML

```bash id="nano2"
nano wordpress-service.yaml
```

### wordpress-service.yaml

```yaml id="service"
apiVersion: v1
kind: Service
```

````
metadata:
  name: wordpress-service

spec:
  type: NodePort
  selector:
    app: wordpress

  ports:
  - port: 80
    targetPort: 80
    nodePort: 30007
```

Step 4: Apply Service

```bash id="applysvc"
kubectl apply -f wordpress-service.yaml
```

📸 Screenshot:
`![Service Apply](../screenshots/lab12/Screenshot%20(726).png)

Step 5: Verify Pods

```bash id="pods"
kubectl get pods
```

📸 Screenshot:
````bash id="pods"
kubectl get pods
```

📸 Screenshot:
`![Pods](../screenshots/lab12/Screenshot%20(726).png)

---

## Step 6: Verify Service

```bash id="svc"
kubectl get svc
```

---

## Step 7: Scale Deployment
````
````

```bash id="scale"
kubectl scale deployment wordpress --replicas=4
```

📸 Screenshot:
`![Scaling](../screenshots/lab12/Screenshot%20(726).png)

Step 8: Verify Scaled Pods

```bash id="pods2"
kubectl get pods
```

📸 Screenshot:
````bash id="pods2"
kubectl get pods
```

📸 Screenshot:
`![Scaled Pods](../screenshots/lab12/Screenshot%20(726).png)

---

## Step 9: Delete a Pod (Self-Healing Test)

```bash id="delete"
kubectl delete pod <pod-name>
```

---

## Step 10: Verify Self-Healing

```bash id="verify"
kubectl get pods
```

📸 Screenshot:
`![Self Healing](../screenshots/lab12/Screenshot%20(726).png)

---

# Commands Summary

| Command | Purpose |
| --- | --- |
| minikube start | Start cluster |
| kubectl apply -f file.yaml | Apply config |
| kubectl get pods | List pods |
| kubectl get svc | List services |
| kubectl scale deployment | Scale pods |
| kubectl delete pod | Delete pod |

```
---

# Observations

* Deployment created 2 pods initially
* Service exposed app on NodePort 30007
* Scaling created additional pods instantly
* Self-healing replaced deleted pod automatically

---

# Conclusion

* Kubernetes manages deployments efficiently
* Scaling is automatic and fast
* Self-healing ensures availability
* Infrastructure complexity is abstracted

---
```